

BIG DATA

S.no	Unit & Topics	Page.no.
	UNIT-1 Introduction to Big Data:	1-36
1	Introduction to Big Data	
2	Big Data Characteristics & Types of Data	
3	Traditional Data vs Big Data	
4	Evolution of Big Data	
5	Challenges with Big Data	
6	Technologies Available for Big Data	
7	Infrastructure for Big Data	
8	Uses of Data Analytics	
9	Desired Properties of a Big Data System	
	Unit-2 Introduction to Hadoop:	36-61
10	Introduction to Hadoop	
11	Core Hadoop Components	
12	Hadoop Ecosystem	
13	Hive Physical Architecture	
14	Hadoop Limitations	
15	RDBMS vs Hadoop	
16	Hadoop Distributed File System (HDFS)	
17	Processing Data with Hadoop	
18	Hadoop YARN	
19	MapReduce Programming	
	Unit-3	62-98
20	Introduction to Hive & Data Type of Hive	
21	Hive Query Language (HiveQL)	
22	Introduction to Apache Pig & Anatomy of Apache Pig	
23	Pig on Hadoop & Use Cases for Apache Pig	
24	Introduction to ETL	
25	Data Types in Pig & Execution Model of Pig	

26	Operators ,function & Data Types in Apache Pig	
	Unit-4	99-109
27	Introduction to NoSQL	
28	NoSQL Business Drivers	
29	NoSQL Data Architectural Patterns	
30	Variations of NoSQL Architectural Patterns	
31	Introduction to MongoDB	
	Unit-5 Mining Social Network Graphs:	110-125
32	Introduction to Social Network Mining	
33	Social Networks as a Graph	
34	Types of Social Networks	
35	Clustering of Social Graphs	
36	Introduction to Recommender Systems	

BIG DATA

UNIT-1

☀ Introduction to Big Data

What is Big Data?

Big Data refers to extremely large, fast, and complex datasets that traditional data-processing tools (like regular databases) cannot store, manage, or analyze efficiently.

This data comes from various sources—social media, sensors, transactions, videos, images, etc.—and keeps growing every second.

Examples:

- Social media posts
- Online shopping transactions
- YouTube videos
- Sensor/IoT data
- Satellite images
- Server logs

1. The 5V Characteristics of Big Data

Big Data is commonly defined using 5 V's:

a. Volume

- Represents the huge amount of data generated every day.
- Example: Billions of Facebook posts and photos uploaded daily.

b. Velocity

- Speed at which data is generated, collected, and processed.
- Example: Stock market updates, live location tracking.

c. Variety

- **Different types of data:**
 - Structured: Tables, rows, columns
 - Semi-structured: XML, JSON
 - Unstructured: Videos, audio, text, images

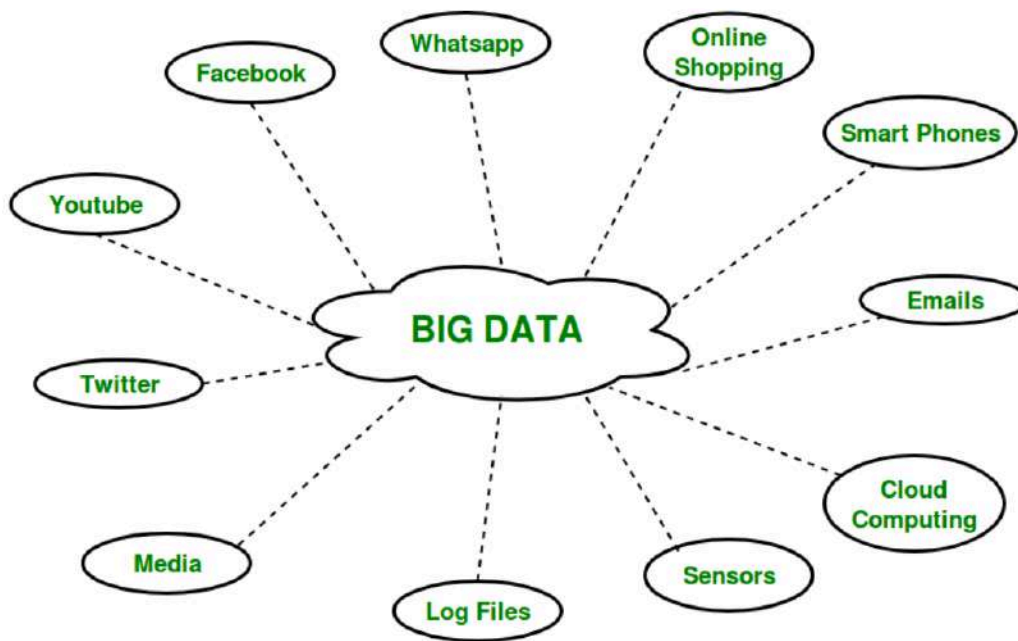
d. Veracity

- Refers to the quality, accuracy, and reliability of data.
- Incorrect data can lead to wrong conclusions.

e. Value

- The most important V: extracting meaningful insights from data to create business value.
- Example: Understanding customer preferences to increase sales.

2. Sources of Big Data



a. Social Media

- Facebook, Instagram, Twitter, YouTube
- Likes, comments, shares, videos

b. Machine/IoT Data

- Sensors, GPS, smart devices, industrial machines

c. Transactional Data

- E-commerce purchases
- Online payments and bank transactions

d. Government & Scientific Data

- Healthcare records
- Weather data, research data

e. Web & Server Logs

- Website clickstream data
- Application logs

3. Importance of Big Data

Big Data plays a crucial role in modern industries:

- Better decision making
- Customer behavior analysis
- Detecting fraud
- Improving business efficiency
- Identifying trends and patterns
- Creating new products and services

Example:

Amazon analyzes user search and buying patterns to show personalized product recommendations.

4. How Big Data is Stored and Processed

Traditional systems cannot handle Big Data. Hence, special technologies are used:

a. Hadoop Ecosystem

A popular open-source framework for storing and processing Big Data.

- HDFS (Hadoop Distributed File System) – storage
- MapReduce – processing
- YARN – resource management
- Hive, Pig – querying and analysis

b. Apache Spark

- Faster than Hadoop MapReduce
- Supports real-time data processing
- Used in machine learning and streaming applications

c. NoSQL Databases

Used for handling large, unstructured datasets.

- MongoDB
- Cassandra
- CouchDB

d. Cloud Platforms

Cloud makes Big Data storage and processing easy.

- Amazon Web Services (AWS)
- Microsoft Azure
- Google Cloud Platform (GCP)

5. Applications of Big Data

a. Healthcare

- Disease prediction
- Analyzing patient data

b. Business & Marketing

- Customer segmentation
- Targeted advertising

c. Banking & Finance

- Fraud detection
- Risk analysis

d. Transportation

- Traffic management
- Route optimization (used by Uber, Ola)

e. Social Media Platforms

- Trend analysis
- Sentiment analysis (understanding user emotions)

6. Future of Big Data

Big Data is the backbone of modern technological advancements. With AI, Machine Learning, Cloud Computing, and IoT, the importance of Big Data will grow even more.

Future applications include:

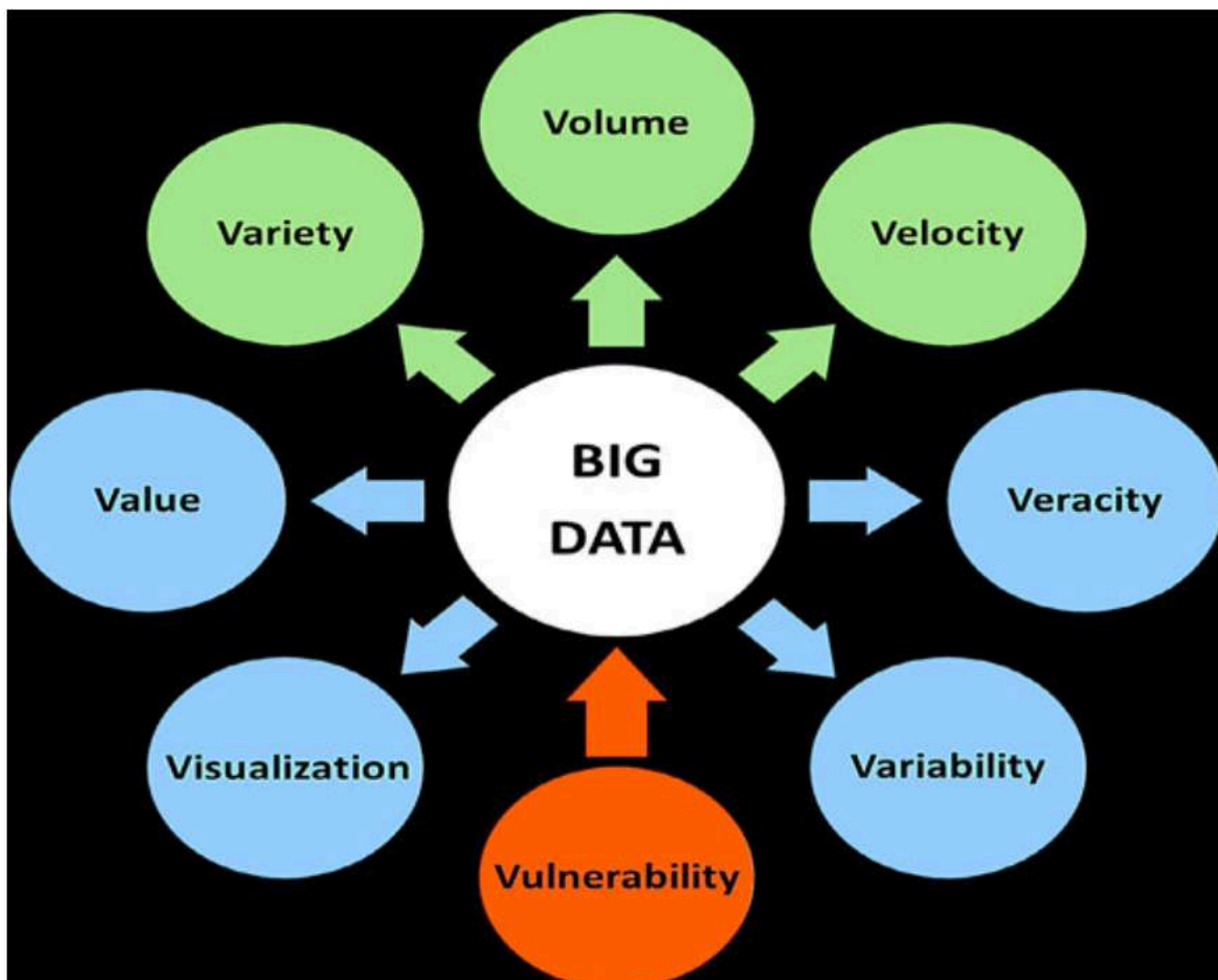
- Smart cities
- Automated systems
- Advanced healthcare analytics
- Personalized digital services

Big Data Characteristics

Big Data is commonly described through specific characteristics that define its nature and complexity. Initially, Big Data was described using the 3 V's, but over time, more characteristics were added. Today, Big Data is usually explained using 5 V's or even 7 V's.

Below are the main characteristics of Big Data explained in detail:

Diagram:



1. Volume (Amount of Data)

Meaning:

Volume refers to the enormous amount of data generated every second from different sources.

Examples:

- Facebook generates petabytes of data daily.
- YouTube users upload hundreds of hours of video every minute.
- Online shopping websites store millions of customer transactions.

Why it matters:

Traditional databases cannot store or handle such massive datasets. Big Data systems like Hadoop and cloud storage are needed.

2. Velocity (Speed of Data Generation)

Meaning:

Velocity indicates the speed at which data is created, collected, and processed.

Examples:

- Stock market data changes within milliseconds.
- GPS tracking systems update location data every second.
- Social media posts, likes, comments are generated rapidly.

Why it matters:

Real-time data processing systems (like Apache Spark) are required to analyze data quickly for immediate decisions.

3. Variety (Different Types of Data)

Meaning:

Variety refers to the multiple formats and types of data generated from different sources.

Types of Data:

1. Structured data

- Organized, in rows & columns
- Example: Database tables, Excel sheets

2. Semi-structured data

- Partially organized
- Example: HTML, XML, JSON

3. Unstructured data

- No fixed structure
- Example: Images, videos, social media posts, audio files

Why it matters:

Handling diverse data types requires NoSQL databases like MongoDB, Cassandra, etc.

4. Veracity (Trustworthiness and Quality of Data)

Meaning:

Veracity refers to the accuracy, reliability, and quality of data.

Challenges:

- Data may be incomplete
- Data may contain duplicates
- Data may be incorrect or noisy

Example:

- Fake social media profiles produce misleading data
- Incorrect sensor readings

Why it matters:

Poor-quality data leads to incorrect decisions. Hence, data cleaning and validation are essential.

5. Value (Importance of Data)

Meaning:

Value means the usefulness or benefits that organizations can gain from big data.

Examples:

- Predicting customer behavior
- Improving business strategies
- Detecting fraud in banks
- Optimizing transportation routes

Why it matters:

Even if data is large, fast, and diverse, it is meaningless unless it generates value.

Additional Characteristics (7V Model)

Some modern Big Data frameworks include two additional characteristics:

6. Variability

Meaning:

Data flow is inconsistent and can change rapidly.

Examples:

- Social media trends change moment to moment
- Seasonal spikes in online shopping
- Weather patterns showing unpredictable changes

7. Visualization

Meaning:

Big data must be presented in visual formats (graphs, charts, dashboards) to make it understandable.

Tools used:

- Tableau
- Power BI
- QlikView

Summary of Big Data Characteristics

Characteristic	Meaning	Example
Volume	Large amount of data	Facebook data, YouTube uploads
Velocity	Speed of data generation	Stock market, GPS
Variety	Different data types	Text, images, videos
Veracity	Accuracy & reliability	Authentic vs fake data
Value	Useful insights	Customer behavior
Variability	Data flow inconsistency	Trend changes
Visualization	Visual representation	Dashboards

Types of Big Data

Big Data is broadly classified into 3 main types:

1. Structured Data
2. Unstructured Data
3. Semi-Structured Data

1. Structured Data

Structured data is organized and arranged in a fixed format (rows, columns, tables). It can be easily stored, processed, and analyzed using traditional databases (like SQL).

Characteristics:

- Highly organized
- Easy to search and retrieve
- Follows a definite schema
- Stored in relational databases

Examples:

- Bank transaction records
- Employee details (name, salary, ID)
- Student records in a table
- Sales records (Excel sheets)
- ATM transaction logs

Tools used:

- SQL databases (MySQL, Oracle, PostgreSQL)
- Data warehouses

2. Unstructured Data

Unstructured data does not have a predefined format or structure. It is zero or low organization and requires advanced tools to process (like Hadoop, NoSQL, AI).

Characteristics:

- Very complex and difficult to analyze
- Does not follow any schema
- Cannot be stored in traditional relational databases directly

Examples:

- Images, videos, audio
- Social media posts (tweets, comments, reels)
- Emails
- PDFs, documents
- Website content
- CCTV footage

Tools used:

- Hadoop (HDFS)
- Apache Spark
- NoSQL databases (MongoDB, Cassandra)

3. Semi-Structured Data

Definition:

Semi-structured data does not follow a rigid structure like tables, but has some organizational properties such as tags or markers.

This type of data lies between structured and unstructured.

Characteristics:

- Flexible structure
- Contains metadata
- Easier to analyze than unstructured data
- Does not require a fixed schema

Examples:

- JSON files
- XML files
- HTML pages
- Email (body is unstructured, but headers are structured)
- Log files
- Sensor data with tags

Tools used:

- NoSQL databases
- Big Data frameworks
- Document stores (MongoDB)

Types of Big Data

Type of Data	Structure	Examples	Storage / Tools
Structured	Organized in tables	Banking records, Excel sheets	SQL Databases
Unstructured	No fixed format	Videos, images, social media posts	Hadoop, Spark
Semi-Structured	Partially organized	JSON, XML, log files	NoSQL, MongoDB

4. Other categories of Big Data

Sometimes Big Data is also described based on its source, such as:

1. Machine-generated Data

- Sensors, IoT devices
- Smart meters
- GPS signals

2. Human-generated Data

- Social media content
- Emails
- Call center messages

3. Process-generated Data

- Transaction records
- Business process logs

Traditional Data vs Big Data

Big Data and Traditional Data differ in many ways such as size, speed, storage, tools, processing methods, flexibility, and data types.

Let's understand each parameter one by one.

1. Definition

Traditional Data

- Data that is small in size, structured, and stored in traditional databases (RDBMS).
- Used by older data systems with limited capacity.

Big Data

- Data that is very large, fast, and complex, coming from multiple sources.
- Cannot be handled by traditional systems due to its volume, variety, and high speed.

2. Data Size (Volume)

Traditional Data

- Comes in MBs, GBs
- Example: Excel files, small SQL databases

Big Data

- Comes in TBs, PBs, EB levels
- Example: Facebook posts, YouTube videos, social media data

3. Data Types (Variety)

Traditional Data

- Only structured (rows and columns)
- Example: Bank records, student tables

Big Data

- Structured + Semi-Structured + Unstructured
- Example:
 - Structured → SQL tables
 - Semi-structured → JSON, XML
 - Unstructured → Videos, images, audio, posts

4. Processing Speed (Velocity)

Traditional Data

- Slow processing
- Batch processing only
- Limited data flow

Big Data

- High-speed processing
- Real-time + Near real-time
- Continuous data streaming

5. Storage Systems

Traditional Data

- Stored in RDBMS (Relational Databases)
- Requires a single server or limited storage

Big Data

- Stored in distributed systems
- Multiple servers store data in parallel
- Examples:
 - Hadoop HDFS
 - Cloud storage

6. Data Processing Methods

Traditional Data

- Uses SQL queries
- Centralized processing
- Works with small data

Big Data

- Uses parallel and distributed processing
- Tools:
 - MapReduce
 - Spark
 - NoSQL
 - Machine learning tools

7. Scalability

Traditional Data

- Vertical scaling (increase RAM, CPU of one machine)
- Costly and limited

Big Data

- Horizontal scaling (add more machines)
- Cheaper, faster, more flexible

8. Cost

Traditional Data

- Expensive database servers
- High maintenance cost

Big Data

- Cost-effective
- Uses commodity hardware
- Open-source tools available (Hadoop, Spark)

9. Data Accuracy & Quality

Traditional Data

- High accuracy
- Clean and verified data

Big Data

- High volume → may have errors/noise
- Requires cleaning (veracity issues)

10. Applications

Traditional Data

- Small businesses
- Banking records
- Inventory data
- School/office databases

Big Data

- Social media platforms
- E-commerce companies
- Healthcare predictions
- Transportation (Uber, Ola)
- Weather forecasting
- AI & Machine Learning

Comparison Table: Traditional Data vs Big Data

Feature	Traditional Data	Big Data
Size	MB–GB	TB–PB–EB
Type	Structured	Structured, Semi-structured, Unstructured
Processing	Slow, batch	Fast, real-time
Storage	RDBMS	Hadoop, NoSQL, Cloud
Scalability	Vertical	Horizontal
Cost	High	Low (open-source)
Tools	SQL	Hadoop, Spark, NoSQL
Speed	Low velocity	High velocity
Sources	Limited	Social media, IoT, sensors, logs

Evolution of Big Data

Big Data has not come suddenly.

It evolved slowly over decades due to growth in computing, internet, storage, and user behavior.

1: Early Data (1960–1980)

✓ Computers first started storing data

- Mainframe computers were introduced.
- Data was small, mostly text-based.
- Stored on:
 - Magnetic tapes
 - Floppy disks

✓ Characteristics

- Limited storage (few KBs or MBs)
- Structured data only
- No concept of “big” data

✓ Example

- Bank transaction records
- Simple payroll & billing systems

2: Database Era (1980–1990)

✓ Introduction of RDBMS

- Oracle, IBM DB2, and SQL-based systems became popular.
- Data stored in tables (rows & columns).

✓ Why important?

- Easy to store and retrieve data
- Used by banks, schools, companies

✓ Limitation

- Could handle only structured data and small sizes.

This era focused on *organizing* data.

3: Internet Explosion (1990–2005)

✓ What changed?

- The internet became widespread.
- Websites, emails, and online shopping started booming.
- User-generated data increased massively.

✓ New sources of data appeared:

- Emails
- Online forms
- Websites
- Digital photos
- Mobile phone usage

✓ Limitation

Traditional RDBMS could not handle:

- Large size
- Variety (images, videos)
- High-speed generation

4: Birth of Big Data Concept (2000–2010)

✓ Who Popularized the Term?

The term "Big Data" became popular when Doug Laney introduced the 3Vs (Volume, Velocity, Variety) model in 2001.

✓ What was happening at this time?

- Social media started growing (Facebook, YouTube, Twitter).
- Smartphones increased digital footprints.
- Companies began analyzing huge datasets.

✓ Google's Contribution

Google introduced:

- Google File System (GFS)
- MapReduce programming model

These innovations enabled large-scale distributed data processing.

This stage marked the beginning of Big Data technologies.

5: Hadoop Revolution (2006–2015)

✓ Birth of Hadoop

Inspired by Google's papers, Doug Cutting and Mike Cafarella created Hadoop, an open-source Big Data framework.

✓ Why has Hadoop changed everything?

- Can store data across multiple machines
- Fault tolerance (no data loss)
- Very cost-effective
- Works with commodity hardware

✓ Hadoop Ecosystem Expanded:

- HDFS
- MapReduce
- YARN
- Hive
- Pig
- HBase
- Sqoop

✓ Industries Start Adopting Big Data

- E-commerce
- Banking
- Social media
- Healthcare

6: Real-Time Big Data & NoSQL (2010–Present)

✓ Limitation of Hadoop

Hadoop is slow for real-time processing (batch-oriented).

✓ New Technologies Introduced

To handle fast and varied data:

1. Real-time tools

- Apache Spark
- Apache Storm
- Apache Flink

2. NoSQL Databases

- MongoDB
- Cassandra
- CouchDB

These handle unstructured and semi-structured data.

✓ Modern Big Data Applications

- Recommendation engines (Amazon, Netflix)
- Fraud detection
- Autonomous vehicles
- Social media analytics

Big Data is now used everywhere.

7: Cloud-Based Big Data (2015–Now)

✓ Shift to Cloud

Due to huge data growth, companies moved to cloud platforms:

- Amazon Web Services (AWS)
- Google Cloud
- Microsoft Azure

✓ Benefits

- Unlimited storage
- On-demand processing
- Scalable
- Low cost compared to physical hardware

This stage made Big Data globally accessible.

8: Big Data + AI + Machine Learning (2020–Now)

✓ What is happening now?

Big Data drives Artificial Intelligence (AI) and Machine Learning (ML).

AI algorithms require large amounts of data to learn patterns.

✓ Modern technologies

- Deep Learning
- Neural Networks
- Data Lakes
- MLOps
- Predictive Analytics

✓ Applications

- Self-driving cars
- Virtual assistants (Alexa, Siri)
- Medical diagnosis
- Personalized advertising
- Chatbots

Big Data + AI is shaping the future.

Evolution of Big Data

Stage	Time	Key Development
1	1960–80	Early computers, small data
2	1980–90	RDBMS & SQL
3	1990–2005	Internet explosion
4	2000–10	Big Data concept (3Vs)
5	2006–15	Hadoop era begins
6	2010–Now	Spark & NoSQL, real-time processing
7	2015–Now	Cloud Big Data
8	2020–Now	Big Data + AI + ML



Challenges with Big Data

Big Data brings many advantages, but it also comes with several challenges due to its enormous size, high speed, and complexity.

Let's understand all the challenges one-by-one.

1. Data Storage and Management

✓ Why is this a challenge?

Big Data is extremely large (TBs, PBs, even EB).

Traditional storage systems cannot store such huge data.

✓ Problems:

- Requires large disk space
- Needs distributed storage
- Hardware cost increases
- Data may be scattered across multiple locations

✓ Example:

Facebook, Google, Amazon generate enormous data daily and cannot rely on a single server.

2. Data Processing Speed

✓ Why is this difficult?

Big Data is generated at high speed (velocity):

- Online transactions
- Social media activity
- IoT sensor data

Traditional systems cannot process data in real-time.

✓ Real Examples:

- Stock market data changes in milliseconds
- GPS tracking updates every second

Organizations need fast systems like Apache Spark.

3. Data Variety

✓ Why does variety cause challenges?

Big Data has many types:

- Structured
- Semi-structured
- Unstructured

Unstructured data (images, videos, audio) is difficult to:

- Store
- Process
- Analyze

✓ Example:

Analyzing millions of YouTube videos requires special tools and huge computational power.

4. Data Quality (Veracity Issues)

✓ What is the challenge?

Big Data often contains:

- Incomplete data
- Duplicates
- Incorrect information
- Noise

Bad data = Wrong conclusions

✓ Example:

Fake social media likes/comments can mislead sentiment analysis.

5. Data Security and Privacy

✓ Why is security a major challenge?

Big Data is collected from:

- Social media
- Financial transactions
- Health records
- IoT devices

This sensitive data is highly vulnerable to:

- Hacking
- Cyberattacks
- Unauthorized access

✓ Examples:

- Data breaches in banks
- Social media privacy violations

Organizations must use strong encryption & authentication.

6. Data Integration

✓ Why is integration difficult?

Data comes from multiple sources:

- Websites
- Mobile apps
- Sensors
- Databases
- Cloud platforms

Combining all this data consistently is challenging.

✓ Example:

Merging customer data from website + app + offline store requires high accuracy.

7. Scalability Issues

✓ The challenge:

Systems must scale as data grows.

Traditional systems scale vertically (upgrade one machine).

Big Data systems need horizontal scaling (adding more machines).

✓ Problems:

- Cost
- Infrastructure complexity
- Network management

8. High Cost of Big Data Tools and Technology

✓ Why costly?

Many Big Data technologies require:

- High-end hardware
- Large clusters
- Skilled professionals
- Cloud storage

✓ Example:

Running Hadoop or Spark clusters needs multiple servers and maintenance.

9. Skilled Workforce Shortage

✓ Why difficult?

Big Data systems need experts in:

- Hadoop
- Spark
- NoSQL
- Machine Learning
- Cloud computing

But trained professionals are limited and expensive.

10. Real-Time Data Analysis

✓ Challenge:

Analyzing huge data in real time requires:

- Fast processing engines
- Low-latency networks
- High availability

✓ Example:

Fraud detection in banks must happen instantly.

11. Data Governance and Compliance

✓ Problems:

- Rules like GDPR, HIPAA require proper data handling.
- Organizations must follow strict laws regarding:
 - Data collection
 - Storage
 - Usage
 - Sharing

Non-compliance leads to heavy penalties.

12. Data Visualization

✓ Why challenging?

Representing huge complex data visually is difficult.

Charts, graphs, dashboards must be:

- Clear
- Accurate
- Real-time

Traditional visualization tools struggle with Big Data.

13. Fault Tolerance and System Failure

✓ Challenge:

Big Data systems run on many distributed machines.

If one machine fails → data loss or system crash can occur.

✓ Required solution:

- Replication
- Backup
- Recovery mechanisms

Technologies Available for Big Data

Big Data technologies are divided into several categories based on their purpose:

1. Storage Technologies
2. Processing Technologies
3. Databases (NoSQL + NewSQL)
4. Analytics & Machine Learning Tools
5. Data Ingestion Tools
6. Data Visualization Tools
7. Cloud-Based Big Data Platforms

Let's understand each category thoroughly.

1. Big Data Storage Technologies

These technologies store large, distributed datasets across multiple machines.

a. Hadoop Distributed File System (HDFS)

✓ What is it?

- A distributed storage system used by Hadoop.
- Stores big data across thousands of machines.

✓ Features:

- Fault tolerant
- High scalability
- Cost-effective (uses normal hardware)

✓ Why is it used?

For storing TBs and PBs of structured & unstructured data.

b. Google File System (GFS)

- Developed by Google.
- Inspired the creation of Hadoop.
- Highly scalable distributed file system.

c. Cloud Storage Systems

Used for modern Big Data storage.

Examples:

- Amazon S3
- Google Cloud Storage
- Microsoft Azure Blob Storage

Features:

- On-demand storage
- No need for physical hardware
- Secure & scalable

2. Big Data Processing Technologies

Processing involves analyzing and transforming huge datasets.

a. MapReduce (Hadoop Processing Engine)

✓ What it does:

Breaks large tasks into smaller tasks processed across many machines.

✓ Components:

- Map → Splits task
- Reduce → Combines results

✓ Use:

Batch processing of huge datasets.

b. Apache Spark

✓ Why is Spark important?

- Faster than MapReduce
- Uses in-memory processing
- Supports real-time analysis

✓ Spark Components:

- Spark SQL
- Spark Streaming
- MLlib (Machine Learning)
- GraphX

c. Apache Flink

- Real-time streaming engine
- Low latency
- Used in banking, fraud detection, IoT

d. Apache Storm

- Real-time processing
- Ideal for live data (Twitter streams, weather updates)

e. Apache Samza

- Works with Kafka
- Used for real-time pipelines and streaming analytics

3. Databases for Big Data (NoSQL & NewSQL)

Traditional SQL databases cannot handle Big Data, so NoSQL technologies are used.

a. NoSQL Databases

✓ Types of NoSQL:

1. Document-based → Example: MongoDB
2. Column-based → Example: Cassandra, HBase
3. Key-Value Stores → Example: Redis
4. Graph Databases → Example: Neo4j

★ MongoDB

- Document-oriented
- Stores JSON-like documents
- Used for flexible, unstructured data

★ Cassandra

- Distributed, scalable
- High availability
- Used by Netflix, Facebook

★ HBase

- Column-oriented
- Works on top of HDFS
- Ideal for large datasets

b. NewSQL Databases

- Combines SQL with Big Data scalability
- Supports ACID properties + high performance

Examples:

- Google Spanner
- VoltDB
- CockroachDB

4. Big Data Analytics & Machine Learning Tools

These tools analyze data, build models, and generate insights.

a. Apache Hive

- SQL-like query language for Big Data
- Converts queries into MapReduce jobs
- Used for data warehousing on Hadoop

b. Apache Pig

- High-level scripting language (Pig Latin)
- Used for data transformation on Hadoop

c. R Programming

- Statistical analysis
- Data visualization
- Used by researchers & data scientists

d. Python Libraries

- Pandas
- NumPy
- SciPy
- Matplotlib
- Scikit-learn (ML library)

Used for analytics, modeling & data science.

e. Apache Mahout

- Machine learning on Hadoop
- Used for clustering, classification, recommendations

f. RapidMiner

- Drag-and-drop analytics tool
- No coding required
- Used for ML, data mining

5. Data Ingestion & ETL Tools

These tools collect data from various sources and load it into Big Data systems.

A. Apache Kafka

- High-speed messaging system
- Handles real-time data streams
- Used by Uber, Netflix, LinkedIn

B. Apache Sqoop

- Transfers data between Hadoop & relational databases

C. Apache Flume

- Collects log data from servers
- Used for web logs, events, social media

D. Talend

- ETL tool for data integration
- Connects multiple systems

6. Big Data Visualization Tools

Visualization helps understand huge datasets using charts and dashboards.

A. Tableau

- Interactive dashboards
- Used for business intelligence

B. Power BI

- Data visualization by Microsoft
- Integration with Excel, SQL, cloud

C. QlikView / Qlik Sense

- Visual analytics
- Used in enterprise-level reporting

D. Google Data Studio

- Free cloud-based visualization tool

7. Cloud Platforms for Big Data

Modern companies use the cloud for Big Data storage and analytics.

A. Amazon Web Services (AWS)

Big Data tools include:

- EMR (Elastic MapReduce)
- Redshift
- AWS Glue
- Kinesis (real-time)

B. Google Cloud Platform (GCP)

Tools:

- BigQuery
- Dataproc
- Dataflow
- Cloud Storage

C. Microsoft Azure

Tools:

- Azure HDInsight
- Azure Databricks
- Data Lake Storage

Infrastructure for Big Data

Big Data infrastructure refers to the hardware, software, network, and tools needed to store, process, manage, and analyze huge volumes of data.

A proper Big Data infrastructure must support the 3Vs (Volume, Velocity, Variety) and allow scalable, fast, and efficient data handling.

1. Storage Infrastructure

Storage is the backbone of Big Data because the data is huge (TB–PB).
Traditional storage (single server, RDBMS) cannot handle Big Data.

Big Data uses distributed storage systems, meaning data is split across many machines.

✓ Key Storage Technologies:

1. **HDFS (Hadoop Distributed File System)**
 - Stores data across clusters
 - Fault-tolerant
 - High scalability
2. **Cloud Storage**
 - AWS S3
 - Google Cloud Storage
 - Azure Blob Storage
 - Unlimited storage & auto-scaling
3. **Data Lakes**
 - Stores raw, unprocessed data
 - Supports structured + unstructured data
 - Examples: AWS Lake Formation, Azure Data Lake
4. **Distributed File Systems**
 - Google File System (GFS)
 - GlusterFS
 - CephFS

2. Compute / Processing Infrastructure

Processing infrastructure handles analysis, transformation, computation, and running algorithms on Big Data.

Traditional CPUs cannot process huge datasets quickly.

Big Data uses distributed computing across clusters.

✓ Processing Frameworks:

1. **MapReduce**
 - Batch processing
 - Breaks large tasks into smaller parts
2. **Apache Spark**
 - Fast in-memory processing
 - Supports real-time analytics
3. **Apache Flink, Storm, Samza**
 - Real-time stream processing
 - Used for fraud detection, live dashboards
4. **Distributed Clusters**
 - Thousands of servers work together
 - Each performs parallel processing

3. Database Infrastructure

Big Data supports multiple database types because data is not only structured.

✓ NoSQL Databases:

- MongoDB (Document-based)
- Cassandra (Column-based)
- HBase (Column-based, built on HDFS)
- Redis (Key-value store)
- Neo4j (Graph database)

✓ SQL / NewSQL Databases:

- Google Spanner
- VoltDB
- CockroachDB

✓ Why NoSQL?

- Handles unstructured data
- Scales horizontally
- Faster reads/writes

4. Data Ingestion Infrastructure

This layer collects data from multiple sources and feeds it into Big Data systems.

✓ Tools:

1. Apache Kafka

- High-speed data streaming
- Used by Uber, Netflix

2. Apache Flume

- Collects log data

3. Apache Sqoop

- Transfers data between SQL databases and Hadoop

4. NiFi

- Connects, transfers, and automates data pipelines

5. Networking Infrastructure

Big Data systems run on clusters, so network must be:

- High-speed
- Low-latency
- Secure

✓ Requirements:

- High-bandwidth switches
- Fast Ethernet (10Gbps to 100Gbps)
- Load balancing
- Secure VPNs

✓ Why is it important?

Big Data processing needs to move huge data blocks across nodes.

6. Server & Hardware Infrastructure

Big Data requires large clusters of machines.

✓ Hardware Components:

1. **Commodity Servers**
 - Low-cost machines used in large quantities
 - Used in Hadoop clusters
2. **CPU / GPU Servers**
 - GPU used for AI & ML processing
3. **Memory (RAM)**
 - Needed for in-memory processing (Spark)
4. **High-Capacity Disks**
 - SSDs for fast access
 - HDDs for large storage
5. **Clusters**
 - A collection of machines working together

7. Processing Framework Infrastructure

This includes resource management and job scheduling tools.

✓ Tools:

1. YARN (Yet Another Resource Negotiator)
 - Manages cluster resources in Hadoop
2. Mesos
 - Divides resources for large-scale applications
3. Kubernetes
 - Manages containerized applications

8. Visualization Infrastructure

Big Data dashboards help visualize insights clearly.

✓ Tools:

- Tableau
- Power BI
- QlikView
- Google Data Studio
- Apache Superset

These tools connect to Big Data systems and generate reports.

9. Security Infrastructure

Big Data needs strong security because it contains sensitive information.

✓ Components:

1. Data Encryption
2. Authentication (Kerberos)
3. Authorization (Ranger, Sentry)
4. Firewall protection
5. Auditing systems

10. Cloud Infrastructure for Big Data

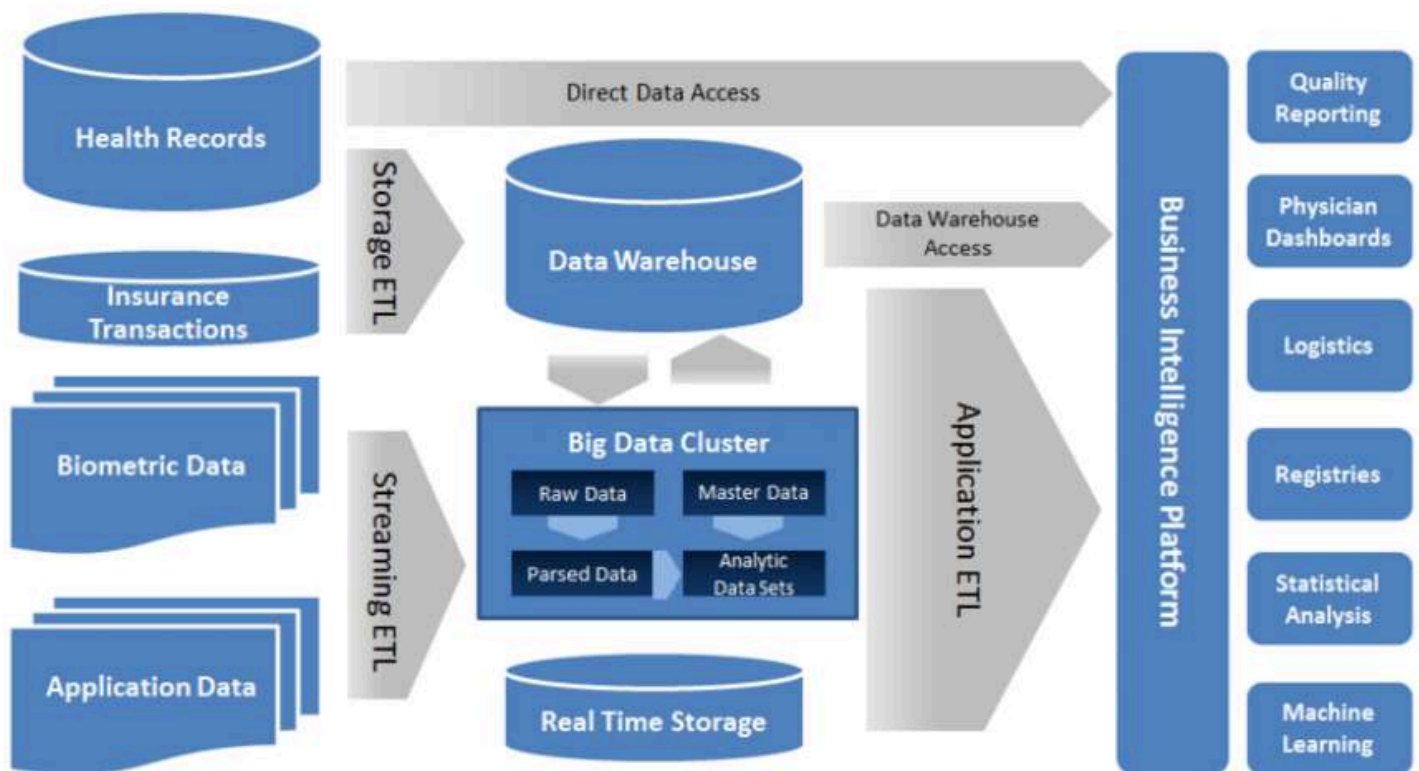
Cloud is widely used due to:

- Low cost
- High scalability
- No hardware maintenance

✓ Major Cloud Platforms:

1. Amazon AWS
 - EMR, S3, Redshift, Glue, Kinesis
2. Google Cloud
 - BigQuery, Dataflow, Dataproc
3. Microsoft Azure
 - HDInsight, Azure Databricks, Data Lake

Big Data Infrastructure Diagram



Uses of Data Analytics

Data Analytics refers to the process of examining raw data to draw meaningful insights, patterns, and trends that can help in decision-making, forecasting, and optimization.

It is widely used in business, healthcare, banking, government, education, and technology.

1. Business Analytics

✓ Purpose:

- Improve business performance
- Make data-driven decisions

✓ Applications:

1. Customer Analytics
 - Understand customer behavior, preferences, and trends
 - Example: Amazon uses purchase history to recommend products
2. Sales & Marketing Analytics
 - Predict sales trends, optimize pricing, target advertising
 - Example: Netflix suggests shows based on viewing patterns
3. Supply Chain & Inventory Management
 - Analyze stock levels and demand forecasts
 - Example: Walmart uses analytics to prevent overstock or understock
4. Fraud Detection
 - Detect suspicious transactions in real-time
 - Example: Credit card fraud detection by banks

2. Healthcare Analytics

✓ Purpose:

- Improve patient care
- Reduce costs
- Predict disease outbreaks

✓ Applications:

1. Predictive Analytics
 - Predict risk of diseases based on patient data
 - Example: Hospitals predicting heart attacks
2. Operational Analytics
 - Optimize hospital operations and staff allocation
3. Medical Research
 - Analyze clinical trials and genetic data
4. Patient Engagement
 - Personalized treatment plans and reminders

3. Financial Analytics

✓ Purpose:

- Improve financial decisions
- Manage risk

✓ Applications:

1. Credit Risk Management
 - Evaluate loan eligibility using credit scores
 - Example: Banks using analytics to approve/deny loans
2. Fraud Detection
 - Detect unusual transactions in real-time
3. Investment & Portfolio Analytics
 - Predict stock market trends using historical data
4. Budgeting & Forecasting
 - Plan future finances for businesses

4. Retail and E-commerce Analytics

✓ Purpose:

- Enhance customer experience
- Boost sales and revenue

✓ Applications:

1. Recommendation Systems
 - Suggest products based on previous purchases
 - Example: Amazon, Flipkart
2. Customer Sentiment Analysis
 - Analyze reviews & feedback to improve products
3. Inventory & Pricing Analytics
 - Optimize stock levels, dynamic pricing strategies

5. Manufacturing and Operations Analytics

✓ Purpose:

- Improve efficiency and reduce costs

✓ Applications:

1. Predictive Maintenance
 - Predict machine failures before they happen
 - Example: Car manufacturers like BMW
2. Supply Chain Optimization
 - Optimize routes, reduce delays, and manage warehouses
3. Quality Control
 - Detect defective products using analytics

6. Government and Public Sector Analytics

✓ Purpose:

- Improve governance
- Public safety and policy planning

✓ Applications:

1. Crime Analytics
 - Predict crime hotspots, allocate police efficiently
2. Urban Planning
 - Analyze traffic, population, and infrastructure needs
3. Tax and Revenue Analytics
 - Detect tax evasion and optimize revenue collection
4. Disaster Management
 - Predict natural disasters and manage relief efforts

7. Telecommunications Analytics

✓ Purpose:

- Improve customer retention
- Optimize network performance

✓ Applications:

1. Churn Prediction
 - Identify customers likely to switch providers
2. Network Optimization
 - Analyze network traffic and reduce downtime
3. Fraud Detection
 - Detect fraudulent calls or SIM card misuse

8. Sports Analytics

✓ Purpose:

- Improve player performance
- Enhance team strategy

✓ Applications:

1. Player Performance Analytics
 - Track performance, injury patterns, and fitness
2. Game Strategy Analytics
 - Analyze opponent strategies and plan accordingly
3. Fan Engagement
 - Personalized content & merchandising

9. Education Analytics

✓ Purpose:

- Improve learning outcomes
- Optimize administrative decisions

✓ Applications:

1. Student Performance Analysis
 - Identify struggling students and provide personalized help
2. Course Recommendation
 - Suggest courses based on student interests & performance
3. Administrative Planning
 - Optimize classroom allocation, staffing, and resources

10. Social Media Analytics

✓ Purpose:

- Understand user behavior
- Improve marketing and engagement

✓ Applications:

1. Sentiment Analysis
 - Determine public opinion on products, brands, or policies
2. Trend Analysis
 - Identify trending topics and hashtags
3. Influencer Analytics
 - Measure impact of social media influencers

11. Transportation & Logistics Analytics

✓ Purpose:

- Reduce costs
- Optimize operations

✓ Applications:

1. Route Optimization
 - GPS + traffic analytics to find fastest routes
 - Example: Uber, Ola, Google Maps
2. Predictive Maintenance
 - Predict vehicle or fleet failures
3. Supply Chain Analytics
 - Manage warehouses, deliveries, and fuel usage

12. Energy & Utilities Analytics

✓ Purpose:

- Optimize energy consumption
- Reduce operational costs

✓ Applications:

1. Smart Grid Analytics
 - Monitor and optimize electricity distribution
2. Predictive Maintenance
 - Detect faults in power plants and equipment
3. Consumption Forecasting
 - Predict energy demand based on historical data

Desired Properties of a Big Data System

A Big Data system is designed to store, process, and analyze huge amounts of diverse data efficiently. To be effective, it should satisfy several key properties that ensure performance, reliability, scalability, and usability.

1. Scalability

✓ Definition:

The ability of a system to handle increasing volumes of data or workloads without performance degradation.

✓ Why it's important:

- Big Data grows continuously (TB → PB → EB)
- A scalable system can add more storage or computing resources easily.

✓ Example:

- Hadoop clusters can add more nodes to handle additional data.
- Cloud platforms like AWS, Azure, or GCP automatically scale resources based on demand.

2. Fault Tolerance / Reliability

✓ Definition:

The system should continue functioning even if some components fail.

✓ Why it's important:

- Big Data systems use distributed computing, where many nodes work together.
- Node failure is common due to hardware, network, or software issues.

✓ Mechanisms:

- Data replication (HDFS stores multiple copies)
- Automatic recovery of failed tasks (YARN in Hadoop)

✓ Example:

- If one node in a Hadoop cluster fails, its data is retrieved from another node.

3. High Availability

✓ Definition:

The system should provide continuous access to data with minimal downtime.

✓ Why it's important:

- Businesses require 24/7 access to data for analytics, transactions, or monitoring.

✓ Example:

- Cloud-based Big Data services like Amazon Redshift or Google BigQuery offer high uptime (99.9%+).

4. Performance / Low Latency

✓ Definition:

The system should process and retrieve data quickly, even at a massive scale.

✓ Why it's important:

- Real-time analytics and decision-making require low-latency processing.

✓ Example:

- Apache Spark provides in-memory processing for faster analytics compared to Hadoop MapReduce.
- Fraud detection systems analyze transactions in milliseconds.

5. Data Integrity

✓ Definition:

Data should remain accurate, consistent, and unaltered during storage, transfer, and processing.

✓ Why it's important:

- Reliable analytics depends on correct data.
- Avoids errors in business or scientific conclusions.

✓ Mechanisms:

- Checksums
- Replication
- Transaction management in databases

6. Flexibility / Schema-less Design

✓ Definition:

The system should support structured, semi-structured, and unstructured data.

✓ Why it's important:

- Big Data comes from diverse sources: social media, sensors, logs, images, videos.
- Traditional RDBMS with rigid schema cannot handle this efficiently.

✓ Example:

- NoSQL databases like MongoDB and Cassandra can store JSON or key-value data without a fixed schema.
- Hadoop/HDFS can store any type of data.

7. Security and Privacy

✓ Definition:

The system should protect data from unauthorized access, breaches, and misuse.

✓ Why it's important:

- Big Data contains sensitive information (financial records, health data, social data).
- Legal compliance (GDPR, HIPAA) requires secure handling.

✓ Mechanisms:

- Authentication and Authorization (Kerberos, Ranger, Sentry)
- Encryption (data at rest and in transit)
- Auditing and logging

8. Cost-Effectiveness

✓ Definition:

The system should deliver performance and storage at reasonable cost, using commodity hardware if possible.

✓ Why it's important:

- Big Data requires large storage and compute resources.
- Using expensive enterprise hardware may not be sustainable.

✓ Example:

- Hadoop uses clusters of commodity servers instead of high-end mainframes.
- Cloud services allow pay-as-you-go pricing.

9. Data Locality

✓ Definition:

Processing should ideally occur close to the data rather than transferring large amounts of data over the network.

✓ Why it's important:

- Reduces network congestion and improves processing speed.

✓ Example:

- HDFS moves computation to the nodes where data resides (data locality principle in Hadoop).

10. Ease of Use / Accessibility

✓ Definition:

The system should provide simple interfaces, APIs, and query tools for users to access and analyze data efficiently.

✓ Why it's important:

- Users like analysts, scientists, or business managers may not be programmers.

✓ Example:

- Apache Hive provides SQL-like queries for Hadoop data.
- Tools like Tableau or Power BI provide dashboards and visualization without coding.

11. Support for Real-Time Analytics

✓ Definition:

The system should be capable of processing data in real time for immediate insights.

✓ Why it's important:

- Industries like banking, telecommunications, and e-commerce require real-time decision-making.

✓ Example:

- Spark Streaming, Apache Flink, or Kafka Streams are used for live analytics.

12. Interoperability

✓ Definition:

The system should integrate with other systems, platforms, and tools easily.

✓ Why it's important:

- Big Data environments often include multiple tools for storage, processing, and analytics.

✓ Example:

- Hadoop integrates with Spark, Hive, Pig, HBase, Kafka, and ML tools seamlessly.

UNIT-2

Introduction to Hadoop

Hadoop is an open-source framework designed for storing and processing huge amounts of data in a distributed computing environment. It was developed by Doug Cutting and Mike Cafarella in 2005, inspired by Google's MapReduce and Google File System (GFS).

Hadoop is part of the Apache Software Foundation and is widely used in Big Data applications.

Why Hadoop is Needed

Traditional databases and computing systems cannot handle Big Data efficiently because:

- The volume of data is extremely large (terabytes to petabytes).
- Data is highly varied: structured, semi-structured, and unstructured.
- Data is generated at high speed (real-time streaming).
- High cost and limited scalability of traditional systems.

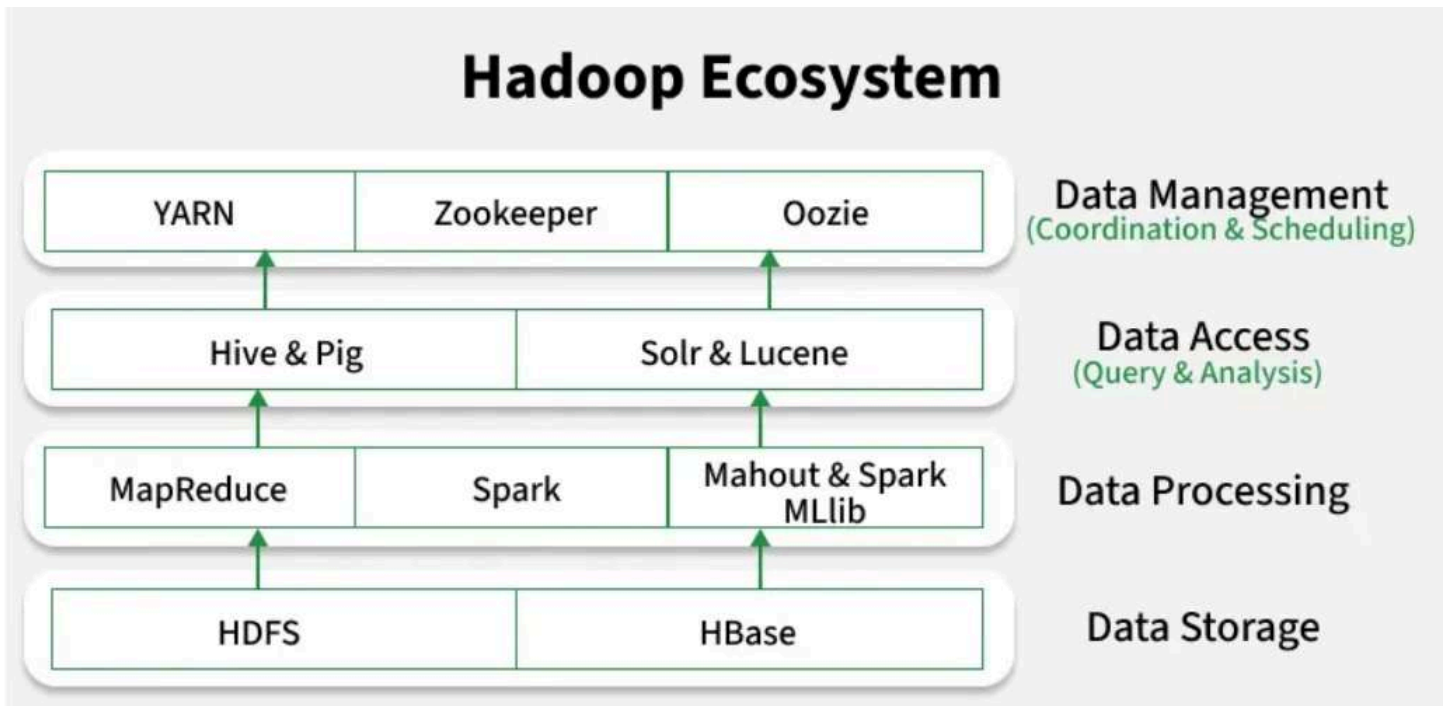
Hadoop provides a scalable, fault-tolerant, and cost-effective solution for processing massive datasets.

Key Features of Hadoop

1. Open Source
 - Free to use, modify, and distribute.
2. Scalable

- Can scale horizontally by adding more nodes to the cluster.
- 3. Fault Tolerant
 - Automatically replicates data across multiple nodes.
- 4. Cost-Effective
 - Uses commodity hardware instead of expensive servers.
- 5. High Throughput
 - Handles large data batches efficiently.
- 6. Flexibility
 - Can store and process structured, semi-structured, and unstructured data.
- 7. Distributed Processing
 - Moves computation closer to where data is stored (data locality principle).

Hadoop Ecosystem / Components



Hadoop is not a single software but a framework with multiple components:

1. Hadoop Distributed File System (HDFS)

- Stores huge data across multiple nodes in a cluster.
- Features:
 - Fault-tolerant (replicates data by default 3 times)
 - High throughput
 - Can store any type of data (structured/unstructured)

2. MapReduce

- Programming model for processing large data sets.
- Steps:
 - Map → Processes input data into key-value pairs
 - Reduce → Aggregates results from Map phase

3. YARN (Yet Another Resource Negotiator)

- Resource management layer of Hadoop.
- Manages computing resources in the cluster.
- Schedules jobs and allocates resources efficiently.

4. Hadoop Common

- Core libraries and utilities used by other Hadoop modules.

Hadoop Architecture

Hadoop follows a Master-Slave architecture:

1. HDFS Architecture

- NameNode (Master)
 - Manages metadata (file names, locations, permissions)
 - Controls the cluster
- DataNode (Slave)
 - Stores actual data blocks
 - Performs read/write operations

2. MapReduce Architecture

- JobTracker (Master) → Schedules jobs and monitors tasks
- TaskTracker (Slave) → Executes tasks on DataNodes

3. YARN Architecture

- ResourceManager (Master) → Allocates resources
- NodeManager (Slave) → Manages tasks on each node

Workflow:

1. Data is stored in HDFS (divided into blocks).
2. MapReduce job is submitted.
3. The job is split into tasks and assigned to nodes (data locality).
4. Results are aggregated by the Reduce phase.
5. Final output is stored back in HDFS.

Advantages of Hadoop

Advantage	Explanation
Scalability	Easily add new nodes to the cluster
Fault Tolerance	Data replicated across nodes
Cost-Effective	Runs on commodity hardware
Flexibility	Handles all types of data
High Throughput	Efficient processing of large datasets
Open Source	Free to use and modify

Limitations of Hadoop

1. Not suitable for small datasets (overhead is high).
2. Complex programming for beginners in MapReduce.
3. Real-time processing is limited (batch-oriented).
4. High latency compared to in-memory systems like Apache Spark.

Hadoop Ecosystem Tools

- Hive → SQL-like queries for Hadoop data
- Pig → Scripting for data transformation
- HBase → NoSQL columnar database
- Sqoop → Data transfer between RDBMS and Hadoop
- Flume → Collect log data into Hadoop
- Oozie → Workflow scheduler
- Mahout → Machine Learning library

Applications of Hadoop

1. Social Media Analytics → Facebook, Twitter analytics
2. E-commerce → Product recommendations (Amazon, Flipkart)
3. Banking & Finance → Fraud detection, risk analysis
4. Healthcare → Patient data analysis, disease prediction
5. Telecom → Network traffic and call data analysis
6. Government → Census, crime analysis, policy planning

Core Hadoop Components

Hadoop is a framework for storing and processing huge volumes of data in a distributed environment. Its core components provide the foundation for the Hadoop ecosystem.

The core components are:

1. Hadoop Distributed File System (HDFS) – Storage Layer
2. MapReduce – Processing Layer
3. YARN (Yet Another Resource Negotiator) – Resource Management
4. Hadoop Common – Utilities and Libraries

1. Hadoop Distributed File System (HDFS)

✓ What is HDFS?

HDFS is a distributed file system that stores very large files across multiple machines. It is designed to run on commodity hardware and provides high throughput and fault tolerance.

✓ Key Features of HDFS:

- Fault Tolerance: Data is automatically replicated across nodes (default: 3 copies).
- High Throughput: Optimized for large-scale batch processing.
- Scalability: Can scale horizontally by adding more DataNodes.
- Flexibility: Can store structured, semi-structured, and unstructured data.

✓ Architecture of HDFS:

1. NameNode (Master)
 - Stores metadata: file names, directories, block locations, permissions.
 - Controls the file system and manages all DataNodes.
 - Critical Node: Failure affects cluster, so backup (Secondary NameNode) is recommended.

2. DataNode (Slave)

- Stores actual data blocks.
- Handles read/write requests from clients.
- Periodically sends heartbeat signals to NameNode to confirm it's alive.

✓ How HDFS Works:

- Large files are divided into blocks (default size: 128 MB).
- Each block is replicated across multiple DataNodes for fault tolerance.
- Clients interact with NameNode to locate blocks, then read/write from DataNodes.

✓ Example:

- A 1 TB file is split into 128 MB blocks → ~8,000 blocks.
- Each block is stored on 3 nodes → total storage used: 3 TB.

2. MapReduce

✓ What is MapReduce?

MapReduce is a programming model for processing large datasets in parallel across a Hadoop cluster.

✓ Phases of MapReduce:

1. Map Phase
 - Input data is split into key-value pairs.
 - Each mapper processes a portion of the data in parallel.
2. Shuffle and Sort
 - Intermediate data from mappers is grouped by key.
3. Reduce Phase
 - Aggregates or summarizes the data from Map phase.
 - Produces the final output.

✓ Key Features:

- Distributed processing: Tasks run on nodes where data is stored (data locality principle).
- Fault Tolerance: Failed tasks are retried on other nodes.
- Scalable: Handles petabytes of data.

✓ Example:

- Word Count:
 - Map: Break text into words → (word, 1)
 - Reduce: Sum all values for each word → (word, total count)

3. YARN (Yet Another Resource Negotiator)

✓ What is YARN?

YARN is the resource management layer of Hadoop 2.x. It manages computing resources and schedules jobs in the cluster.

✓ Components of YARN:

1. ResourceManager (Master)
 - Allocates resources to different applications.
 - Ensures efficient utilization of CPU, memory, and storage.

2. NodeManager (Slave)

- Manages resources on individual nodes.
- Monitors task execution and reports to ResourceManager.

✓ Advantages of YARN:

- Multi-Application Support: Runs MapReduce, Spark, Tez, and other engines.
- Better Resource Utilization: Dynamically allocates resources based on demand.

4. Hadoop Common

✓ What is Hadoop Common?

Hadoop Common is a collection of shared utilities, libraries, and APIs that support other Hadoop modules.

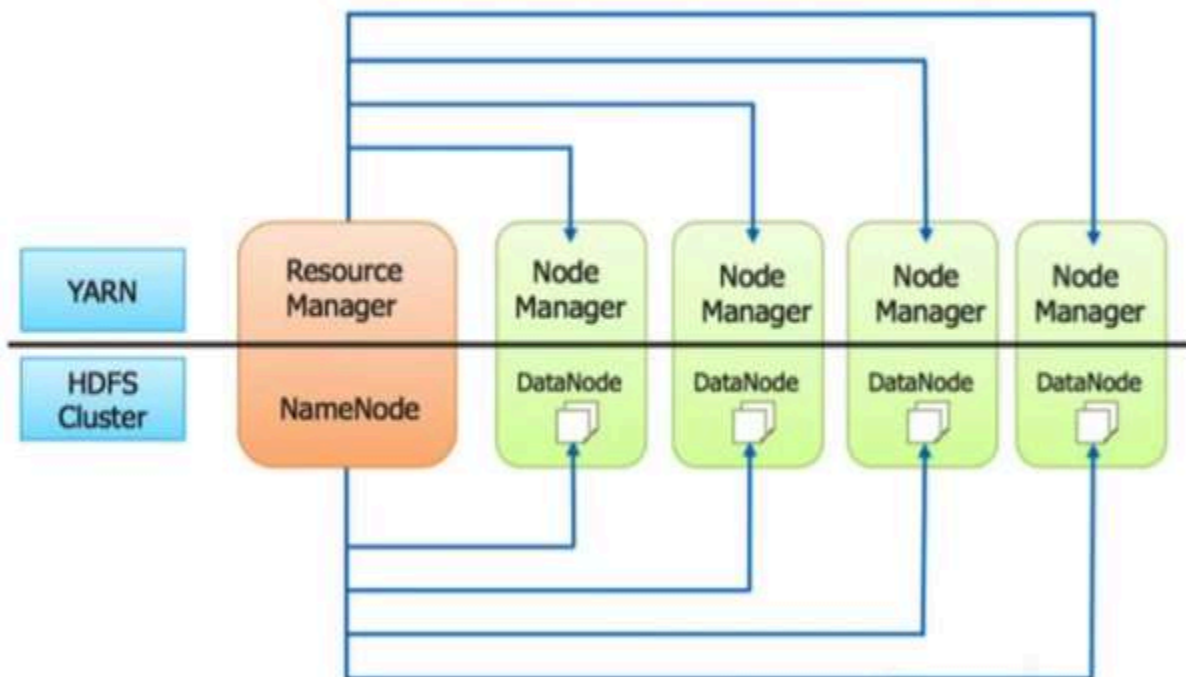
✓ Key Features:

- Provides Java libraries for file system and OS interactions.
- Includes scripts and tools for starting Hadoop services.
- Supports integration with Hadoop ecosystem tools like Hive, Pig, HBase, and Sqoop.

Hadoop Core Architecture Diagram

Hadoop Core Components

SelfReflex



Hadoop Ecosystem – Introduction

The Hadoop Ecosystem is a collection of open-source tools and frameworks that work together to store, process, analyze, and manage Big Data.

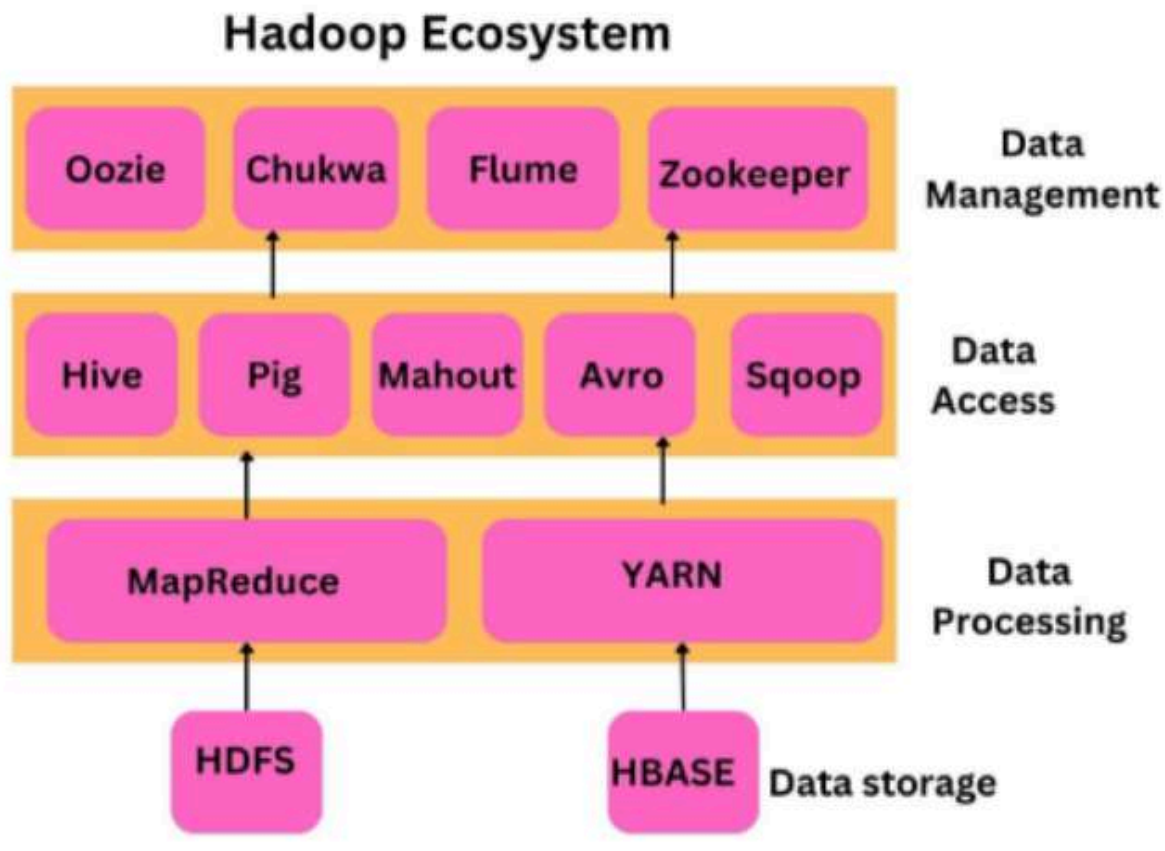
While HDFS and MapReduce form the core of Hadoop, the ecosystem provides enhanced functionality for real-time processing, analytics, workflow management, and data integration.

The ecosystem allows businesses and organizations to handle structured, semi-structured, and unstructured data efficiently.

Major Components of the Hadoop Ecosystem

The Hadoop ecosystem is modular, meaning you can use only the components you need. The major components are:

Diagram:



1. HDFS (Hadoop Distributed File System)

- Role: Storage layer for huge datasets.
- Function: Stores data in blocks across multiple nodes.
- Feature: Fault-tolerant, scalable, cost-effective.

2. MapReduce

- Role: Batch processing framework.
- Function: Processes large datasets in a parallel, distributed manner.
- Example: Word count, log analysis, clickstream analysis.

3. YARN (Yet Another Resource Negotiator)

- Role: Resource and job management layer.
- Function: Allocates cluster resources and schedules tasks.
- Example: Allows MapReduce, Spark, and Tez to run on the same cluster.

4. Hive

- Type: Data Warehouse / SQL-like query tool
- Function: Provides SQL-like querying (HiveQL) for Hadoop data.
- Use Case: Analysts can query large datasets without writing MapReduce code.
- Example: Generate reports, analyze sales trends.

5. Pig

- Type: Scripting / Data Transformation tool
- Function: Uses Pig Latin scripting language to process large datasets.
- Use Case: ETL (Extract, Transform, Load) operations.
- Example: Transform raw logs into structured format for analysis.

6. HBase

- Type: NoSQL database (Column-oriented)
- Function: Provides real-time read/write access to large datasets.
- Use Case: Suitable for random access and high-speed transactions.
- Example: Storing user profiles, social media data, or IoT sensor data.

7. Sqoop

- Type: Data Integration tool
- Function: Transfers data between RDBMS and Hadoop.
- Use Case: Import customer data from MySQL to HDFS or Hive.

8. Flume

- Type: Data Ingestion tool
- Function: Collects, aggregates, and moves large volumes of log or event data into HDFS.
- Use Case: Real-time collection of Twitter feeds, web server logs.

9. Oozie

- Type: Workflow Scheduler
- Function: Automates Hadoop jobs (MapReduce, Hive, Pig, Sqoop) in a defined workflow.
- Use Case: Schedule daily ETL jobs, batch processing pipelines.

10. Zookeeper

- Type: Coordination service
- Function: Manages cluster configuration, synchronization, and naming.
- Use Case: Used by HBase, Kafka, and other distributed systems for reliability.

11. Mahout

- Type: Machine Learning Library
- Function: Provides scalable ML algorithms (classification, clustering, recommendation).
- Use Case: Recommendation engines, customer segmentation.

12. Spark

- Type: Distributed Computing / Processing Engine
- Function: Processes data in-memory for faster analytics.
- Use Case: Real-time analytics, iterative ML tasks, graph processing.

13. Kafka

- Type: Distributed Streaming Platform
- Function: Handles real-time data streams from multiple sources.
- Use Case: Log aggregation, event streaming, real-time analytics pipelines.

Applications of Hadoop Ecosystem

1. E-Commerce → Product recommendation, customer analytics
2. Social Media → Sentiment analysis, trend analysis
3. Banking & Finance → Fraud detection, risk analysis
4. Healthcare → Patient data analysis, disease prediction
5. Telecommunications → Call data analysis, churn prediction
6. Government → Census data analysis, traffic & crime analytics

Hive Physical Architecture

Hive is a data warehouse infrastructure built on top of Hadoop that allows users to query and manage large datasets stored in HDFS using SQL-like queries (HiveQL).

Hive translates SQL queries into MapReduce, Tez, or Spark jobs, making it easier for users to work with Hadoop without writing complex code.

The Hive Architecture can be studied in two ways: Logical Architecture and Physical Architecture. Here, we focus on Physical Architecture.

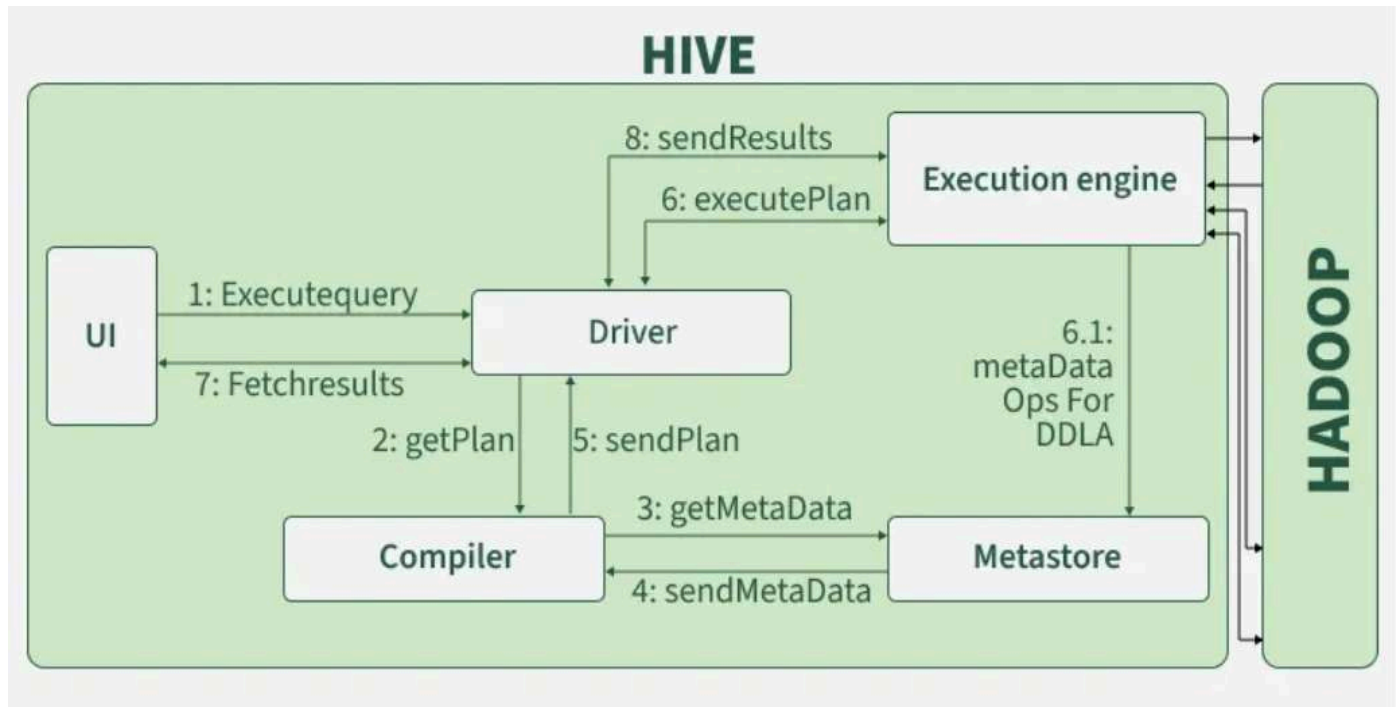
Physical Architecture of Hive

The physical architecture explains how Hive components interact with Hadoop, including data storage, query processing, and execution.

Hive Physical Architecture Components:

1. Hive Clients / User Interface (UI)
2. Driver
3. Compiler
4. Optimizer
5. Execution Engine
6. Metastore
7. HDFS / Hadoop Storage

Diagram:



1. Hive Clients / User Interface (UI)

- Role: Entry point for users to interact with Hive.
- Types of Clients:
 1. Command Line Interface (CLI): Users write HiveQL queries via terminal.
 2. Web UI: Tools like Hue provide browser-based Hive access.
 3. JDBC / ODBC Clients: Applications connect using standard database drivers.

Function: Users submit queries (HiveQL), which are passed to the Hive Driver.

2. Hive Driver

- Role: Acts as a controller and manages the lifecycle of a Hive query.
- Functions:
 - Receives HiveQL queries from clients.
 - Maintains session and query execution context.
 - Passes query to Compiler for parsing.
 - Handles query execution status.

Analogy: The driver is like a database engine coordinator.

3. Compiler

- Role: Converts HiveQL queries into logical execution plans.
- Steps in Compiler:
 1. Parse: Converts query into Abstract Syntax Tree (AST).
 2. Semantic Analysis: Checks table names, column names, types, and query validity.
 3. Logical Plan Generation: Produces Directed Acyclic Graph (DAG) representing operations.
- Output: A logical plan ready for optimization.

4. Optimizer

- Role: Optimizes the logical plan for efficient execution.
- Functions:
 - Rule-based optimization: Pushes filters closer to data, simplifies joins.
 - Cost-based optimization: Chooses efficient execution strategies.
- Output: Optimized physical execution plan, usually converted to MapReduce, Tez, or Spark jobs.

5. Execution Engine

- Role: Executes the physical execution plan.
- Functions:
 - Breaks query into tasks (MapReduce/Tez/Spark).
 - Submits tasks to YARN (Hadoop cluster).
 - Monitors task execution and retrieves results.
- Interaction with Hadoop: Execution engine runs the jobs on HDFS data.

6. Metastore

- Role: Central repository that stores metadata about Hive tables, partitions, schemas, and storage locations.
- Types:
 - Embedded Metastore: Built-in Derby database (for single-user).
 - Standalone Metastore: MySQL/PostgreSQL database (multi-user).
- Importance:
 - HiveQL queries rely on metadata in the metastore.
 - Supports table creation, alteration, and partitioning.

Example Metadata:

- Table name: `sales_data`
- Columns: `id, product, price, date`
- HDFS location: `/user/hive/warehouse/sales_data`

7. HDFS / Hadoop Storage

- Role: Physical storage for Hive data.
- Functions:
 - Stores data in HDFS in files or partitions.
 - Supports multiple formats: Text, ORC, Parquet, Avro, RCFile.
- Interaction: Execution engine reads/writes data from/to HDFS.

Example:

- Hive table `sales_data` is stored at `/user/hive/warehouse/sales_data` in HDFS.
- Query execution reads data blocks, processes them, and writes output back to HDFS.

Hive Physical Architecture Workflow

1. Users submit HiveQL query via CLI, Web UI, or JDBC/ODBC.
2. The driver receives a query and creates execution context.
3. Compiler parses query → semantic analysis → logical plan.
4. Optimizer improves logical plan → physical execution plan (MapReduce/Tez/Spark).
5. Execution Engine submits tasks to YARN.
6. Tasks read/write data from HDFS, process results.
7. Results are returned to the client interface.

Hadoop Limitations

Hadoop is a powerful framework for Big Data storage and processing, but it is not without limitations. Understanding these limitations helps in choosing the right tool for the right use case.

1. Not Suitable for Small Data

- Hadoop is optimized for large-scale batch processing (terabytes to petabytes).
- Overhead of starting MapReduce jobs and distributing tasks can make it slower than traditional databases for small datasets.
- Example: Processing a few gigabytes of data might take longer on Hadoop compared to MySQL or PostgreSQL.

2. Complex Programming

- Native MapReduce programming is difficult for beginners.
- Writing Map and Reduce functions requires Java knowledge, and debugging can be challenging.
- Although tools like Hive and Pig simplify this, they still translate queries into MapReduce, which can be slow.

3. High Latency / Batch Processing Only

- Hadoop is primarily batch-oriented.
- Not designed for real-time or low-latency queries.
- Example: Searching or updating a record in HDFS requires reading the entire file block → slower than traditional databases.
- Real-time frameworks like Spark Streaming, Flink, or Storm are required for low-latency processing.

4. Data Security Limitations

- Early Hadoop versions had limited security features.
- Authentication and authorization were weak compared to RDBMS.
- Modern Hadoop (Hadoop 2.x and above) provides:
 - Kerberos authentication
 - Encryption at rest and in transit
 - Ranger / Sentry for access control
- But configuring these adds complexity.

5. Inefficient for Iterative Processing

- Iterative algorithms like machine learning and graph processing require multiple passes over the same data.
- MapReduce reads and writes data to HDFS between each iteration, causing high I/O overhead.
- Spark and Flink, which process in-memory, are better suited for iterative workloads.

6. Limited SQL Support

- Hive provides SQL-like querying, but:
 - Complex joins and subqueries can be slow.
 - Transactions and ACID support are limited (recent Hive versions improve this).
 - Real-time OLTP-like operations are not feasible.

7. Difficulty in Handling Small Files

- Hadoop performs best with large files.
- Many small files cause:
 - Metadata overload on NameNode
 - Reduced HDFS efficiency and throughput
- Workarounds: Use SequenceFiles or HAR files to combine small files.

8. Requires Skilled Workforce

- Managing Hadoop clusters requires experienced administrators.
- Complexity increases with:
 - Cluster scaling
 - Node failures
 - Security configuration
 - Monitoring and troubleshooting

9. High Memory Usage

- MapReduce jobs, YARN, and tools like Spark require significant RAM.
- Improper memory configuration can lead to task failures or slow performance.

10. Lack of Standardized Ecosystem

- The Hadoop ecosystem is rich but fragmented.
- Tools like Hive, Pig, HBase, Spark, Flume, Oozie, and Kafka require integration and compatibility checks.
- Not all tools are interoperable out-of-the-box, increasing setup and maintenance complexity.

RDBMS vs Hadoop

Feature	RDBMS	Hadoop
Data Type	Structured data only (tables, rows, columns)	Structured, semi-structured, unstructured (text, logs, images, videos)
Schema	Schema-on-write (schema is defined before data insertion)	Schema-on-read (schema applied at query time)
Storage	Single-server or clustered databases; limited to terabytes	Distributed storage (HDFS); can handle petabytes
Scalability	Vertical scaling (adding CPU, RAM, storage to one machine)	Horizontal scaling (adding more nodes to the cluster)
Processing	Transactional (OLTP) and some analytical (OLAP)	Batch processing (MapReduce) and real-time processing (Spark, Flink)
Fault Tolerance	Limited; relies on backups and replication	Built-in fault tolerance via data replication in HDFS
Cost	Expensive; requires high-end servers and licenses	Cost-effective; uses commodity hardware and open-source tools

Performance	High for small to medium datasets; optimized for indexing and transactions	High throughput for large datasets; optimized for parallel processing
Data Volume Handling	Handles GBs to low TBs efficiently	Handles TBs to PBs efficiently
Query Language	SQL (standardized)	HiveQL, Pig Latin, Spark SQL; native MapReduce requires programming
Use Cases	Banking, ERP, HR systems, transactional applications	Social media analytics, log processing, e-commerce, IoT, big data analytics
Latency	Low latency; supports real-time querying	High latency (batch processing), but Spark or Hive on Tez reduces it
Consistency	ACID compliant (Atomicity, Consistency, Isolation, Durability)	Eventual consistency; not ACID by default (some components like HBase support ACID)
Maintenance	Easier to maintain with DBAs	Requires skilled Hadoop administrators; cluster management is complex
Example	Oracle, MySQL, PostgreSQL, SQL Server	Hadoop (HDFS + MapReduce + YARN), Hive, HBase, Spark

Key Differences Explained

1. Data Handling
 - RDBMS handles only structured data.
 - Hadoop handles all types of data: structured, semi-structured, and unstructured.
2. Schema
 - RDBMS uses schema-on-write, meaning data must conform to the table schema before insertion.
 - Hadoop uses schema-on-read, meaning data can be stored in raw form and structured during analysis.
3. Scalability
 - RDBMS scales vertically (upgrade server hardware).
 - Hadoop scales horizontally (add more nodes to cluster), which is cheaper and easier.
4. Fault Tolerance
 - RDBMS relies on backups and replication, which can be costly.
 - Hadoop provides built-in fault tolerance by replicating data across multiple nodes in HDFS.
5. Cost
 - RDBMS often requires licensed software and expensive servers.
 - Hadoop uses commodity hardware and is open-source, reducing infrastructure costs.

6. Processing Type

- RDBMS excels at transactional processing (OLTP) and moderate analytical queries (OLAP).
- Hadoop excels at batch processing of massive datasets and large-scale analytics.

7. Latency

- RDBMS provides real-time responses.
- Hadoop is usually batch-oriented, though tools like Spark improve latency.

8. Maintenance & Skills

- RDBMS is maintained by DBAs with standard procedures.
- Hadoop requires cluster administrators with knowledge of HDFS, YARN, MapReduce, and ecosystem tools.

Use Case Comparison

Use Case	RDBMS	Hadoop
Banking transactions	✓	✗
Inventory management	✓	✗
Social media analytics	✗	✓
Web clickstream analysis	✗	✓
Fraud detection (batch processing)	✗	✓
IoT sensor data storage	✗	✓

Hadoop Distributed File System (HDFS)

HDFS is the primary storage system of Hadoop designed to store very large datasets reliably across multiple commodity hardware nodes. It is highly fault-tolerant, scalable, and optimized for high-throughput batch processing.

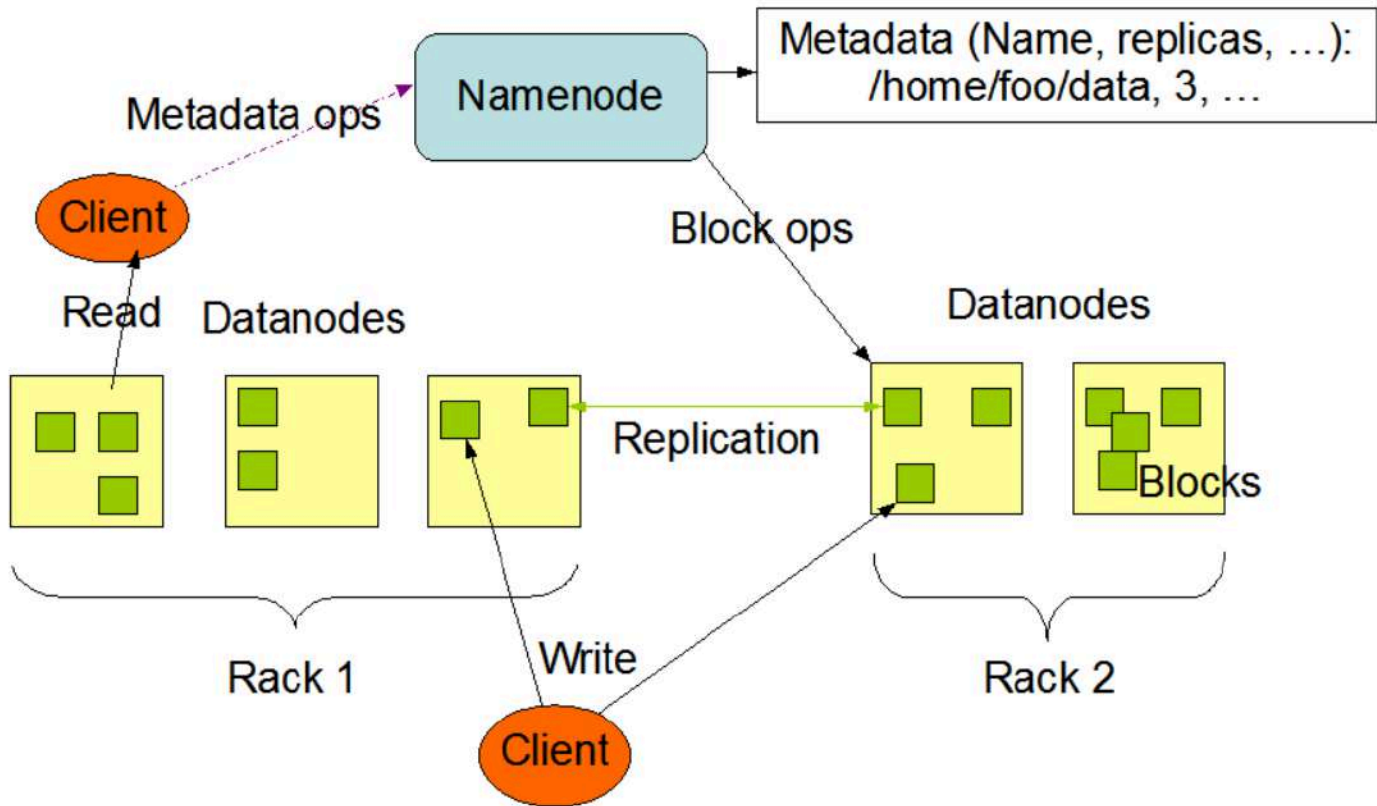
HDFS is inspired by Google File System (GFS) and is a key component of the Hadoop ecosystem.

Key Features of HDFS

1. Distributed Storage
 - Splits large files into blocks and distributes them across multiple nodes.
2. Fault Tolerance
 - Each block is replicated (default: 3 copies) across different DataNodes.
 - Ensures data availability even if a node fails.
3. Scalability
 - Horizontally scalable by adding more DataNodes.
 - Can store petabytes of data.
4. High Throughput
 - Optimized for large file reads and writes, not for random small reads/writes.

5. Cost-Effective
 - Runs on commodity hardware, reducing storage costs.
6. Flexibility
 - Handles structured, semi-structured, and unstructured data.

HDFS Architecture



HDFS has a master-slave architecture consisting of:

1. NameNode (Master)
2. DataNode (Slave)

Additionally, Secondary NameNode assists in metadata management.

1. NameNode

- Role: Master server that manages the file system namespace and metadata.
- Responsibilities:
 - Stores file names, directory structure, block locations, and permissions.
 - Keeps track of which DataNodes store which blocks.
 - Manages cluster health and file system operations (create, delete, rename).
- Important Notes:
 - Critical Node: If NameNode fails, HDFS becomes inaccessible.
 - Backup/HA NameNode can be configured in modern Hadoop.

2. DataNode

- Role: Slave node that stores actual data blocks.
- Responsibilities:
 - Handles read/write requests from clients.
 - Sends heartbeat signals to NameNode to report status.
 - Performs block replication, deletion, and creation as instructed by NameNode.
- Note: A typical HDFS cluster has many DataNodes to store data in a distributed manner.

3. Secondary NameNode

- Role: Assists the primary NameNode, not a real backup.
- Function:
 - Periodically merges edit logs and file system image to reduce NameNode restart time.
 - Helps in checkpointing NameNode metadata.

HDFS File Storage Mechanism

1. Large files are split into fixed-size blocks (default: 128 MB).
2. Each block is replicated across multiple DataNodes (default: 3).
3. NameNode maintains metadata about blocks and their locations.
4. Clients communicate with NameNode to get block locations, then read/write data directly from DataNodes.

Example:

- A 1 TB file → divided into 128 MB blocks → ~8,000 blocks
- Each block has 3 replicas → total storage ~3 TB distributed across nodes.

HDFS Read/Write Process

Reading a File

1. Client requests file read from NameNode.
2. NameNode provides block locations.
3. Client reads data directly from DataNodes in parallel.

Writing a File

1. Client requests file write from NameNode.
2. NameNode provides DataNodes for block storage.
3. Client writes blocks to the first DataNode → replicated to other DataNodes.
4. DataNodes confirm write completion to the NameNode.

HDFS Advantages

1. Fault Tolerance: Automatic replication of data blocks.
2. High Throughput: Designed for large-scale batch processing.
3. Scalable: Can add more nodes without downtime.
4. Cost-Effective: Runs on commodity hardware.
5. Supports Big Data Types: Handles structured, semi-structured, and unstructured data.

HDFS Limitations

1. Not suitable for low-latency or real-time processing.
2. Not efficient for many small files (NameNode memory overhead).
3. Single NameNode dependency (although HA mitigates this).
4. High disk space usage due to replication (default: 3x).
5. Limited transaction support (not ACID-compliant).

Key Takeaways

- HDFS is the backbone of Hadoop for distributed storage.
- Stores large files across multiple DataNodes with replication.
- NameNode manages metadata; DataNodes store blocks.
- Optimized for throughput, scalability, and fault tolerance, but not for real-time operations or small files.

Processing Data with Hadoop

Hadoop is not just a storage system (HDFS); it is a complete ecosystem for processing large-scale data efficiently. The processing is designed to handle massive datasets distributed across clusters in a parallel and fault-tolerant manner.

Hadoop provides several frameworks to process data:

1. MapReduce (Batch processing)
2. Apache Hive (SQL-like queries)
3. Apache Pig (Scripting for data transformation)
4. Apache Spark (In-memory and real-time processing)
5. Other ecosystem tools (HBase, Flume, Kafka for streaming, etc.)

1. MapReduce – Core Processing Framework

MapReduce is the original processing engine for Hadoop, designed to process large datasets in parallel.

Workflow:

1. Input Splitting: HDFS splits large files into blocks (default 128 MB).
2. Map Phase:
 - Processes input blocks in parallel.
 - Produces key-value pairs.
 - Example: Counting words → emits (word, 1) for each word.
3. Shuffle and Sort:
 - Groups all values by key across the cluster.
 - Prepares data for reduced tasks.
4. Reduce Phase:
 - Aggregates results based on key.
 - Example: Sum counts for each word.
5. Output:
 - Writes final results back to HDFS.

Pros: Fault-tolerant, scalable, reliable.

Cons: High latency, complex programming for beginners.

2. Apache Hive – SQL-Like Query Processing

Hive allows users to process Hadoop data using HiveQL, a SQL-like language. It converts queries into MapReduce, Tez, or Spark jobs automatically.

Workflow:

1. The user writes a HiveQL query.
2. Hive Driver receives a query.
3. The query is compiled, optimized, and converted into an execution plan.
4. The execution engine runs the job on the Hadoop cluster.
5. Results are returned to the user.

Pros: Easy for analysts, abstracts MapReduce complexity.

Cons: Batch-oriented, slower for real-time queries.

3. Apache Pig – Scripting for ETL

Pig uses a high-level scripting language (Pig Latin) for data transformations.

Workflow:

1. User writes Pig scripts (load → transform → store).
2. Pig compiler converts scripts into MapReduce jobs.
3. Jobs run on the Hadoop cluster.
4. Output is written to HDFS.

Pros: Suitable for ETL and complex data transformations.

Cons: Less interactive, batch processing only.

4. Apache Spark – In-Memory Processing

Spark provides faster processing than MapReduce by performing in-memory computations.

Workflow:

1. Load data from HDFS, HBase, Hive, or Kafka.
2. Apply transformations and actions using Spark API (RDD, DataFrame, Dataset).
3. Execute parallel processing tasks across clusters.
4. Save results back to HDFS or external storage.

Pros:

- Real-time and batch processing
- Iterative algorithms (ML, graph processing)
- Supports Python, Scala, Java, R

Cons: Requires more memory than MapReduce.

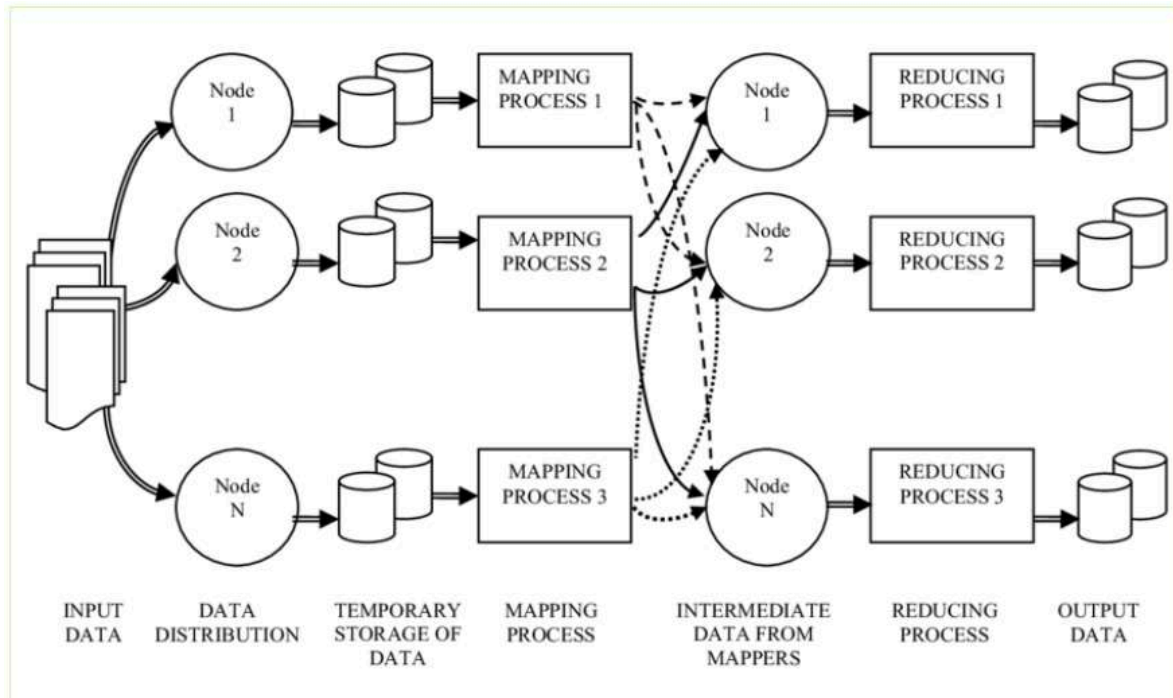
5. Real-Time and Streaming Processing

Hadoop ecosystem supports streaming and near real-time processing through tools like:

- Apache Flume: Collects and moves streaming log/event data to HDFS.
- Apache Kafka: Distributed messaging system for streaming data.
- Apache Storm / Spark Streaming: Processes real-time data streams.

Use Case: Twitter analytics, fraud detection, IoT sensor data.

Processing Flow Diagram



Best Practices for Processing Data with Hadoop

1. Use large files to reduce NameNode metadata overhead.
2. Partition and bucket tables in Hive for faster query performance.
3. Choose the right processing engine:
 - Batch → MapReduce
 - ETL → Pig
 - SQL analysis → Hive
 - Real-time / iterative → Spark
4. Monitor cluster resources via YARN to avoid task failures.
5. Compress data for faster I/O.

✓ Key Takeaways

- Hadoop provides multiple frameworks for processing: MapReduce, Hive, Pig, Spark.
- MapReduce is batch-oriented, Hive and Pig simplify coding, Spark allows in-memory and real-time processing.
- Data flows from sources → ingestion → HDFS → processing → output.
- Correct choice of framework and optimization ensures high throughput and low latency.

Hadoop YARN

YARN (Yet Another Resource Negotiator) is the resource management and job scheduling layer of Hadoop 2.x and above.

Before YARN:

- Hadoop 1.x used MapReduce to manage both processing and resources, which limited the cluster to only MapReduce jobs.

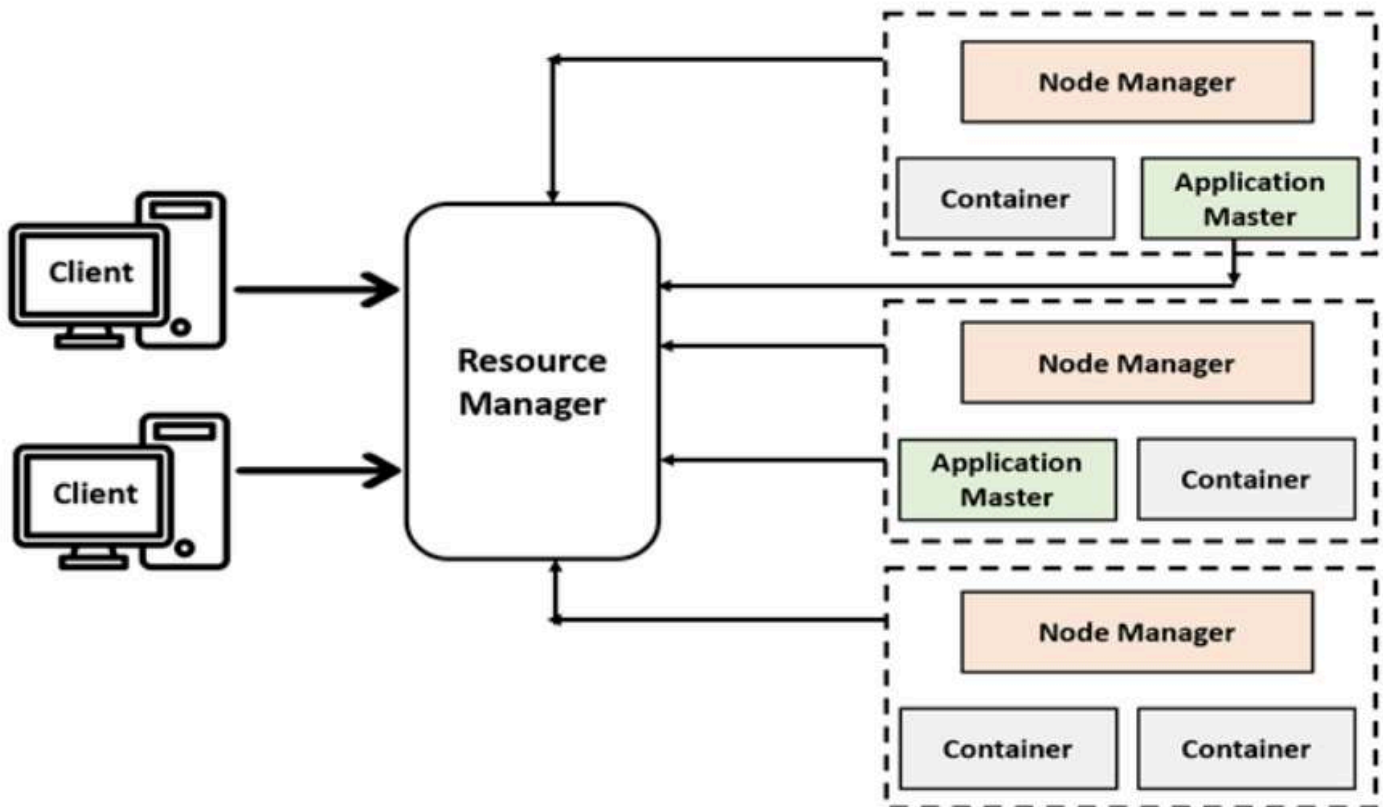
With Hadoop 2.x and YARN:

- Resource management is separated from job execution, allowing multiple processing frameworks (MapReduce, Spark, Tez) to run on the same cluster.
- Provides scalable, efficient, and flexible resource allocation.

Purpose of YARN:

- Manage cluster resources (CPU, memory) efficiently.
- Schedule and monitor user applications.
- Support multiple data processing frameworks simultaneously.

YARN Architecture



YARN has a master-slave architecture consisting of four main components:

1. ResourceManager (Master)
2. NodeManager (Slave)
3. ApplicationMaster (AM)
4. Containers

1. ResourceManager (RM)

- Role: Cluster-level resource manager (master daemon).
- Responsibilities:
 - Tracks resources across all nodes.
 - Allocates containers to applications based on available resources.
 - Implements scheduling policies (FIFO, Capacity Scheduler, Fair Scheduler).
- Components:
 - Scheduler: Allocates resources to applications without monitoring execution.
 - ApplicationManager: Manages job submissions and monitors ApplicationMasters.

Analogy: RM is like a cluster resource boss assigning CPU/memory to applications.

2. NodeManager (NM)

- Role: Slave daemon running on each node in the cluster.
- Responsibilities:
 - Manages resources of the node (CPU, memory, disk).
 - Launches containers on behalf of the ResourceManager.
 - Monitors container resource usage and reports back to RM.

Analogy: NM is like a node-level worker reporting to the boss (RM).

3. ApplicationMaster (AM)

- Role: Per-application job manager.
- Responsibilities:
 - Negotiates resources with RM.
 - Monitors execution of tasks in containers.
 - Handles job-specific scheduling, retries, and progress reporting.

Key Point: Each job (MapReduce, Spark, etc.) has its own ApplicationMaster.

4. Containers

- Role: Logical resource units (CPU + Memory) assigned by RM.
- Responsibilities:
 - Run tasks of the application inside the allocated resource limits.
 - Monitored by NodeManager.

Example:

- Job requires 4 CPUs + 8 GB RAM → RM allocates containers across nodes with required resources.

YARN Workflow

1. User submits an application/job to ResourceManager.
2. RM contacts ApplicationManager and starts ApplicationMaster for that job.
3. ApplicationMaster negotiates containers from RM for executing tasks.
4. NodeManagers launch containers and run tasks.
5. Containers report progress and status back to ApplicationMaster.
6. ApplicationMaster updates RM about job completion.
7. Resources are released for other applications.

YARN Advantages

1. Scalability: Supports clusters with thousands of nodes.
2. Resource Utilization: Dynamically allocates CPU and memory to jobs.
3. Multi-Tenancy: Multiple frameworks (MapReduce, Spark, Tez) can run simultaneously.
4. Fault Tolerance: Handles node failures and retries tasks in new containers.
5. Better Performance: Separates resource management from job execution, reducing bottlenecks of Hadoop 1.x.

YARN vs Hadoop 1.x MapReduce

Feature	Hadoop 1.x	Hadoop 2.x (YARN)
Resource Management	Part of MapReduce	Separate via YARN RM
Multi-Framework Support	Only MapReduce	MapReduce, Spark, Tez, Flink, etc.
Scalability	Limited	Highly scalable (thousands of nodes)
Fault Tolerance	Job failure causes resubmission	Node failures handled; jobs retried automatically
Resource Allocation	Static, fixed slots	Dynamic, container-based allocation

MapReduce Programming

MapReduce is the core data processing paradigm in the Hadoop ecosystem. It allows processing large datasets distributed across a Hadoop cluster in a parallel, fault-tolerant, and scalable way.

It was inspired by Google's MapReduce paper (2004) and implemented in Hadoop for batch processing of big data.

Key Idea:

Divide a job into two main tasks: Map (filtering, sorting, transforming) and Reduce (aggregation, summarization).

MapReduce Programming Components

1. Map Function
2. Shuffle and Sort
3. Reduce Function

1. Map Function

- Purpose: Processes input data record by record and produces intermediate key-value pairs.
- Input: Data from HDFS (text files, CSV, JSON, etc.)
- Output: Key-value pairs (**key**, **value**) for further processing

Example: Word Count problem

- Input: "Hadoop is big data. Hadoop is scalable."
- Map output:

```
(Hadoop, 1)
(is, 1)
(big, 1)
(data, 1)
(Hadoop, 1)
(is, 1)
(scalable, 1)
```

2. Shuffle and Sort

- Purpose: Intermediate phase that:
 1. Groups all values by key
 2. Sorts the keys for efficient Reduce operation
- Example: For word count, all instances of "Hadoop" are grouped:

```
(Hadoop, [1,1])
(is, [1,1])
(big, [1])
(data, [1])
(scalable, [1])
```

3. Reduce Function

- Purpose: Aggregates or summarizes values of the same key to produce final output.
- Input: Key and list of values from shuffle & sort phase.
- Output: Final result stored in HDFS

Example: Continuing word count:

```
(Hadoop, 2)
(is, 2)
(big, 1)
(data, 1)
(scalable, 1)
```

MapReduce Workflow

1. The user submits a MapReduce job.
2. Hadoop splits input files into input splits (blocks) and sends them to map tasks.
3. Map tasks process records and produce intermediate key-value pairs.
4. Shuffle & Sort groups keys and sorts them.
5. Reduce tasks, aggregate data and produce final results.
6. Output is written back to HDFS.

MapReduce Programming Structure (Java Example)

1. Mapper Class – Defines Map logic

```
public class WordCountMapper extends Mapper<LongWritable, Text, Text,
IntWritable> {
    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();

    public void map(LongWritable key, Text value, Context context) throws
IOException, InterruptedException {
        String[] words = value.toString().split(" ");
        for(String w: words) {
            word.set(w);
            context.write(word, one);
        }
    }
}
```

2. Reducer Class – Defines Reduce logic

```
public class WordCountReducer extends Reducer<Text, IntWritable, Text,
IntWritable> {
    public void reduce(Text key, Iterable<IntWritable> values, Context
context) throws IOException, InterruptedException {
        int sum = 0;
        for(IntWritable val : values) {
            sum += val.get();
        }
        context.write(key, new IntWritable(sum));
    }
}
```

3. Driver Class – Configures and runs the job

```
public class WordCountDriver {
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "word count");
        job.setJarByClass(WordCountDriver.class);
        job.setMapperClass(WordCountMapper.class);
        job.setReducerClass(WordCountReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

MapReduce Features

1. Parallel Processing: Jobs are split into multiple tasks running on different nodes.
2. Fault Tolerance: Failed tasks are automatically re-executed on other nodes.
3. Scalability: Handles petabytes of data across thousands of nodes.
4. Data Locality: Map tasks run close to data blocks on HDFS to reduce network traffic.
5. High Throughput: Optimized for large-scale batch processing.

Advantages of MapReduce

- Handles very large datasets efficiently.
- Easy to scale horizontally by adding nodes.
- Fault-tolerant and reliable.
- Integrates seamlessly with HDFS.

Limitations of MapReduce

- Not suitable for real-time processing (high latency).
- Iterative algorithms (ML, graph processing) are slow due to repeated HDFS I/O.
- Complex programming in Java (Hive, Pig, or Spark can simplify this).
- Poor for many small files; optimized for large files.

MapReduce Use Cases

1. Word Count – Text analysis.
2. Log Analysis – Web server log processing.
3. Data Transformation – ETL tasks.
4. Analytics – Counting, grouping, and aggregating large datasets.
5. Machine Learning – Initial preprocessing of large-scale datasets.

✓ Key Takeaways

- MapReduce is the heart of Hadoop's processing engine.
- Map phase filters and transforms data; Reduce phase aggregates results.
- Works efficiently on large, distributed datasets, but not ideal for real-time or iterative tasks.
- Other tools like Hive, Pig, Spark can simplify programming and improve performance.

UNIT-3

Introduction to Hive

Apache Hive is a data warehouse framework built on top of Hadoop. It allows data summarization, querying, and analysis of large datasets stored in HDFS using a SQL-like language called HiveQL.

Hive was developed by Facebook to simplify Big Data querying without writing complex MapReduce programs.

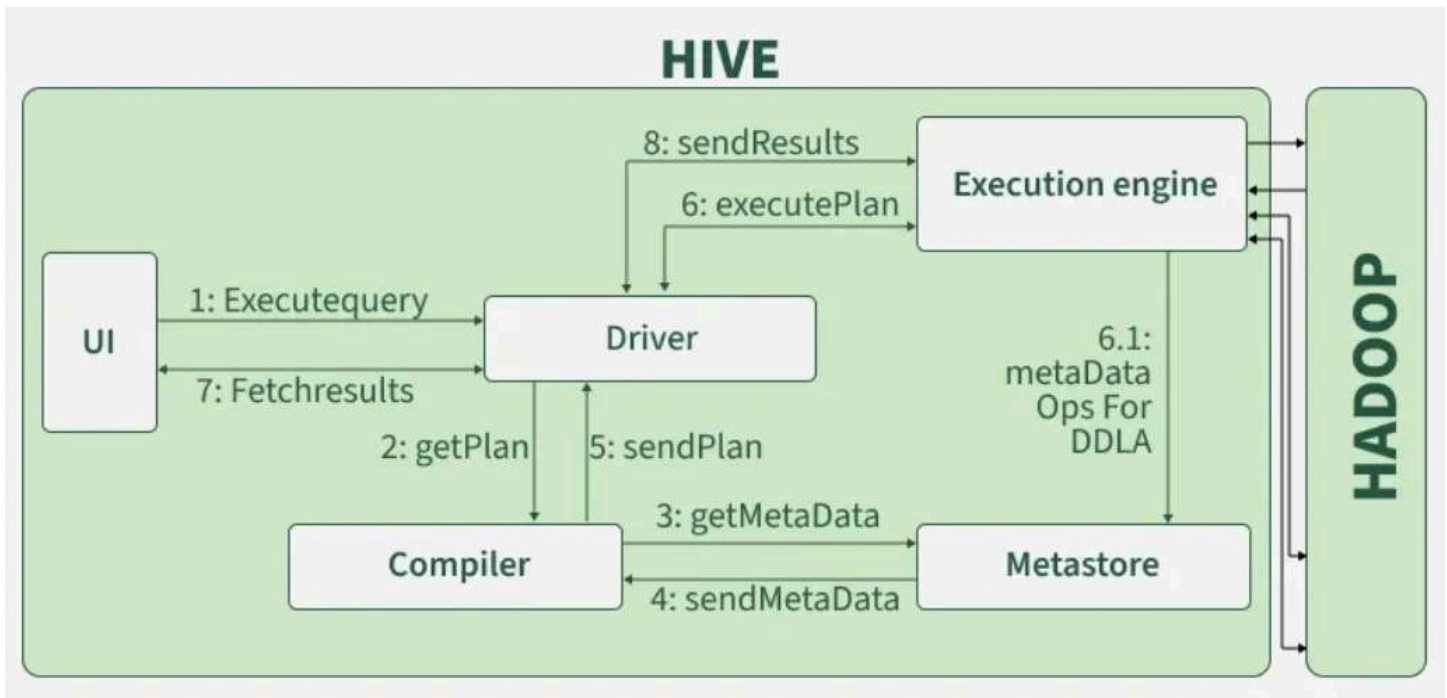
Key Features of Hive:

1. SQL-like Query Language (HiveQL): Makes it easier for analysts familiar with SQL.
2. Runs on Hadoop: Translates queries into MapReduce, Tez, or Spark jobs.
3. Handles Large Datasets: Efficiently processes structured and semi-structured data stored in HDFS.
4. Extensible: Supports custom UDFs (User Defined Functions) for complex processing.
5. Schema on Read: Data can be stored in raw format, and schema is applied at query time.

Use Cases of Hive:

- Data summarization and aggregation
- Reporting and analytics on large datasets
- ETL (Extract, Transform, Load) operations
- Log analysis and web analytics

Hive Architecture



Hive architecture consists of several layers, including user interface, compiler, execution engine, metastore, and storage layer.

1. User Interface Layer

Hive provides multiple interfaces for users to interact with the system:

1. Hive CLI (Command Line Interface) – Classic interface to execute HiveQL queries.
2. Beeline – JDBC client to connect to HiveServer2.
3. Web UI – Web-based interface for monitoring and executing queries.
4. Thrift/ODBC/JDBC APIs – For external applications and BI tools.

2. Hive Driver

- Role: Acts as a controller for the Hive execution process.
- Responsibilities:
 - Receives HiveQL queries from the user.
 - Passes queries to the compiler for parsing and optimization.
 - Maintains session handles and monitors query execution.

3. Compiler

- Role: Converts HiveQL queries into a series of MapReduce/Tez/Spark jobs.
- Responsibilities:
 - Parsing: Converts HiveQL into an abstract syntax tree (AST).
 - Semantic Analysis: Validates the query against the metadata in the metastore.
 - Logical Plan Generation: Creates logical execution plan.
 - Optimization: Applies optimizations such as predicate pushdown, column pruning, join reordering.
 - Physical Plan Generation: Converts logical plans into MapReduce/Tez/Spark jobs.

4. Execution Engine

- Role: Executes the physical plan generated by the compiler.
- Responsibilities:
 - Submits MapReduce/Tez/Spark jobs to the Hadoop cluster.
 - Coordinates execution and monitors task progress and failures.
 - Collects output and returns results to the Hive driver.

5. Metastore

- Role: Central repository of metadata (database, tables, columns, data types, HDFS locations, partitions).
- Responsibilities:
 - Stores schema information and table definitions.
 - Maintains partitions, buckets, and indexes.
 - Enables query compilation and optimization by providing metadata.
- Types:
 - Embedded Metastore: Runs in the same JVM as Hive (for testing/small deployments).
 - Standalone Metastore: Runs as a separate service (for production, supports multiple Hive clients).

6. Storage Layer

- Hive stores actual data in HDFS (Hadoop Distributed File System).
- Data formats supported:
 - Text files, CSV, JSON
 - SequenceFile, ORC (Optimized Row Columnar), Parquet
- Hive does not manage data physically; it references files in HDFS.

Hive Processing Flow

1. The user submits a HiveQL query via CLI, Beeline, or JDBC.
2. The driver receives the query and passes it to the compiler.
3. Compiler parses, validates, and generates execution plans.
4. Execution Engine executes MapReduce/Tez/Spark jobs on Hadoop cluster.
5. Data is fetched from HDFS, processed, and written back to HDFS if needed.
6. The query result is returned to the user.

Advantages of Hive

1. SQL-like Interface: Easy for analysts familiar with SQL.
2. Handles Large Datasets: Efficiently processes petabytes of data.
3. Extensible: Supports UDFs, UDAFs, UDTFs.
4. Integrates with Hadoop Ecosystem: Works with HDFS, HBase, Spark.
5. Schema-on-Read: Data can be raw; schema applied during query execution.

Limitations of Hive

1. Not suitable for real-time queries (high latency due to MapReduce).
2. Inefficient for small datasets or low-latency requirements.
3. Complex queries with joins and nested queries can be slow.
4. Update/Delete operations are limited (Hive is mostly append-only).



Hive Data Types

In Hive, every column in a table is assigned a data type.

Data types in Hive are divided into primitive types (basic) and complex types (collections and structures).

Hive supports structured and semi-structured data, making it flexible for Big Data analytics.

1. Primitive Data Types

Primitive types represent basic single-value types.

Data Type	Description	Example
TINYINT	1-byte signed integer	100
SMALLINT	2-byte signed integer	32000
INT / INTEGER	4-byte signed integer	100000
BIGINT	8-byte signed integer	10000000000
FLOAT	4-byte single-precision floating-point	3.14
DOUBLE	8-byte double-precision floating-point	3.1415926

BOOLEAN	True/False values	TRUE, FALSE
STRING	Sequence of characters	"Hello Hive"
CHAR(n)	Fixed-length string of n characters	CHAR(10) → "Hive "
VARCHAR(n)	Variable-length string up to n characters	VARCHAR(20) → "HiveSQL"
TIMESTAMP	Date and time value	2025-11-26 10:30:00
DATE	Date value only	2025-11-26
BINARY	Raw binary data	Used for images, files, etc.
DECIMAL / NUMERIC	Arbitrary-precision number	DECIMAL(10,2) → 12345678.90

Notes:

- Use INT for integer values unless very large numbers are required → BIGINT.
- Use DECIMAL for precise financial calculations to avoid floating-point errors.

2. Complex Data Types

Complex types allow storing multiple values, nested structures, or key-value pairs.

a) ARRAY

- Represents an ordered list of elements.
- All elements must be of the same type.
- Syntax:

`ARRAY<element_type>`

- Example:

```
CREATE TABLE students (
  name STRING,
  marks ARRAY<INT>
);
```

- Usage:

```
SELECT name, marks[0] FROM students;
```

b) MAP

- Represents a key-value pair collection.
- Keys and values can be of any Hive type, but keys must be unique.
- Syntax:

MAP<key_type, value_type>

- Example:

```
CREATE TABLE employee (  
  name STRING,  
  properties MAP<STRING, STRING>  
);
```

- Usage:

```
SELECT properties['department'] FROM employee;
```

c) STRUCT

- Represents a record with multiple fields, each field can be of a different type.
- Syntax:

STRUCT<field1:type1, field2:type2, ...>

- Example:

```
CREATE TABLE person (  
  name STRING,  
  address STRUCT<city:STRING, zip:INT>  
);
```

- Usage:

```
SELECT address.city FROM person;
```

d) UNIONTYPE

- Represents a field that can hold different types, but only one value at a time.
- Syntax:

UNIONTYPE<type1, type2, ...>

- Example:

```
CREATE TABLE mixed_data (  
  info UNIONTYPE<INT, STRING>  
);
```

- Usage:
- Useful when a column can have multiple possible types.

3. Collection Type Nesting

Hive supports nested complex types:

Example:

```
CREATE TABLE nested_example (  
  id INT,  
  info ARRAY<STRUCT<name:STRING, scores:MAP<STRING,INT>>>  
>);
```

- Here:
 - `info` is an array of structs.
 - Each struct contains a name and a map of scores.

Query Example:

```
SELECT info[0].scores['math'] FROM nested_example;
```

4. Choosing Data Types in Hive

Requirement	Recommended Type
Whole numbers (small)	INT
Whole numbers (large)	BIGINT
Decimal numbers	DECIMAL
True/False	BOOLEAN
Text/String	STRING, VARCHAR(n), CHAR(n)
Date/Time	DATE, TIMESTAMP
Key-Value	MAP
List/Array	ARRAY
Structured Record	STRUCT
Multiple possible types	UNIONTYPE

Introduction to Hive Query Language (HiveQL)

HiveQL is a SQL-like language used to query and manage large datasets stored in Hadoop HDFS through Hive.

- Designed for data analysts and developers familiar with SQL.
- Automatically translates queries into MapReduce, Tez, or Spark jobs for execution on Hadoop.
- Supports data definition, data manipulation, and data query operations.

Key Feature: HiveQL uses schema-on-read, meaning the schema is applied when reading the data, not while writing.

1. Types of Hive Queries

HiveQL supports three main categories of queries:

1. Data Definition Language (DDL) – Create, alter, or drop tables/databases.
2. Data Manipulation Language (DML) – Load, insert, update, or delete data in tables.
3. Data Query Language (DQL) – Select or retrieve data with filtering, grouping, and sorting.

1.1 Data Definition Language (DDL)

DDL commands manage databases, tables, and schemas.

Command	Description	Example
CREATE DATABASE	Creates a new database	CREATE DATABASE company_db;
DROP DATABASE	Deletes a database	DROP DATABASE company_db;
CREATE TABLE	Creates a table in Hive	CREATE TABLE employees (id INT, name STRING, salary FLOAT) ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' STORED AS TEXTFILE;
DROP TABLE	Deletes a table	DROP TABLE employees;
ALTER TABLE	Modifies table structure	ALTER TABLE employees ADD COLUMNS (department STRING);
DESCRIBE TABLE	Shows table schema	DESCRIBE employees;
SHOW TABLES	Lists all tables in database	SHOW TABLES;
SHOW DATABASES	Lists all databases	SHOW DATABASES;

1.2 Data Manipulation Language (DML)

DML commands insert, load, or modify data in Hive tables.

Command	Description	Example
LOAD DATA	Loads data from HDFS or local file system	LOAD DATA INPATH '/user/hadoop/emp.txt' INTO TABLE employees;
INSERT INTO	Inserts data into a table	INSERT INTO TABLE employees VALUES (1, 'John', 50000);
INSERT OVERWRITE	Replaces existing data in table	INSERT OVERWRITE TABLE employees SELECT * FROM temp_employees;
UPDATE	Updates data (Hive 0.14+)	UPDATE employees SET salary = 60000 WHERE id = 1;
DELETE	Deletes rows (Hive 0.14+)	DELETE FROM employees WHERE id = 1;

Note: Hive is primarily append-only, so UPDATE and DELETE are limited and slower compared to traditional RDBMS.

1.3 Data Query Language (DQL)

DQL commands retrieve data from tables.

Clause/Command	Description	Example
SELECT	Retrieves columns	SELECT name, salary FROM employees;
WHERE	Filters rows based on condition	SELECT * FROM employees WHERE salary > 50000;
GROUP BY	Groups data	SELECT department, COUNT(*) FROM employees GROUP BY department;
ORDER BY	Sorts results	SELECT * FROM employees ORDER BY salary DESC;
LIMIT	Limits number of results	SELECT * FROM employees LIMIT 5;

JOIN	Combines data from multiple tables	<code>SELECT e.name, d.name FROM employees e JOIN department d ON e.department_id = d.id;</code>
HAVING	Filters aggregated data	<code>SELECT department, COUNT(*) FROM employees GROUP BY department HAVING COUNT(*) > 5;</code>
DISTINCT	Returns unique values	<code>SELECT DISTINCT department FROM employees;</code>

2. Partitioning and Bucketing Queries

Hive supports partitioning and bucketing to improve query performance:

- Partitioning: Divides table data into subdirectories based on column values.

```
CREATE TABLE sales (
  id INT,
  product STRING,
  amount FLOAT
) PARTITIONED BY (year INT, month INT);
```

- Bucketing: Divides data into a fixed number of files (buckets) based on the hash of a column.

```
CREATE TABLE students (
  id INT,
  name STRING
) CLUSTERED BY (id) INTO 4 BUCKETS;
```

3. HiveQL Functions

Hive supports built-in functions for data processing:

1. Aggregate Functions: `SUM()`, `COUNT()`, `AVG()`, `MAX()`, `MIN()`
2. String Functions: `CONCAT()`, `UPPER()`, `LOWER()`, `SUBSTRING()`
3. Date Functions: `CURRENT_DATE()`, `YEAR()`, `MONTH()`, `DATEDIFF()`
4. Mathematical Functions: `ROUND()`, `FLOOR()`, `CEIL()`
5. Conditional Functions: `IF()`, `COALESCE()`, `CASE WHEN ... THEN ... ELSE ... END`

Example:

```
SELECT department, AVG(salary) as avg_salary
FROM employees
WHERE salary > 30000
GROUP BY department
ORDER BY avg_salary DESC;
```


4. Views in Hive

- Views are virtual tables created from a query.
- Syntax:

```
CREATE VIEW high_salary AS
SELECT name, salary
FROM employees
WHERE salary > 50000;
```

- Can query like a regular table:

```
SELECT * FROM high_salary;
```

5. Advantages of HiveQL

1. SQL-like syntax simplifies Big Data queries.
2. Supports large-scale data analysis using Hadoop.
3. Integrates with MapReduce, Tez, Spark automatically.
4. Supports partitioning and bucketing for faster queries.
5. Compatible with BI tools via JDBC/ODBC.

6. Limitations of HiveQL

1. Not suitable for real-time or transactional queries.
2. Slow for small datasets due to MapReduce overhead.
3. Limited support for UPDATE/DELETE operations.
4. Complex nested queries can be slow and resource-intensive.

✓ Key Takeaways

- HiveQL is a SQL-like language for querying and managing Hadoop data.
- Supports DDL, DML, and DQL operations.
- Works well for data summarization, analytics, and ETL on large datasets.
- Can be extended with UDFs, UDAFs, and custom SerDes for advanced processing.

Introduction to Apache Pig

Apache Pig is a high-level platform for creating programs that run on Hadoop. It provides a simpler scripting language called Pig Latin for processing and analyzing large datasets stored in HDFS.

- Developed by Yahoo! to make Hadoop data processing easier and more accessible to developers who may not be familiar with Java or MapReduce.
- Pig programs are translated into MapReduce jobs (or Tez/Spark jobs) that run on Hadoop clusters.

Key Idea:

Pig abstracts the complexity of writing low-level MapReduce programs, allowing developers to focus on data transformations using a script-like syntax.

Key Features of Apache Pig

1. Ease of Programming: Pig Latin is simple, declarative, and procedural, reducing coding complexity.
2. Extensibility: Supports user-defined functions (UDFs) in Java, Python, or other languages.
3. Optimization: Pig automatically optimizes execution plans for better performance.
4. Handles Large Datasets: Efficiently processes petabytes of structured and semi-structured data.
5. Data Flow Language: Pig scripts specify data flows rather than low-level MapReduce tasks.
6. Flexible: Can process structured, semi-structured, and unstructured data.

Pig Components

Pig consists of several components that help process data:

1. Pig Latin Language – High-level scripting language.
2. Pig Engine – Translates Pig Latin scripts into MapReduce/Tez/Spark jobs.
3. Pig Execution Modes – Determines how scripts are executed.
4. Pig Storage – Interfaces with HDFS and other storage systems.

1. Pig Latin Language

- Pig Latin is a data flow language that lets you define a series of transformations on datasets.
- Data Transformations Include:
 - Loading data from HDFS
 - Filtering, grouping, joining, or sorting data
 - Storing the results back to HDFS

Example Pig Latin Script (Word Count):

```
-- Load input file
lines = LOAD '/user/hadoop/input.txt' AS (line:chararray);

-- Split lines into words
words = FOREACH lines GENERATE FLATTEN(TOKENIZE(line)) AS word;

-- Group by word
grouped = GROUP words BY word;

-- Count occurrences
word_count = FOREACH grouped GENERATE group, COUNT(words);

-- Store result
STORE word_count INTO '/user/hadoop/output';
```

- This script performs word count without writing a single MapReduce job manually.

2. Pig Execution Modes

Pig can run in two modes:

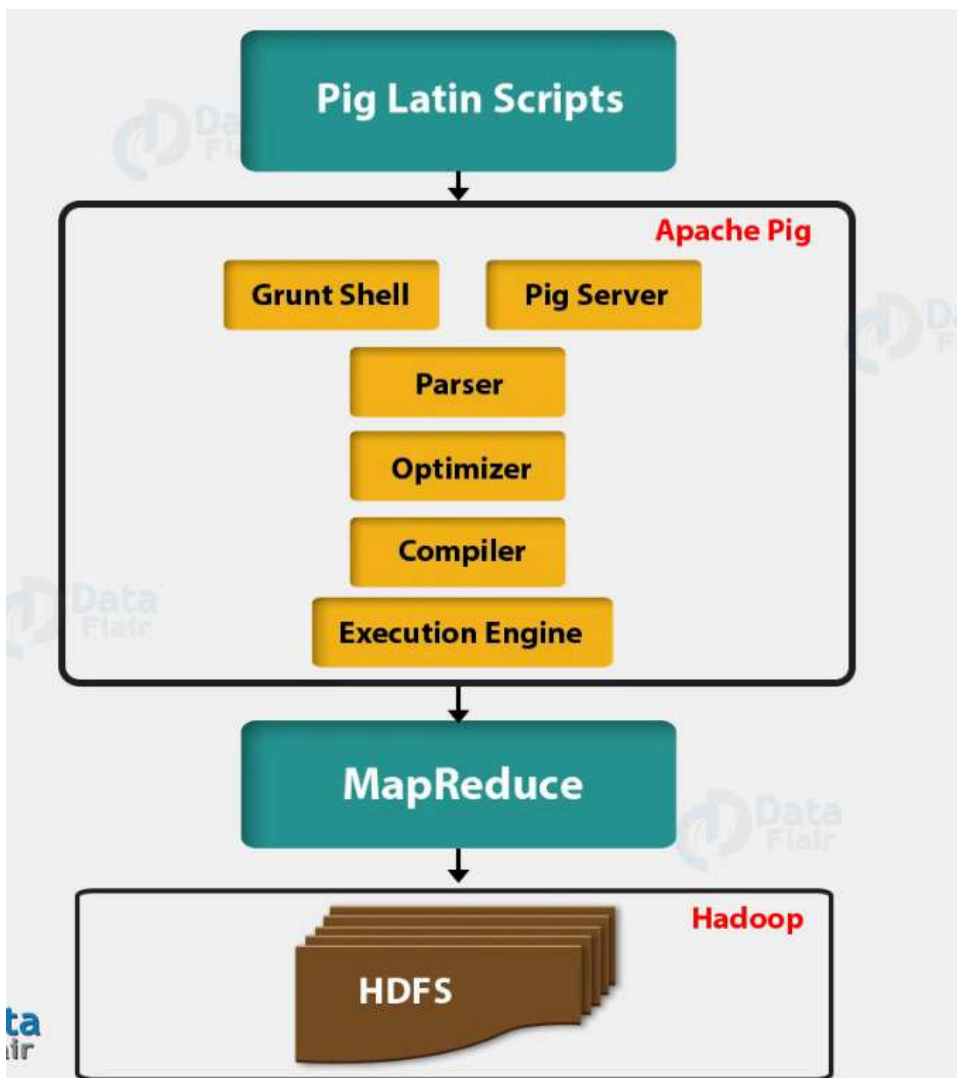
Mode	Description
Local Mode	Runs Pig on a single machine, using a local file system. Good for small datasets and testing.
MapReduce Mode (Hadoop Mode)	Runs Pig scripts on a Hadoop cluster, generates MapReduce jobs. Suitable for large-scale datasets.

3. Pig Components / Architecture

Pig's architecture can be divided into three layers:

1. Pig Latin Layer – Where users write scripts.
2. Execution Layer – Translates scripts into MapReduce jobs using the Pig Engine.
3. Storage Layer – Interacts with HDFS or other storage systems.

Architecture Diagram (Text Version):



4. Pig Data Model

Pig uses a nested data model that can handle structured and semi-structured data:

- Atom: Primitive types like `int`, `float`, `chararray`, `bytearray`.
- Tuple: Ordered set of fields (similar to a row).
- Bag: Collection of tuples (similar to a table).
- Map: Key-value pairs.

Example:

```
students = { (1, 'John', 90), (2, 'Alice', 85) } -- Bag of Tuples
```

5. Advantages of Pig

1. Simplifies Hadoop Programming: Reduces need for Java MapReduce code.
2. Handles Large Datasets: Efficient processing over HDFS and Hadoop clusters.
3. Flexible Data Processing: Can work with structured, semi-structured, and unstructured data.
4. Supports Extensibility: Developers can write UDFs for custom processing.
5. Optimized Execution: Pig optimizes the execution plan automatically.

6. Limitations of Pig

1. Not Ideal for Interactive Queries: Higher latency compared to Hive or Impala.
2. Steeper Learning Curve than SQL: Pig Latin is procedural, different from declarative SQL.
3. Limited Transaction Support: Pig is primarily for batch processing.
4. Debugging Can Be Tricky: Errors in scripts may require log inspection.

7. Use Cases of Pig

- Log processing and analysis
- ETL (Extract, Transform, Load) operations
- Data cleansing and filtering
- Large-scale analytics (clickstream, social media data)
- Preprocessing data for Machine Learning pipelines

Anatomy of Apache Pig

The anatomy of Pig describes how Pig scripts are processed, compiled, and executed on Hadoop. It explains the components of Pig and how they interact with each other and HDFS.

1. Pig Latin Script Layer

- This is the top layer, where users write Pig Latin scripts.
- Pig Latin is a high-level procedural language for specifying data transformations and analysis.
- Users can perform operations like LOAD, FILTER, JOIN, GROUP, FOREACH, and STORE.

Example Pig Latin Script (Word Count):

```
lines = LOAD '/user/hadoop/input.txt' AS (line:chararray);
words = FOREACH lines GENERATE FLATTEN(TOKENIZE(line)) AS word;
grouped = GROUP words BY word;
word_count = FOREACH grouped GENERATE group, COUNT(words);
STORE word_count INTO '/user/hadoop/output';
```

- At this layer, Pig focuses on what to compute rather than how to execute.

2. Parser

- The parser takes the Pig Latin script and checks syntax.
- Generates a logical plan that represents the sequence of operations in the script.
- Detects errors such as typos, invalid commands, or unsupported data types before execution.

Output: Logical Plan → a DAG (Directed Acyclic Graph) of operators.

3. Logical Plan

- The logical plan is an internal representation of the Pig script.
- It contains logical operators like LOAD, FILTER, JOIN, GROUP, and FOREACH.
- At this stage, no physical execution decisions are made; it's purely abstract.

Example:

For the word count script, the logical plan will include:

- LOAD → Tokenize → Flatten → GROUP → COUNT → STORE

4. Optimizer

- The optimizer improves the logical plan for better performance.
- Pig automatically applies logical and physical optimizations:
 1. Projection Pushdown: Only reads required columns.
 2. Filter Pushdown: Moves filters as close to data as possible.
 3. Combining Operations: Merges multiple operations into a single MapReduce job when possible.
 4. Reordering Joins: Optimizes join order to reduce data transfer.
- Output: Optimized logical plan ready for physical execution.

5. Physical Plan / Execution Plan

- Converts the logical plan into a physical plan with specific operators mapped to MapReduce jobs (or Tez/Spark jobs).
- Determines:
 - How many MapReduce jobs are needed?
 - How data flows between tasks.
 - How intermediate results are stored.

Output: Physical execution plan ready to run on Hadoop.

6. Execution Engine

- Executes the physical plan on the Hadoop cluster.
- Responsible for:
 - Submitting MapReduce/Tez/Spark jobs to Hadoop.
 - Monitoring job execution and progress.
 - Handling failures and task re-execution.
 - Writing final results to HDFS or other storage systems.

7. Storage Layer (HDFS / Local File System)

- Pig interacts with storage systems where the data resides:
 - HDFS (Hadoop Distributed File System): For distributed storage on clusters.
 - Local File System: For running Pig in local mode (single machine).
- Pig uses schema-on-read, meaning it applies schema to data at the time of query execution, not during storage.

8. Data Model in Pig

Pig supports a nested data model, which affects how the execution engine processes data:

Type	Description
Atom	Primitive data type (int, float, chararray, bytearray)
Tuple	Ordered collection of fields (like a row in a table)
Bag	Collection of tuples (like a table)
Map	Collection of key-value pairs

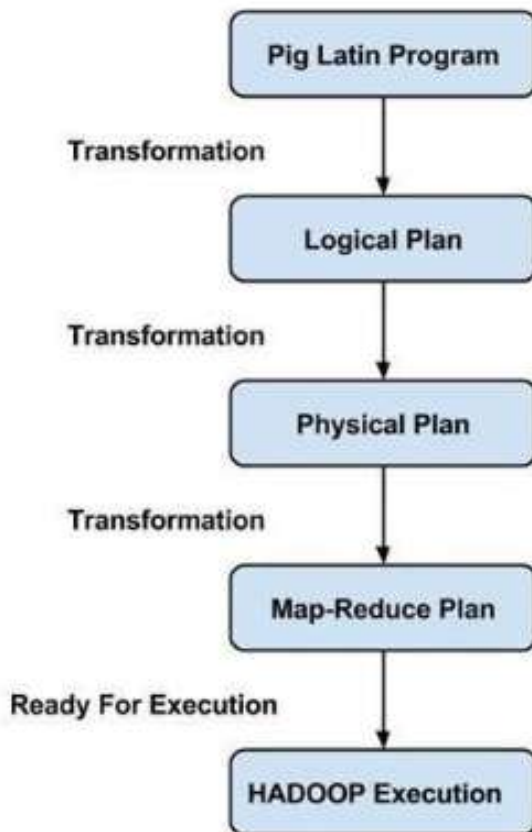
Example:

```
students = { (1, 'John', 90), (2, 'Alice', 85) }  -- Bag of Tuples
```

9. Pig Execution Flow (Step by Step)

1. Write Pig Script – User writes Pig Latin code.
2. Parser – Checks syntax and generates logical plans.
3. Optimizer – Optimizes logical plan (pushdown, join reordering, etc.).
4. Physical Plan Generation – Converts logical plans into MapReduce jobs.
5. Execution Engine – Submits jobs to Hadoop, monitors execution, and writes results.
6. Storage Layer – Reads input data from HDFS and writes output to HDFS.

Pig Anatomy Diagram



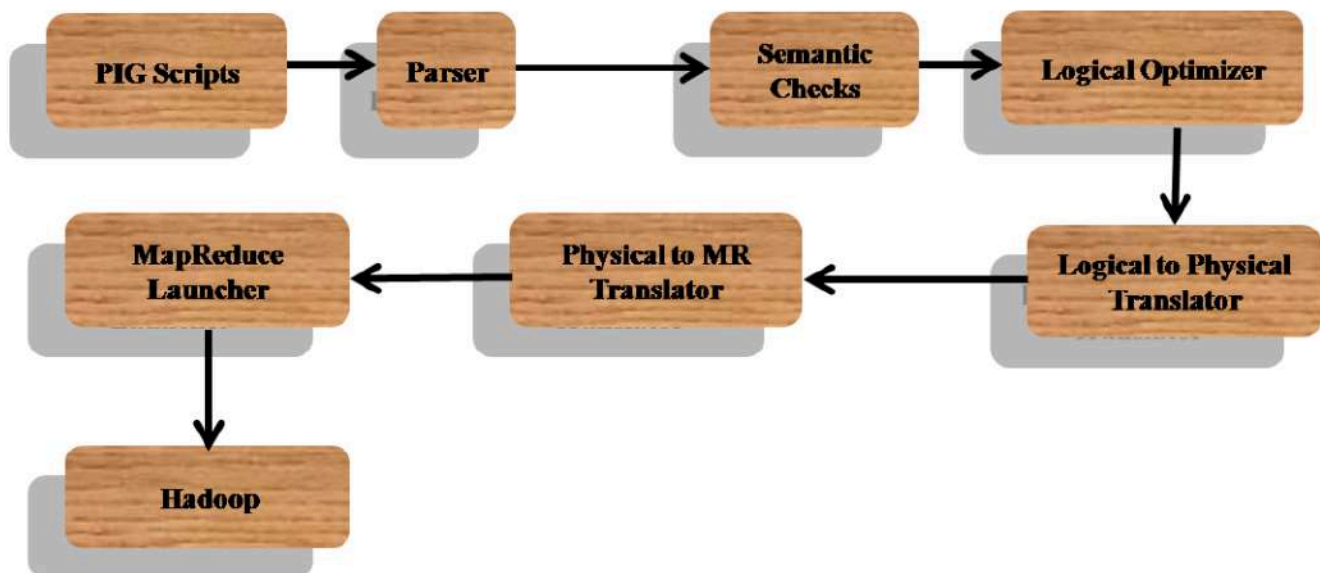
Apache Pig is a high-level platform for processing large datasets stored in Hadoop HDFS. When Pig runs on Hadoop, it translates Pig Latin scripts into MapReduce jobs (or other execution frameworks like Tez or Spark), which execute on the Hadoop cluster.

Key Idea: Pig provides simplified scripting for Hadoop, allowing developers to process big data without writing Java MapReduce programs.

1. How Pig Works on Hadoop

1. Pig Script Written: User writes a Pig Latin script specifying data transformations (LOAD, FILTER, JOIN, GROUP, etc.).
2. Parser and Logical Plan: Pig parses the script and generates a logical plan (abstract representation of operations).
3. Optimizer: Optimizes the logical plan (filter pushdown, join optimization, combining operations).
4. Physical Plan Generation: Converts logical plans into Hadoop MapReduce jobs (or Tez/Spark jobs).
5. Execution Engine: Submits MapReduce jobs to Hadoop YARN, monitors execution, and handles failures.
6. Data Storage: Reads data from HDFS and writes output back to HDFS.

Flow Diagram (Pig on Hadoop):



2. Pig Execution Modes on Hadoop

Pig supports two main execution modes:

Mode	Description
Local Mode	Pig runs on a single machine. Uses a local file system. Good for testing and small datasets.
MapReduce Mode (Hadoop Mode)	Pig runs on a Hadoop cluster. Translates scripts into MapReduce jobs, executed via YARN. Suitable for large-scale datasets.

Key Points for MapReduce Mode:

- Input and output are in HDFS.
- Pig scripts may generate multiple MapReduce jobs depending on transformations.
- Pig relies on Hadoop YARN for resource management and job scheduling.

3. Pig on Hadoop Architecture

Pig on Hadoop architecture includes the following layers:

1. Pig Latin Script Layer: User writes the script specifying transformations.
2. Parser & Logical Plan: Parses the script, creates an abstract representation.
3. Optimizer: Optimizes operations for efficient execution.
4. Physical Plan / MapReduce Jobs: Converts the plan into Hadoop MapReduce jobs.
5. Execution Engine: Executes MapReduce jobs on Hadoop cluster via YARN.
6. HDFS Storage Layer: Reads input data from HDFS and writes results back.

Data Flow Example:

- LOAD: Reads data from HDFS
- TRANSFORMATIONS (FILTER, JOIN, GROUP): Performed as intermediate MapReduce jobs
- STORE: Writes results back to HDFS

4. Advantages of Running Pig on Hadoop

1. Simplified Big Data Processing: No need to write Java MapReduce code.
2. Handles Large Datasets: Can process terabytes to petabytes of data stored in HDFS.
3. Optimized Execution: Pig automatically optimizes jobs for Hadoop execution.
4. Flexibility: Can process structured, semi-structured, and unstructured data.
5. Extensibility: Custom UDFs can be used in Pig scripts for advanced transformations.
6. Integration with Hadoop Ecosystem: Works well with Hive, HBase, HDFS, and other Hadoop tools.

5. Example – Pig on Hadoop

Suppose we have a log file stored in HDFS at `/user/hadoop/logs.txt` and want to count occurrences of each word:

```
-- Load data from HDFS
logs = LOAD '/user/hadoop/logs.txt' AS (line:chararray);

-- Split lines into words
words = FOREACH logs GENERATE FLATTEN(TOKENIZE(line)) AS word;

-- Group by word
grouped = GROUP words BY word;

-- Count occurrences
word_count = FOREACH grouped GENERATE group, COUNT(words);

-- Store result back to HDFS
STORE word_count INTO '/user/hadoop/output';
```

Execution Flow on Hadoop:

1. Script is converted into a logical plan.
2. Optimizer combines operations.
3. Physical plan generates MapReduce jobs.
4. Jobs are executed on Hadoop clusters.
5. Output is stored in HDFS at `/user/hadoop/output`.

6. Best Practices for Pig on Hadoop

1. Use partitioned and bucketed tables in HDFS for large datasets.
2. Minimize intermediate data to reduce MapReduce jobs and execution time.
3. Use built-in functions and UDFs efficiently.
4. Run EXPLAIN on Pig scripts to check execution plans before running large jobs.
5. Monitor jobs in YARN Resource Manager to identify bottlenecks.

Use Cases for Apache Pig

Apache Pig is primarily used for large-scale data processing and analytics on Hadoop, especially when dealing with ETL (Extract, Transform, Load), data cleansing, and transformations.

1. ETL (Extract, Transform, Load) Operations

Scenario: Moving raw data from various sources into HDFS, transforming it into a structured format, and loading it into Hive or other systems.

Pig Role: Simplifies data extraction, cleaning, transformation, and loading into target systems.

Example:

Extract log files from servers

Remove invalid records

Aggregate and load cleaned data into Hive tables

Pig Advantage: Easier scripting compared to writing multiple MapReduce jobs.

2. Log Processing and Analysis

Scenario: Analyzing web server logs, application logs, or clickstream data to understand user behavior.

Pig Role:

Parse large logs stored in HDFS

Filter records (e.g., only errors or specific URLs)

Count visits, clicks, or user sessions

Example: Counting page views per user from server logs.

Pig Advantage: Handles huge log files efficiently using Hadoop's distributed storage and processing.

3. Data Cleansing

Scenario: Cleaning messy or inconsistent data before analytics.

Pig Role:

- Remove null or invalid values

- Standardize formats (e.g., dates, strings)

- Deduplicate data

Example: Cleaning customer data before loading into a data warehouse.

Pig Advantage: Can perform complex transformations using Pig Latin scripts with minimal effort.

4. Large-Scale Data Aggregation

Scenario: Aggregating data from multiple sources for reporting or analytics.

Pig Role:

- Group data by key (e.g., region, product)

- Compute sums, averages, counts

Example: Calculating total sales per product category from millions of transactions stored in HDFS.

Pig Advantage: Executes aggregation efficiently over terabytes of data using Hadoop MapReduce.

5. Joining Datasets

Scenario: Combining multiple datasets for richer analysis.

Pig Role:

- Join large datasets using common keys (inner, outer, or skewed joins)

Example: Joining user profiles with transaction data to analyze spending patterns.

Pig Advantage: Handles large-scale joins better than writing custom MapReduce code.

6. Data Transformation for Machine Learning

Scenario: Preparing features for machine learning models from raw Hadoop data.

Pig Role:

- Aggregate, normalize, or encode raw data

- Convert data into structured tables suitable for ML algorithms

Example: Transforming clickstream data into user feature vectors for recommendation systems.

Pig Advantage: Simplifies feature engineering at scale on distributed datasets.

7. Semi-Structured and Unstructured Data Processing

Scenario: Working with JSON, XML, logs, or text files.

Pig Role:

Parse nested or irregular data using Pig's complex data types (tuples, bags, maps)

Example: Extracting user metadata from JSON logs and computing statistics.

Pig Advantage: Handles complex nested structures more easily than Hive.

8. Ad Hoc Data Exploration

Scenario: Quickly analyzing new datasets before creating permanent workflows.

Pig Role:

Run ad hoc queries and transformations on large data

Test logic before implementing production pipelines

Example: Counting distinct users from a new log dataset.

Pig Advantage: Faster prototyping than writing MapReduce code from scratch.

Introduction to ETL

ETL stands for Extract, Transform, and Load, which is a data integration process that collects data from multiple sources, transforms it into a suitable format, and loads it into a target data warehouse or database for analysis.

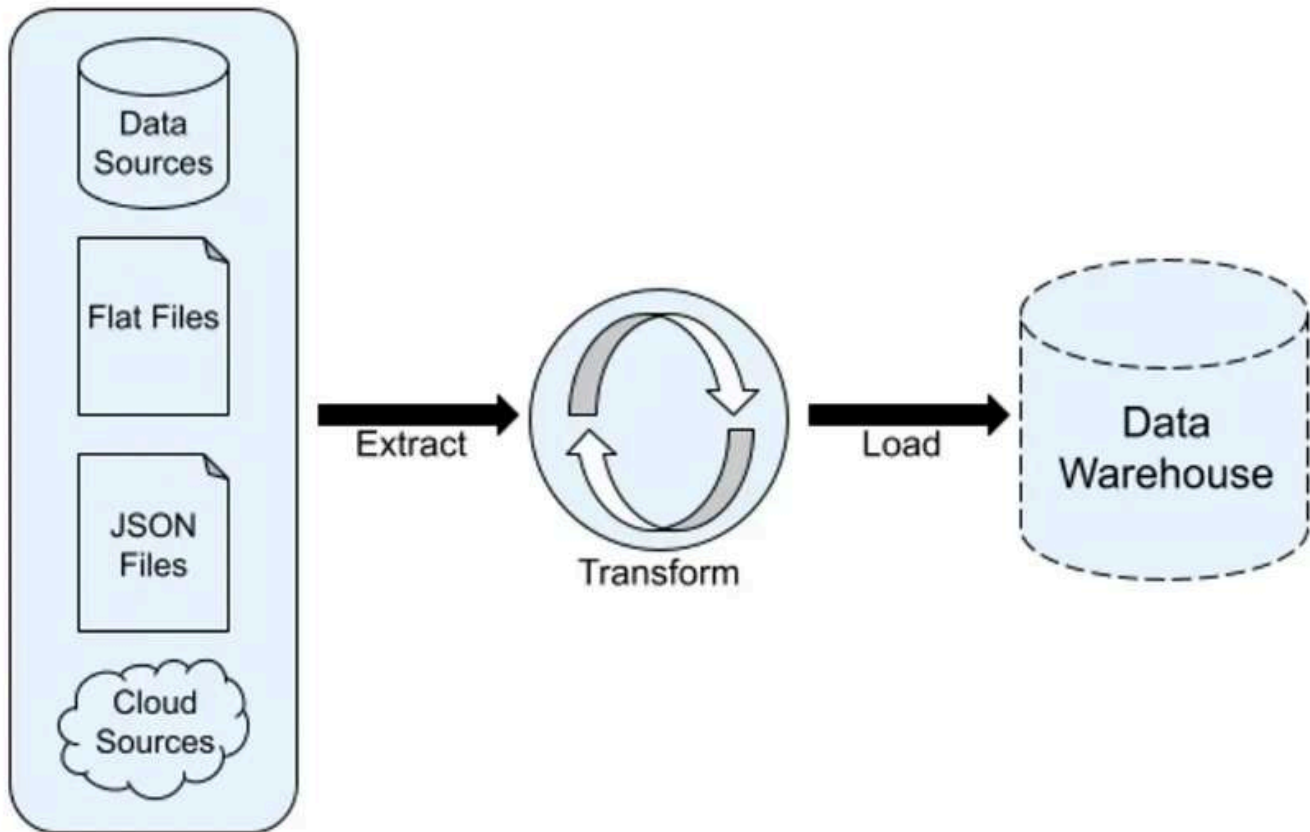
ETL is a core component in Business Intelligence (BI), data warehousing, and big data analytics.

1. ETL Process Overview

The ETL process has three main stages:

1. Extract (E): Pulling data from different source systems.
2. Transform (T): Converting extracted data into a structured, clean, and consistent format.
3. Load (L): Storing the transformed data into a target system, such as a data warehouse or Hadoop HDFS.

ETL Architecture



2. Stage 1: Extract

Objective: Collect data from various sources for further processing.

Key Points:

- Sources can include:
 - Relational databases (RDBMS)
 - Flat files (CSV, TXT)
 - APIs or web services
 - Logs, IoT sensors, social media, or streaming data
- Challenges:
 - Data may be in different formats
 - High data volumes
 - Need for minimal impact on source systems

Techniques for Extraction:

1. Full Extraction: Copy all data from the source.
2. Incremental Extraction: Copy only new or changed data since last extraction.

Tools/Technologies: Apache Sqoop, Talend, Informatica, custom scripts.

3. Stage 2: Transform

Objective: Convert raw extracted data into a consistent, structured, and analyzable format.

Transformations include:

1. Data Cleansing: Remove duplicates, correct errors, handle null values.
2. Data Standardization: Convert formats (dates, currencies, text case).
3. Data Aggregation: Summarize data (sum, count, average).
4. Data Integration: Merge data from multiple sources.
5. Data Enrichment: Add additional derived values or reference data.
6. Data Filtering: Remove irrelevant or unwanted records.
7. Data Sorting/Joining: Order data or combine tables.

Example:

Raw sales data from multiple stores:

```
StoreID, Date, Sales
001, 01-02-2025, 500
002, 01/02/2025, 700
```

•

After transformation:

```
StoreID, Date, Sales
001, 2025-02-01, 500
002, 2025-02-01, 700
```

•

Tools/Technologies: Pig, Hive, Talend, Informatica, Apache Spark, Python scripts.

4. Stage 3: Load

Objective: Store the transformed data into the target system for analytics.

Key Points:

- Target Systems:
 - Data warehouses (e.g., Snowflake, Redshift, Teradata)
 - Hadoop HDFS for big data storage
 - NoSQL databases (HBase, MongoDB)
- Loading Strategies:
 - Full Load: Replace existing data entirely.
 - Incremental Load: Append or update only new/modified records.
- Challenges:
 - Large volumes can affect performance
 - Must ensure data consistency and integrity

Tools/Technologies: Hive, HDFS, Sqoop, Talend, Informatica, Spark.

5. ETL Architecture in Big Data

Big Data ETL involves distributed processing for large datasets.

Components:

1. Data Sources: RDBMS, logs, social media, sensors, cloud storage.
2. ETL Engine / Processing Layer:
 - Batch processing: Hadoop MapReduce, Pig, Hive, Spark
 - Streaming processing: Apache Flink, Spark Streaming, Kafka Streams
3. Data Storage / Target Layer:
 - HDFS, Data Lake, NoSQL, Cloud Storage, Data Warehouse

Architecture Diagram (Text Version):



6. Advantages of ETL

1. Centralized data storage for analytics.
2. Improved data quality through cleansing and standardization.
3. Enables faster and accurate decision-making.
4. Scalable for large datasets in Hadoop or cloud.
5. Supports both batch and real-time data processing.

7. ETL Use Cases

1. Data Warehousing: Consolidating multiple operational systems into a data warehouse.
2. Business Analytics: Preparing data for dashboards, reports, and BI tools.
3. Big Data Analytics: Transforming unstructured or semi-structured data for Hadoop-based analytics.
4. Customer 360: Combining data from multiple sources to analyze customer behavior.
5. Machine Learning Pipelines: Preprocessing data for ML models.

✓ Key Takeaways

- ETL is a three-step process: Extract → Transform → Load.
- Extract pulls data from multiple sources.
- Transform cleanses, aggregates, and standardizes data.
- Load stores data into target systems for analysis.
- In Big Data, tools like Pig, Hive, Spark, and Hadoop HDFS are used for scalable ETL.

Data Types in Pig

Pig supports a flexible nested data model, which allows handling structured, semi-structured, and unstructured data. The data types can be classified as atomic (primitive) or complex types.

A. Atomic (Primitive) Data Types

Data Type	Description	Example
int	32-bit signed integer	<code>age = 25</code>
long	64-bit signed integer	<code>population = 1000000</code>
float	Single-precision floating-point	<code>price = 45.75</code>
double	Double-precision floating-point	<code>temperature = 36.6</code>
chararray	Sequence of characters (String)	<code>name = 'John'</code>
bytearray	Raw binary data	Used when type is unknown or mixed

Notes:

- Pig is strongly typed, but type checking happens at runtime.
- `bytearray` is flexible but requires casting for operations.

B. Complex Data Types

Pig supports nested types to model hierarchical data:

Type	Description	Example
Tuple	Ordered collection of fields (like a row in a table)	<code>(1, 'John', 5000)</code>
Bag	Collection of tuples (like a table or dataset)	<code>{(1, 'John', 5000), (2, 'Alice', 6000)}</code>
Map	Key-value pairs	<code>[name#'John', age#25]</code>

Example of Nested Data Types:

```
-- Bag of Tuples
students = { (1, 'John', 90), (2, 'Alice', 85) }

-- Tuple inside a bag
student_record = (101, 'Bob', {(Math, 90), (Science, 85)})

-- Map example
student_map = [name#'John', age#25, city#'NY']
```


Key Points:

- Bags allow duplicates.
- Tuples are ordered, maps are unordered.
- Pig can handle complex semi-structured data like JSON or XML using these types.

2. Running Pig

Pig can be run in two primary modes:

A. Local Mode

- Runs Pig on a single machine using a local file system.
- Good for testing and small datasets.
- No Hadoop cluster is required.

Command to start Pig in local mode:

```
pig -x local
```

Example Pig Script (Local Mode):

```
-- Load local file
data = LOAD 'input.txt' AS (name:chararray, age:int);

-- Filter age > 25
filtered = FILTER data BY age > 25;

-- Store output locally
STORE filtered INTO 'output.txt';
```

B. MapReduce Mode (Hadoop Mode)

- Runs Pig on Hadoop cluster, using HDFS for input/output.
- Pig scripts are converted into MapReduce jobs executed by Hadoop.
- Suitable for large datasets.

Command to start Pig in Hadoop mode:

```
pig -x mapreduce
```

Example Pig Script (Hadoop Mode):

```
-- Load data from HDFS
data = LOAD '/user/hadoop/input.txt' AS (name:chararray, age:int);

-- Filter users older than 30
filtered = FILTER data BY age > 30;

-- Store results back to HDFS
STORE filtered INTO '/user/hadoop/output';
```

Workflow in Hadoop Mode:

1. Pig parses the script → Logical Plan
2. Optimizes the plan → Physical Plan
3. Converts to MapReduce jobs
4. Executes on Hadoop cluster → Writes results to HDFS

3. Pig Script Execution Commands

Command	Description
LOAD	Reads data from a source (HDFS or local)
STORE	Writes data to target (HDFS or local)
DUMP	Displays output on console (for testing)
FILTER	Filters tuples based on condition
FOREACH...GENERATE	Applies transformations on data
GROUP	Groups data by key
JOIN	Joins two datasets
ORDER BY	Sorts data
LIMIT	Limits number of rows
DESCRIBE	Shows schema of dataset
ILLUSTRATE	Shows step-by-step data flow for debugging

4. Example: Pig Data Types & Script

```
-- Load HDFS file
students = LOAD '/user/hadoop/students.txt'
           AS (id:int, name:chararray, marks:float);

-- Filter students with marks > 50
passed = FILTER students BY marks > 50;

-- Group by marks range
grouped = GROUP passed BY marks;

-- Count students in each group
result = FOREACH grouped GENERATE group, COUNT(passed);

-- Store result to HDFS
STORE result INTO '/user/hadoop/student_result';
```

Explanation:

- `id:int, name:chararray, marks:float` → Primitive types
- `passed` → Bag of Tuples
- `grouped` → Bag grouped by marks

Execution Model of Pig

The execution model of Pig describes how a Pig Latin script is parsed, optimized, converted, and executed to process data. Pig abstracts the complexity of MapReduce, but under the hood, it follows a structured execution flow.

1. Overview

Pig execution can be understood as 6 main stages:

1. Pig Latin Script → User writes a high-level script.
2. Parsing → Syntax check and creation of a Logical Plan.
3. Logical Plan → Abstract representation of operations (LOAD, FILTER, JOIN).
4. Optimization → Logical plan is optimized for better performance.
5. Physical Plan / MapReduce Plan → Logical operations are converted into MapReduce jobs (or Tez/Spark jobs).
6. Execution Engine → Jobs are executed on Hadoop YARN or local mode, results are stored in HDFS or local file system.

2. Detailed Execution Steps

Step 1: Pig Latin Script Layer

- User writes Pig Latin scripts specifying what to do (transformations, aggregations).
- Example:
 - `data = LOAD '/user/hadoop/input.txt' AS (name:chararray, age:int);`
 - `adults = FILTER data BY age >= 18;`
 - `STORE adults INTO '/user/hadoop/output';`
- At this layer, Pig is declarative, focusing on data operations rather than execution mechanics.

Step 2: Parser

- Pig parses the script to check syntax and validate statements.
- Creates a Logical Plan, a Directed Acyclic Graph (DAG) of logical operators.
- Errors like invalid column names or wrong syntax are caught here.

Step 3: Logical Plan

- Represents abstract operations on the data.
- Operations include: LOAD, FILTER, FOREACH, GROUP, JOIN, STORE.
- No physical execution decisions are made at this stage.
- Example:
 - Logical Plan for above script:
 - `LOAD '/input.txt' AS (name, age)`
 - `|`
 - `FILTER age >= 18`
 - `|`
 - `STORE '/output'`

Step 4: Optimizer

- Pig optimizes the logical plan to improve performance.
- Common optimizations:
 1. Projection Pushdown → Only read required columns.
 2. Filter Pushdown → Apply filters as early as possible.
 3. Combine Operations → Merge operations into fewer MapReduce jobs.
 4. Join Reordering → Optimize the order of joins for efficiency.
- Produces an optimized logical plan.

Step 5: Physical Plan / MapReduce Plan

- Converts the optimized logical plan into a physical plan, which maps each operation to MapReduce tasks.
- Determines:
 - How many MapReduce jobs are needed?
 - Data flow between map and reduce tasks.
 - Intermediate storage requirements.

Example:

- LOAD + FILTER → Map-only job
- GROUP + COUNT → MapReduce job

Step 6: Execution Engine

- Executes the physical plan:
 - Local Mode: Runs on a single machine using a local file system.
 - MapReduce Mode: Executes on Hadoop cluster, submits MapReduce jobs via YARN, monitors progress, handles failures.
- Writes results to HDFS or local file system depending on mode.

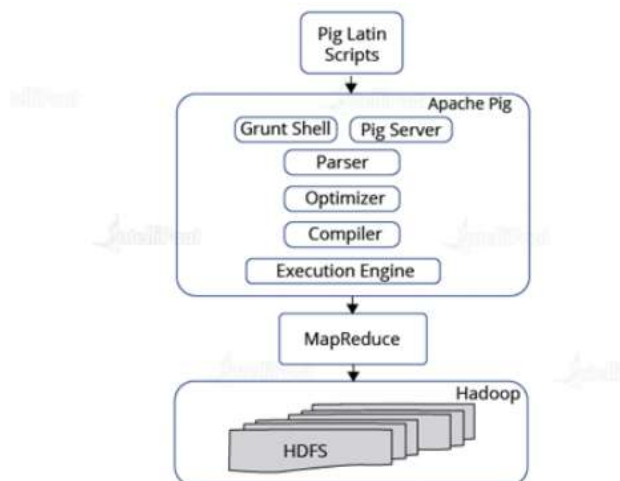
Command to run Pig in Hadoop mode:

- `pig -x mapreduce script.pig`

3. Pig Execution Modes

Mode	Description	Use Case
Local Mode	Runs on a single machine, uses local file system	Testing, small datasets
MapReduce Mode (Hadoop Mode)	Runs on Hadoop cluster, converts Pig Latin scripts to MapReduce jobs	Large-scale datasets

4. Pig Execution Flow Diagram



Operators in Apache Pig

Operators in Pig are commands or functions that perform data processing tasks on relations (datasets).

They are similar to SQL operators but work on bags, tuples, and maps.

Pig operators are mainly divided into three types:

1. Relational Operators
2. Stream Operators
3. Arithmetic/Expression Operators

1. Relational Operators

Relational operators operate on relations (bags of tuples). They are used to manipulate, filter, group, join, and sort data.

Operator	Description	Example
LOAD	Loads data from a source (HDFS or local file)	<code>data = LOAD '/user/hadoop/input.txt' AS (name:chararray, age:int);</code>
STORE	Stores data to HDFS or local file system	<code>STORE data INTO '/user/hadoop/output';</code>
DUMP	Displays data on console (for testing)	<code>DUMP data;</code>
FILTER	Filters rows based on a condition	<code>adults = FILTER data BY age >= 18;</code>
FOREACH...GENERATE	Projects and transforms data	<code>names = FOREACH data GENERATE name;</code>
GROUP	Groups data by key	<code>grouped = GROUP data BY age;</code>
JOIN	Joins two relations on a key	<code>joined = JOIN data1 BY id, data2 BY id;</code>
CROSS	Cartesian product of two relations	<code>crossed = CROSS data1, data2;</code>
DISTINCT	Removes duplicate tuples	<code>unique_data = DISTINCT data;</code>
ORDER BY	Sorts data	<code>sorted = ORDER data BY age DESC;</code>
LIMIT	Limits number of rows	<code>top10 = LIMIT data 10;</code>

2. Stream Operators

Stream operators allow you to run external scripts or programs on each tuple of a relation.

Operator	Description	Example
STREAM	Runs a custom script (Python, Perl, shell) on input	<pre>processed = STREAM data THROUGH 'python script.py';</pre>

Use Case:

- Apply custom transformations that are not built into Pig.
- Useful for advanced data preprocessing or NLP tasks.

3. Arithmetic and Expression Operators

Arithmetic operators work on numeric fields in relation to perform calculations.

Operator	Description	Example
+	Addition	<pre>sum = FOREACH data GENERATE age + 5;</pre>
-	Subtraction	<pre>diff = FOREACH data GENERATE age - 2;</pre>
*	Multiplication	<pre>total = FOREACH data GENERATE age * 2;</pre>
/	Division	<pre>avg = FOREACH data GENERATE age / 2;</pre>
%	Modulus	<pre>mod = FOREACH data GENERATE age % 3;</pre>
<pre>==, !=, >, <, >=, <=</pre>	Comparison	<pre>adults = FILTER data BY age >= 18;</pre>
<pre>AND, OR, NOT</pre>	Logical operators	<pre>valid = FILTER data BY age > 18 AND city == 'NY';</pre>

Notes:

- Arithmetic and logical operators are used inside FILTER and FOREACH...GENERATE statements.

4. Built-in Functions (Complement to Operators)

Pig also has many built-in functions that work with operators:

Function	Description
<code>COUNT()</code>	Counts number of tuples
<code>SUM()</code>	Sum of numeric column
<code>AVG()</code>	Average of numeric column
<code>MAX()</code>	Maximum value
<code>MIN()</code>	Minimum value
<code>TOKENIZE()</code>	Splits a string into words
<code>FLATTEN()</code>	Converts a bag of tuples into individual tuples

Example using operators and functions:

```
data = LOAD '/user/hadoop/students.txt' AS (name:chararray, marks:int);
passed = FILTER data BY marks >= 50;
grouped = GROUP passed BY marks;
result = FOREACH grouped GENERATE group, COUNT(passed);
STORE result INTO '/user/hadoop/output';
```

- `FILTER` → Relational operator
- `GROUP` → Relational operator
- `FOREACH...GENERATE` + `COUNT()` → Projection and aggregation

✓ Key Takeaways

- Operators are the core building blocks of Pig scripts.
- Relational operators manipulate data in bags and tuples.
- Stream operators allow integration with external scripts.
- Arithmetic/Expression operators perform numeric and logical operations.
- Pig operators combined with built-in functions enable complex ETL and data processing tasks.

Functions in Apache Pig

Functions in Pig are used to perform operations on data, such as computing aggregates, transforming values, manipulating strings, or working with complex data types.

Pig provides a rich set of built-in functions (BIFs), and also allows user-defined functions (UDFs) for custom processing.

1. Types of Functions in Pig

1. Built-in Functions (BIFs) – Provided by Pig
2. User-Defined Functions (UDFs) – Written by the user in Java, Python, or other languages
3. Algebraic Functions – Special built-in functions optimized for distributed execution

A. Built-in Functions (BIFs)

These functions are pre-defined and can be used in FOREACH...GENERATE, FILTER, GROUP, and other operations.

1. Aggregate Functions

Function	Description	Example
<code>COUNT()</code>	Counts the number of tuples	<code>COUNT(students)</code>
<code>SUM()</code>	Computes sum of numeric values	<code>SUM(marks)</code>
<code>AVG()</code>	Computes average	<code>AVG(marks)</code>
<code>MIN()</code>	Finds minimum value	<code>MIN(age)</code>
<code>MAX()</code>	Finds maximum value	<code>MAX(marks)</code>

Example:

```
data = LOAD 'students.txt' AS (name:chararray, marks:int);
grouped = GROUP data ALL;
result = FOREACH grouped GENERATE COUNT(data), AVG(data.marks);
```

2. String Functions

Function	Description	Example
<code>CONCAT()</code>	Concatenate strings	<code>CONCAT(first_name, ' ', last_name)</code>
<code>UPPER()</code>	Convert to uppercase	<code>UPPER(name)</code>
<code>LOWER()</code>	Convert to lowercase	<code>LOWER(name)</code>
<code>TRIM()</code>	Remove leading/trailing spaces	<code>TRIM(name)</code>
<code>SUBSTRING()</code>	Extract substring	<code>SUBSTRING(name, 0, 3)</code>
<code>TOKENIZE()</code>	Split string into words	<code>TOKENIZE('Hello World')</code>

3. Mathematical Functions

Function	Description	Example
<code>ABS()</code>	Absolute value	<code>ABS(-5) → 5</code>
<code>ROUND()</code>	Round a number	<code>ROUND(4.6) → 5</code>
<code>FLOOR()</code>	Floor of number	<code>FLOOR(4.6) → 4</code>
<code>CEIL()</code>	Ceiling of number	<code>CEIL(4.3) → 5</code>
<code>RAND()</code>	Random number	<code>RAND()</code>

4. Date Functions

Function	Description	Example
<code>CURRENT_TIME()</code>	Current system time	<code>CURRENT_TIME()</code>
<code>GetDay()</code> , <code>GetMonth()</code> , <code>GetYear()</code>	Extract day/month/year	<code>GetYear(date)</code>

5. Bag and Map Functions

Function	Description	Example
<code>FLATTEN()</code>	Converts a bag of tuples into individual tuples	<code>FOREACH grouped GENERATE FLATTEN(bag_data)</code>
<code>SIZE()</code>	Returns the number of elements in a bag or map	<code>SIZE(bag_data)</code>
<code>TOTUPLE()</code>	Converts a collection into a tuple	<code>TOTUPLE(bag_data)</code>

B. User-Defined Functions (UDFs)

- Purpose: Custom functions for operations not provided by Pig's BIFs.
- Languages Supported: Java, Python (via Jython), JavaScript, Ruby.
- Example (Python UDF):

```
-- Python UDF to square a number
DEFINE square myudfs.Square('square.py');

data = LOAD 'numbers.txt' AS (num:int);
squared = FOREACH data GENERATE square(num);
```

UDFs can be used for:

- Custom aggregations
- Text parsing
- Complex calculations

C. Algebraic Functions

- Algebraic functions can be partially computed and merged across nodes, which improves distributed computation efficiency.
- Example: `SUM()`, `AVG()`, `COUNT()` are algebraic functions.

6. Example Using Pig Functions

Suppose we have a dataset `students.txt`:

```
John,85
Alice,90
Bob,75
```

Pig Script:

```
-- Load data
students = LOAD 'students.txt' AS (name:chararray, marks:int);

-- Convert name to uppercase
upper_names = FOREACH students GENERATE UPPER(name), marks;

-- Filter marks >= 80
top_students = FILTER upper_names BY marks >= 80;

-- Group all data to calculate average
grouped = GROUP top_students ALL;
result = FOREACH grouped GENERATE AVG(top_students.marks),
COUNT(top_students);

-- Store result
STORE result INTO 'output';
```

Explanation:

- `UPPER()` → String function
- `FILTER` → Relational operator
- `AVG()` and `COUNT()` → Aggregate functions

Data Types in Apache Pig

Pig supports a flexible data model that can handle structured, semi-structured, and unstructured data. Data in Pig is organized as relations, which are bags of tuples, and each tuple contains fields of specific types.

Pig data types are divided into:

1. Primitive (Atomic) Data Types
2. Complex Data Types

1. Primitive (Atomic) Data Types

These are basic types for single values.

Data Type	Description	Example
int	32-bit signed integer	<code>age = 25</code>
long	64-bit signed integer	<code>population = 1000000</code>
float	Single-precision floating-point number	<code>price = 45.75</code>
double	Double-precision floating-point number	<code>temperature = 36.6</code>
chararray	Sequence of characters (String)	<code>name = 'John'</code>
bytearray	Raw binary data (unknown or mixed type)	Can store any type, often used before casting

Notes:

- Pig is strongly typed, but type checking is done at runtime.
- `bytearray` is flexible, but operations often require casting to specific types.

2. Complex Data Types

Complex types allow nested structures like bags and maps.

Type	Description	Example
Tuple	Ordered collection of fields (like a row)	<code>(1, 'John', 5000)</code>
Bag	Collection of tuples (like a table or dataset)	<code>{{(1, 'John', 5000), (2, 'Alice', 6000)}}</code>
Map	Key-value pairs, keys are always chararray	<code>[name#' John', age#25]</code>

Example of Nested Types:

```
-- Bag of tuples
students = { (1, 'John', 90), (2, 'Alice', 85) }

-- Tuple inside a bag
student_record = (101, 'Bob', {(Math, 90), (Science, 85)})

-- Map example
student_map = [name#'John', age#25, city#'NY']
```

Key Points:

- Bags allow duplicates, tuples do not.
- Tuples are ordered, maps are unordered.
- Complex types are ideal for semi-structured data, like JSON or XML.

3. Pig Data Type Summary Table

Category	Data Type	Description
Primitive	int	32-bit integer
Primitive	long	64-bit integer
Primitive	float	Single-precision number
Primitive	double	Double-precision number
Primitive	chararray	String
Primitive	bytearray	Raw/unknown type
Complex	Tuple	Ordered collection of fields
Complex	Bag	Collection of tuples
Complex	Map	Key-value pairs

4. Example: Using Data Types in Pig

```
-- Load data with different types
data = LOAD 'students.txt' AS (id:int, name:chararray, marks:float,
info:bytearray);

-- Accessing complex type (bag of tuples)
students_bag = FOREACH data GENERATE id, name, FLATTEN(info);

-- Using map
students_map = FOREACH data GENERATE [name#name, marks#marks];
```

Explanation:

- `id:int`, `name:chararray`, `marks:float` → Primitive types
- `info:bytearray` → Can be converted to tuple or bag using **FLATTEN**
- Map example shows key-value representation

UNIT-4

Introduction to NoSQL

NoSQL stands for “Not Only SQL”. It refers to a class of database management systems that are designed to store, manage, and process large volumes of structured, semi-structured, and unstructured data in a distributed and scalable manner.

Unlike traditional RDBMS (Relational Database Management Systems), which rely on tables, rows, and fixed schemas, NoSQL databases are schema-less or have flexible schemas, enabling them to handle big data, high-velocity data, and unstructured data efficiently.

1. Why NoSQL?

Traditional relational databases face limitations when dealing with:

1. Massive Data Volumes – Petabytes of structured and unstructured data.
2. High-Velocity Data – Real-time streaming data from IoT, social media, or logs.
3. Scalability Issues – RDBMS scale vertically (adding more resources to one server), which becomes expensive.
4. Schema Rigidity – Fixed schemas make frequent data structure changes difficult.

NoSQL databases were designed to overcome these limitations by providing:

- Horizontal scalability (scale out using multiple nodes)
- High availability and fault tolerance
- Flexible schema design
- Efficient handling of large-scale unstructured or semi-structured data

2. Characteristics of NoSQL

1. Schema-less / Flexible Schema – Supports dynamic data models.
2. Distributed Architecture – Data is partitioned across multiple nodes (sharding).
3. High Availability – Supports replication for fault tolerance.
4. Scalability – Can scale horizontally by adding more servers.
5. Eventual Consistency – Some NoSQL systems follow the BASE model (Basically Available, Soft state, Eventually consistent) instead of strict ACID.
6. High Performance – Optimized for fast reads and writes on large datasets.

3. Types of NoSQL Databases

NoSQL databases are generally classified into four major categories:

Type	Description	Example
Key-Value Stores	Store data as key-value pairs, ideal for caching or session management	Redis, DynamoDB, Riak
Document Stores	Store data as JSON, BSON, or XML documents, allows flexible fields	MongoDB, CouchDB
Column-Family Stores	Store data in columns instead of rows, suitable for analytical queries	Cassandra, HBase
Graph Databases	Store data as nodes, edges, and properties to represent relationships	Neo4j, ArangoDB

Brief Comparison:

Aspect	Key-Value	Document	Column-Family	Graph
Data Model	Key → Value	JSON/BSON	Column Families	Nodes & Edges
Query	By Key	Rich queries on document fields	By column	Graph traversal
Use Case	Cache, session store	Web apps, CMS	Big data analytics	Social networks, recommendation engines

4. Advantages of NoSQL

1. Flexible Schema Design – Easily adapt to changing data structures.
2. Horizontal Scalability – Handle increasing loads by adding nodes.
3. High Performance – Fast reads/writes for large datasets.
4. Handles Big Data – Can manage structured, semi-structured, and unstructured data.
5. High Availability – Data replication ensures minimal downtime.

5. Disadvantages / Challenges of NoSQL

1. Eventual Consistency – Some NoSQL databases do not support full ACID transactions.
2. Complexity of Queries – Not as mature as SQL for complex queries and joins.
3. Tooling and Ecosystem – Less mature compared to RDBMS in terms of analytics and reporting tools.
4. Data Redundancy – Replication and denormalization can increase storage requirements.

6. Use Cases of NoSQL

1. Big Data Applications – Social media, IoT, logs, clickstream data.
2. Real-Time Analytics – E-commerce recommendation engines, fraud detection.
3. Content Management Systems (CMS) – Flexible storage for unstructured content.
4. Gaming Applications – Player profiles, session management, leaderboards.
5. Graph-Based Applications – Social networks, knowledge graphs, supply chain management.
6. Caching and Session Storage – High-speed retrieval using key-value stores like Redis.

7. NoSQL vs RDBMS

Feature	RDBMS	NoSQL
Schema	Fixed schema	Flexible / schema-less
Scalability	Vertical scaling	Horizontal scaling
Query Language	SQL	Various (MongoDB query, CQL, Gremlin)
Data Type	Structured	Structured, semi-structured, unstructured
ACID Support	Strong ACID	BASE (Eventual consistency)
Example	MySQL, Oracle	MongoDB, Cassandra, Redis

NoSQL Business Drivers

Organizations are increasingly adopting NoSQL databases because modern applications require flexible, scalable, and high-performance data management solutions. The key business drivers include:

1. Rapid Growth of Data (Big Data)

- Modern businesses generate massive volumes of data from multiple sources: social media, IoT devices, logs, e-commerce transactions, and sensors.
- Traditional RDBMS struggles with scaling and storing such huge data.
- NoSQL databases can handle petabytes of data efficiently through horizontal scaling.

Business Impact: Enables businesses to analyze and monetize large datasets, such as user behavior analysis, recommendation systems, and predictive analytics.

2. Need for High Availability and Reliability

- Modern applications require 24/7 uptime with minimal downtime.
- NoSQL databases provide replication, clustering, and fault tolerance, ensuring high availability.
- Examples: Redis, Cassandra, and MongoDB replicate data across multiple nodes to prevent single points of failure.

Business Impact: Critical for e-commerce, financial services, and online platforms, where downtime can cause revenue loss.

3. Flexible and Evolving Data Structures

- Traditional relational databases require a fixed schema, which makes adapting to changing data structures cumbersome.
- NoSQL supports schema-less or dynamic schemas, allowing businesses to store structured, semi-structured, and unstructured data.

Business Impact: Accelerates product development cycles and supports rapid feature changes in web and mobile apps.

4. Real-Time Data Processing Requirements

- Many applications need real-time insights from incoming data streams: e.g., fraud detection, IoT sensor monitoring, live analytics.
- NoSQL databases like Redis, MongoDB, Cassandra provide low-latency read/write operations and support real-time analytics.

Business Impact: Enables businesses to make instant decisions, improve customer experience, and respond quickly to market changes.

5. Horizontal Scalability to Reduce Costs

- Scaling relational databases vertically (adding CPU, memory) is expensive and limited.
- NoSQL databases scale horizontally, allowing companies to add more commodity servers as demand grows.

Business Impact: Cost-effective scaling for large-scale applications like social media, gaming, and streaming platforms.

6. Support for Multi-Model Data

- Businesses need to work with varied data types: documents, graphs, key-value pairs, or wide columns.
- NoSQL databases support multi-model storage to handle diverse data.

Business Impact: Enables unified data management for analytics, recommendation engines, content management, and social network applications.

7. Enhanced Performance for Specific Use Cases

- Certain applications (like caching, session management, and leaderboards) require extremely fast read/write operations.
- NoSQL databases like Redis and Memcached are optimized for speed and performance in such scenarios.

Business Impact: Improves user experience, reduces latency, and increases application responsiveness.

8. Cloud and Distributed Architecture Adoption

- Cloud-based applications require databases that scale across distributed systems.
- NoSQL is designed for cloud-native deployment, supporting multi-region replication and distributed consistency.

Business Impact: Facilitates global application deployment, disaster recovery, and geographic redundancy.

✓ Summary Table: NoSQL Business Drivers

Driver	Business Benefit
Big Data Handling	Process massive datasets efficiently
High Availability	Minimal downtime, reliable services
Flexible Schema	Rapid feature development and evolving data
Real-Time Processing	Immediate insights for fast decision-making
Horizontal Scalability	Cost-effective growth and scaling
Multi-Model Support	Unified handling of diverse data types
High Performance	Faster reads/writes, better UX
Cloud & Distributed Deployment	Global availability, fault tolerance

NoSQL Data Architectural Patterns

NoSQL databases support various data models and architectural patterns to handle different types of workloads, scalability, and flexibility. Understanding these patterns helps design the right database solution for business needs.

NoSQL data architectures are generally categorized into four primary patterns:

1. Key-Value Pattern

Description

- Data is stored as key-value pairs, similar to a hash table.
- Each key is unique and maps to a value, which can be simple data (string, number) or complex objects (JSON, XML, binary).
- Extremely fast read and write operations.

Characteristics

- Schema-less
- Optimized for lookup by key
- Simple data model

Examples

- Redis, Riak, DynamoDB, Aerospike

Use Cases

- Session management
- Caching
- User preferences storage
- Shopping cart data

2. Document Pattern

Description

- Data is stored as documents (JSON, BSON, XML).
- Each document contains a self-describing data structure, supporting nested fields and arrays.
- Documents are stored in collections instead of tables.

Characteristics

- Flexible schema (each document can have a different structure)
- Supports rich queries on document fields
- Often provides secondary indexes

Examples

- MongoDB, CouchDB, RavenDB

Use Cases

- Content management systems (CMS)
- Product catalogs
- Blogging platforms
- E-commerce applications

3. Column-Family Pattern

Description

- Data is stored in columns rather than rows.
- Columns are grouped into column families, which can have different sets of columns per row.
- Designed for wide tables with sparse data.

Characteristics

- Highly scalable for big data analytics
- Efficient for aggregate queries on large datasets
- Supports distributed storage and replication

Examples

- Apache Cassandra, HBase, ScyllaDB

Use Cases

- Time-series data
- Event logging
- Analytics platforms
- Recommendation engines

4. Graph Pattern

Description

- Data is modeled as nodes, edges, and properties.
- Nodes represent entities (e.g., people, products), edges represent relationships, and properties hold metadata.
- Optimized for relationship-heavy queries.

Characteristics

- Schema-less or flexible schema
- Supports graph traversals
- Ideal for complex relationships

Examples

- Neo4j, ArangoDB, JanusGraph

Use Cases

- Social networks (friends, followers)
- Recommendation engines
- Fraud detection
- Knowledge graphs

5. Multi-Model Pattern (Optional / Hybrid)

Description

- Supports multiple data models within the same database (e.g., document + graph).
- Useful when applications require different access patterns on the same data.

Examples

- ArangoDB (Document + Graph)
- OrientDB (Document + Graph)

Use Cases

- Applications with heterogeneous data requirements
- Combining analytics and relationship queries

Comparison of NoSQL Patterns

Pattern	Data Model	Query Type	Use Case
Key-Value	Key → Value	By Key	Caching, session management
Document	JSON/BSON	Rich queries	CMS, e-commerce
Column-Family	Columns & Column Families	Aggregations	Analytics, logs
Graph	Nodes & Edges	Traversals	Social networks, recommendations
Multi-Model	Mixed	Mixed	Complex applications with diverse requirements

Variations of NoSQL Architectural Patterns

NoSQL databases come in different architectural patterns that are designed to handle Big Data challenges such as volume, velocity, variety, and veracity. Each pattern offers unique advantages for storing, processing, and analyzing large datasets.

The main variations are:

1. Key-Value Store Architecture

Overview

- Stores data as key-value pairs.
- Each key is unique, and the value can be simple or complex (JSON, binary).
- Extremely fast for lookups by key.

Big Data Use

- Ideal for high-throughput applications where quick access to specific data is required.
- Commonly used in caching, session storage, or temporary data.

Example NoSQL DBs

- Redis, DynamoDB, Riak

Big Data Scenario:

- Storing user sessions for millions of users in real-time.
- Handling IoT sensor readings for rapid ingestion and retrieval.

2. Document-Oriented Architecture

Overview

- Stores documents (JSON, BSON, XML) in collections.
- Documents are self-describing, allowing dynamic and nested data structures.

Big Data Use

- Useful for semi-structured or unstructured data in Big Data pipelines.
- Supports rich queries, indexing, and aggregation.

Example NoSQL DBs

- MongoDB, CouchDB, Couchbase

Big Data Scenario:

- Storing social media posts, comments, and metadata for analysis.
- Handling content management systems with heterogeneous content formats.

3. Column-Family Architecture

Overview

- Data is stored in columns grouped into families instead of traditional rows.
- Each row can have different columns, allowing sparse data storage.
- Optimized for read/write efficiency on large datasets.

Big Data Use

- Handles time-series, log data, and analytics workloads efficiently.
- Excellent for high-volume ingestion and aggregation queries.

Example NoSQL DBs

- Apache Cassandra, HBase, ScyllaDB

Big Data Scenario:

- Storing web server logs with billions of entries per day.
- Running analytics queries on large-scale datasets for trend analysis.

4. Graph-Based Architecture

Overview

- Data is modeled as nodes, edges, and properties.
- Ideal for representing relationships between entities.

Big Data Use

- Analyzes complex relationships in large datasets.
- Supports graph traversal and connected data analytics.

Example NoSQL DBs

- Neo4j, ArangoDB, JanusGraph

Big Data Scenario:

- Detecting fraud patterns in financial transactions.
- Building social networks or recommendation engines.

5. Multi-Model Architecture

Overview

- Combines two or more NoSQL patterns in a single database.
- Supports document, graph, and key-value data models simultaneously.

Big Data Use

- Allows flexible and unified storage of heterogeneous Big Data.
- Supports applications requiring diverse data access patterns.

Example NoSQL DBs

- ArangoDB, OrientDB

Big Data Scenario:

- Social media analytics combining user profiles (document) and friendship graphs (graph).
- Real-time recommendation systems using hybrid data models.

How NoSQL Patterns Help Manage Big Data

Pattern	Big Data Capability	Example
Key-Value	High-speed ingestion and retrieval	IoT sensor data
Document	Flexible schema, semi-structured data	Social media posts
Column-Family	Scalable analytics, sparse data storage	Log aggregation, metrics
Graph	Complex relationships and traversals	Fraud detection, recommendations
Multi-Model	Unified storage for heterogeneous data	Social networks, multi-source analytics

Introduction to MongoDB

MongoDB is a popular NoSQL, open-source, document-oriented database that is designed to handle large volumes of unstructured, semi-structured, and structured data efficiently. Unlike traditional relational databases, MongoDB uses a flexible schema, allowing developers to store data in JSON-like documents (BSON).

The name MongoDB comes from the word “humongous”, reflecting its ability to handle huge amounts of data.

1. Key Features of MongoDB

1. Document-Oriented Storage
 - Data is stored in documents instead of rows and columns.
 - Each document is a BSON (Binary JSON) object, which can contain nested fields, arrays, and key-value pairs.
2. Schema Flexibility
 - MongoDB allows dynamic schemas, meaning each document in a collection can have a different structure.
 - Useful for applications with rapidly evolving requirements.
3. High Performance
 - Supports fast read and write operations.
 - Optimized for handling large datasets in real-time applications.
4. Scalability
 - Supports horizontal scaling through sharding (distributing data across multiple servers).
 - Easily handles high-volume and high-velocity data.
5. Indexing and Querying
 - Supports secondary indexes, compound indexes, text search, and geospatial indexes.
 - Provides a rich query language for filtering, aggregating, and transforming data.

6. Replication and High Availability
 - Uses replica sets to provide data redundancy and automatic failover.
 - Ensures high availability in case of server failures.
7. Aggregation Framework
 - Provides a powerful pipeline-based aggregation framework for analyzing and transforming data.

2. MongoDB vs Traditional RDBMS

Feature	MongoDB	RDBMS
Data Model	Document (JSON/BSON)	Table (Rows & Columns)
Schema	Dynamic / Flexible	Fixed / Rigid
Query Language	MongoDB Query Language (MQL)	SQL
Scalability	Horizontal (Sharding)	Vertical (Scaling Up)
Relationships	Embedded documents / References	Joins
Transactions	Multi-document ACID (v4+)	ACID transactions
Use Case	Big Data, Web Apps, IoT	OLTP, structured data

3. MongoDB Architecture Overview

1. Database – A container for collections.
2. Collection – Similar to a table in RDBMS, stores documents.
3. Document – A single record (BSON object).
4. Field – Key-value pair inside a document.
5. Index – Improves query performance.

Example: Document Structure

```
{
  "_id": ObjectId("651234abcd1234abcd567890"),
  "name": "John Doe",
  "age": 28,
  "email": "john@example.com",
  "skills": ["Java", "MongoDB", "Python"],
  "address": {
    "street": "123 Main St",
    "city": "New York",
    "zip": "10001"
  }
}
```

- **_id** → Primary key (automatically generated if not provided)
- **skills** → Array field
- **address** → Embedded document

4. Advantages of MongoDB

1. Handles large-scale, unstructured data efficiently.
2. Flexible schema enables rapid development.
3. High performance for read/write-heavy workloads.
4. Supports horizontal scaling and distributed systems.
5. Powerful querying and aggregation capabilities.
6. Built-in replication and fault tolerance.

5. Use Cases of MongoDB

- Web and Mobile Applications – Flexible schema for dynamic user data.
- Content Management Systems (CMS) – Stores semi-structured content.
- Big Data Analytics – Aggregates and analyzes large datasets.
- IoT Applications – Stores high-volume sensor data.
- Real-Time Analytics & Logging – Event logs and monitoring data.

UNIT-5

☀ Mining Social Network Graphs:

Introduction to Social Network Mining

Social Network Mining (SNM) refers to the process of analyzing social networks to extract useful patterns, insights, and knowledge from the interactions between individuals, groups, or organizations.

A social network is a structure made up of nodes (individuals or entities) and edges (relationships or interactions). Mining these networks helps us understand behavior, influence, and information flow within communities.

Social Network Mining combines techniques from:

- Graph theory – to model relationships.
- Data mining & machine learning – to find patterns and predict behavior.
- Statistics & network analysis – to measure centrality, clusters, and connectivity.

Key Concepts

1. Node – Represents an individual, organization, or entity.
2. Edge – Represents a relationship, friendship, interaction, or communication between nodes.
3. Degree – Number of edges connected to a node.
4. Centrality Measures – Identify important nodes (influencers).
5. Community Detection – Finding clusters or groups within the network.
6. Network Metrics – Density, betweenness, closeness, and eigenvector centrality.

Applications of Social Network Mining

Social Network Mining has diverse applications across industries due to the rise of social media, online communities, and digital interactions.

1. Marketing and Customer Insights

- Influencer Detection – Identify users with high influence to target for marketing campaigns.
- Recommendation Systems – Suggest products or content based on social connections and behavior.
- Customer Segmentation – Analyze communities to segment users by preferences or interests.

Example:

- Brands use social network analysis to detect key influencers on Twitter or Instagram for advertising campaigns.

2. Fraud Detection and Security

- Detect fraudulent behavior in financial networks, insurance claims, or online transactions.
- Analyze suspicious communication patterns in social and communication networks.

Example:

- Banks analyze money transfer networks to detect potential fraud rings.

3. Healthcare and Epidemiology

- Track disease spread through social interactions and contact networks.
- Identify super-spreaders and high-risk communities for intervention.

Example:

- Social network analysis helped track COVID-19 contact networks and optimize vaccination strategies.

4. Academic and Research Analysis

- Analyze collaboration networks among researchers, authors, or institutions.
- Identify key contributors and emerging research areas.

Example:

- Mapping co-authorship networks to identify influential researchers in AI.

5. Social Media Analytics

- Study trending topics, sentiment, and public opinion.
- Detects communities, interest groups, and influencers.
- Predict virality of content.

Example:

- Facebook and Twitter analytics use SNM to recommend friends, pages, or trending topics.

6. Organizational Network Analysis (ONA)

- Examine communication patterns within an organization.
- Identify knowledge hubs, collaboration bottlenecks, and key employees.

Example:

- Companies analyze email or Slack networks to improve workflow and collaboration efficiency.

7. Political and Social Studies

- Analyze social movements, voting patterns, and public opinion networks.
- Understand information propagation and influence in communities.

Example:

- Study how misinformation spreads on social media networks.

8. Recommender Systems

- Social connections and behaviors are used to recommend friends, products, movies, or music.

Example:

- Netflix or Spotify recommends content based on friends' preferences and community trends.

9. Law Enforcement and Crime Analysis

- Map criminal networks to understand relationships and hierarchy.
- Identify key suspects and criminal groups.

Example:

- Police use social network analysis to track gang activities and detect organized crime networks.

Benefits of Social Network Mining

1. Identify key influencers and decision-makers.
2. Predict trends and behaviors based on network patterns.
3. Detect fraud and abnormal behavior early.
4. Optimize marketing, advertising, and recommendation systems.
5. Improve collaboration and knowledge flow within organizations.

✓ Key Takeaways

- Social Network Mining is about extracting meaningful insights from networks of relationships.
- Combines graph theory, data mining, and machine learning.
- Applications span marketing, healthcare, fraud detection, social media, organizational analysis, political science, and crime analysis.
- Essential in the Big Data era, where social interactions generate massive, complex datasets.

Social Networks as a Graph

A social network is a structure representing relationships and interactions among entities such as individuals, organizations, or groups. To analyze these networks systematically, they are often represented as graphs in graph theory.

A graph is a mathematical representation consisting of:

- Nodes (Vertices) – Represent entities (people, organizations, or objects).
- Edges (Links) – Represent relationships, interactions, or connections between nodes.

Representing social networks as graphs allows us to analyze structure, patterns, and dynamics using mathematical and computational techniques.

1. Graph Representation in Social Networks

Basic Elements

Element	Description
Node (Vertex)	Represents an individual, user, organization, or entity in the network.
Edge (Link)	Represents a relationship or interaction (friendship, following, message exchange).
Directed Edge	A one-way relationship, e.g., Twitter follower.
Undirected Edge	A mutual relationship, e.g., Facebook friendship.
Weighted Edge	Edge with a value representing strength or frequency of interaction.
Unweighted Edge	Simple connection with no weight.

Graph Types

1. Directed Graph (Digraph)
 - Edges have a direction ($A \rightarrow B$).
 - Example: Twitter following network.
2. Undirected Graph
 - Edges have no direction ($A - B$).
 - Example: Facebook friendship network.
3. Weighted Graph
 - Edges carry a weight representing interaction strength or frequency.
 - Example: Number of messages exchanged between two users.
4. Unweighted Graph
 - All edges are considered equal; only presence or absence of connection matters.
5. Bipartite Graph
 - Nodes are divided into two disjoint sets, edges connect nodes from different sets.
 - Example: Users and movies in a recommendation system.

2. Graph Properties in Social Networks

Property	Description	Example
Degree	Number of edges connected to a node	Number of friends a user has
In-degree / Out-degree	Number of incoming/outgoing edges in directed graphs	Number of followers / followings
Path	Sequence of edges connecting nodes	Connection path between two users
Distance	Shortest path between two nodes	Minimum hops to reach a friend
Centrality	Measures node importance	Influencers, hubs
Betweenness Centrality	Node lies on many shortest paths	Gatekeeper or mediator in the network
Clustering Coefficient	Degree to which nodes cluster together	Tight-knit communities or friend groups
Community / Cluster	Group of nodes densely connected internally	Fans of a sports team or hobby group
Density	Ratio of actual edges to possible edges	Measures connectivity level in the network

3. Mathematical Representation

- Graph $G = (V, E)$
 - $V \rightarrow$ Set of vertices (nodes)
 - $E \rightarrow$ Set of edges (connections)

Example:

Let a social network have 3 users: Alice, Bob, and Carol.

- Alice and Bob are friends.
- Bob follows Carol.

Graph Representation:

- Nodes (V) = {Alice, Bob, Carol}
- Edges (E) = {(Alice, Bob), (Bob $\rightarrow$ Carol)}

4. Graph-Based Analysis in Social Networks

1. Node-Level Analysis
 - Focus on individual users.
 - Measures: degree, centrality, influence.

2. Edge-Level Analysis
 - Study relationships between users.
 - Example: frequency, strength, reciprocity of interactions.
3. Group-Level (Community) Analysis
 - Identify clusters or communities of closely connected users.
 - Algorithms: modularity, Girvan–Newman, Louvain method.
4. Network-Level Analysis
 - Examine overall network structure.
 - Metrics: density, average path length, diameter, connectivity.

5. Applications of Representing Social Networks as Graphs

1. Influencer Detection – Identify key nodes with high centrality.
2. Community Detection – Find interest groups, fan bases, or clusters.
3. Recommendation Systems – Suggest friends, content, or products based on network structure.
4. Information Diffusion – Model how news, trends, or viral content spread.
5. Fraud Detection – Detects unusual patterns in transaction or communication networks.
6. Network Optimization – Improve connectivity and resource distribution.

6. Advantages of Graph Representation

- Captures complex relationships naturally.
- Supports quantitative analysis with graph theory.
- Enables visualization of social networks for insights.
- Works well with graph algorithms like shortest path, clustering, and influence maximization.

✓ Key Takeaways

- Social networks are effectively modeled as graphs, with nodes representing entities and edges representing relationships.
- Different graph types (directed, undirected, weighted, bipartite) model different kinds of social interactions.
- Graph metrics and algorithms allow detailed analysis at node, edge, group, and network levels.
- This representation is critical for marketing, recommendations, fraud detection, community analysis, and information propagation studies.

Types of Social Networks

Social networks are platforms or structures that allow individuals, groups, or organizations to interact, share information, and form relationships. They can be classified based on purpose, nature of interactions, and content shared.

1. General Social Networks

Overview

- Platforms for connecting people globally.
- Focus on friendship, communication, and content sharing.

Key Features

- Profile creation, messaging, posting updates, sharing media.
- Connects people based on friendship, interests, or location.

Examples

- Facebook, Google+, VKontakte

Use Cases

- Keeping in touch with friends and family.
- Socializing and sharing life updates.

2. Professional Networks

Overview

- Focus on career development, networking, and business connections.

Key Features

- Professional profiles, endorsements, job postings, and business updates.
- Connects users based on skills, industry, or professional interest.

Examples

- LinkedIn, Xing

Use Cases

- Job hunting and recruitment.
- Professional networking and collaboration.
- Industry insights and knowledge sharing.

3. Microblogging Networks

Overview

- Platforms for posting short updates or micro-blogs.

Key Features

- Text-limited posts, multimedia content, hashtags, retweets/reposts.
- Real-time sharing of news, thoughts, and trends.

Examples

- Twitter (X), Tumblr, Weibo

Use Cases

- Real-time news sharing.
- Influencer communication and marketing campaigns.
- Trend tracking and sentiment analysis.

4. Media Sharing Networks

Overview

- Focused on sharing multimedia content like photos, videos, and audio.

Key Features

- Uploading, commenting, liking, and sharing media.
- Allows community formation around shared content.

Examples

- Instagram, YouTube, TikTok, Snapchat

Use Cases

- Brand marketing through visual content.
- Creative expression and user-generated content.
- Video tutorials, entertainment, and influencer campaigns.

5. Interest-Based Networks (Community Networks)

Overview

- Platforms built around specific interests, hobbies, or topics.

Key Features

- Discussion forums, groups, Q&A, content sharing.
- Connects people with shared passions.

Examples

- Reddit, Quora, Stack Overflow, Goodreads

Use Cases

- Knowledge sharing and discussion.
- Community building around hobbies or professional interests.
- Crowdsourced solutions to problems.

6. Blogging and Publishing Networks

Overview

- Focused on long-form content creation and publishing.

Key Features

- Articles, blogs, newsletters, comments, and subscriptions.
- Users can follow authors or topics.

Examples

- Medium, WordPress, Blogger

Use Cases

- Personal or professional blogging.
- Content marketing and thought leadership.
- Knowledge sharing and storytelling.

7. Social Bookmarking Networks

Overview

- Platforms for sharing, organizing, and discovering web content.

Key Features

- Users can save links, tag them, and share with communities.
- Content discovery through voting, tagging, and recommendations.

Examples

- Pinterest, Digg, Delicious

Use Cases

- Collaborative research and idea sharing.
- Marketing and traffic generation through content sharing.
- Curating visual content or resources.

8. Review and Rating Networks

Overview

- Platforms that allow users to review and rate products, services, or experiences.

Key Features

- User reviews, ratings, comments, recommendations.
- Helps in decision-making for potential buyers.

Examples

- TripAdvisor, Yelp, Glassdoor

Use Cases

- Influencing consumer decisions.
- Business feedback and reputation management.
- Social proof for products and services.

9. Anonymous / Ephemeral Networks

Overview

- Platforms where users can share content anonymously or temporarily.

Key Features

- Short-lived posts, anonymity, privacy-focused interactions.
- Content disappears after a set time or remains anonymous.

Examples

- Snapchat, Yik Yak, Secret

Use Cases

- Sharing private thoughts or confessions.
- Temporary promotions or campaigns.
- Enhancing user privacy and candid interactions.

10. Hybrid / Multi-Purpose Networks

Overview

- Combines features from different types of networks.

Key Features

- Integration of messaging, media sharing, blogging, and communities.

Examples

- Facebook (general + media sharing + groups)
- WeChat (messaging + payments + social features)

Use Cases

- Comprehensive social interaction platform.
- E-commerce integration and digital ecosystem building.

Clustering of Social Graphs

Clustering in social network analysis refers to the process of grouping nodes in a social graph such that nodes within the same cluster (community) are densely connected to each other, while connections between different clusters are sparser.

It is also known as community detection.

1. Why Clustering is Important

- Social networks often contain naturally occurring communities (friend groups, interest groups, professional circles).
- Clustering helps in:
 - Identifying influential groups
 - Recommending friends, content, or products
 - Detecting anomalies or fraud
 - Understanding network structure and dynamics

2. Graph Representation

- Social networks are modeled as graphs $G = (V, E)$:
 - $V \rightarrow$ set of nodes (individuals)
 - $E \rightarrow$ set of edges (relationships)
- Clustering looks for subsets of nodes $C \subseteq V$ such that nodes in C are highly interconnected.

3. Clustering Methods

a) Hierarchical Clustering

- Builds a tree of clusters (dendrogram).
- Two approaches:
 1. Agglomerative: Start with individual nodes $\rightarrow$ merge based on similarity.
 2. Divisive: Start with the entire graph $\rightarrow$ split into smaller clusters.

Pros: Good for visualizing nested communities

Cons: Computationally expensive for large networks

b) Partitioning Methods

- Divide the network into k clusters optimizing a specific criterion.
- Example: k-means clustering applied to graph embeddings.

Pros: Simple and widely used

Cons: Requires predefined number of clusters

c) Modularity-Based Clustering

- Modularity measures the strength of division of a network into clusters.
- Algorithms find partitions that maximize modularity.

Popular Algorithms:

- Girvan–Newman Algorithm
- Louvain Method

Steps of Louvain Method:

1. Assign each node to its own community.
2. Iteratively move nodes to neighboring communities to maximize modularity gain.
3. Aggregate nodes in the same community into a single node and repeat.

Pros: Efficient for large networks

Cons: May produce uneven community sizes

d) Spectral Clustering

- Uses eigenvalues and eigenvectors of the graph Laplacian.
- Nodes are embedded into a lower-dimensional space → traditional clustering (k-means) applied.

Pros: Mathematically rigorous

Cons: Computationally heavy for very large graphs

e) Overlapping Community Detection

- Real-world social networks often have nodes belonging to multiple communities.
- Algorithms like Clique Percolation Method (CPM) detect overlapping communities.

4. Direct Discovery of Communities

Direct Discovery refers to finding communities without predefining clusters or labels.

Steps:

1. Graph Representation
 - Model the social network as nodes (users) and edges (connections).
2. Measure Node Similarity or Edge Density
 - Metrics: Jaccard similarity, common neighbors, edge weight, etc.
3. Detect Dense Subgraphs
 - Find subgraphs with higher internal connectivity compared to external connectivity.
4. Refine Communities
 - Remove weak connections or merge small clusters.
5. Validate Communities
 - Metrics: Modularity, conductance, clustering coefficient.

Algorithms for Direct Discovery:

Algorithm	Approach	Notes
Girvan–Newman	Edge betweenness	Removes edges with high betweenness to separate communities
Louvain	Modularity maximization	Efficient for large-scale networks
Infomap	Information-theoretic	Uses random walks to detect community flow
Label Propagation	Iterative node labeling	Fast and scalable, allows overlapping communities

5. Applications of Community Detection in Social Graphs

1. Marketing and Influencer Targeting – Identify communities for targeted campaigns.
2. Recommendation Systems – Recommend friends, content, or products within communities.
3. Fraud and Anomaly Detection – Detect suspicious clusters or abnormal connections.
4. Social Network Analysis – Understand network structure, hierarchy, and dynamics.
5. Content Personalization – Provide relevant content to a community based on preferences.

6. Key Metrics to Evaluate Clusters

Metric	Purpose
Modularity	Measures density of edges within clusters vs between clusters
Conductance	Measures the ratio of cut edges to total edges in a cluster
Clustering Coefficient	Measures how close neighbors of a node are to being a clique
Edge Density	Fraction of possible edges that exist within a cluster

✓ Key Takeaways

- Social graph clustering = grouping nodes into communities based on connectivity.
- Direct community discovery identifies communities without prior knowledge of cluster numbers.
- Methods: Hierarchical, modularity-based, spectral, label propagation, overlapping community detection.
- Applications: Marketing, recommendations, fraud detection, social analysis, content personalization.
- Metrics: Modularity, conductance, clustering coefficient, edge density.

Introduction to Recommender Systems

A Recommender System (RS) is a type of information filtering system that predicts and suggests items of interest to users based on their preferences, behaviors, and interactions. Recommender systems have become an essential part of e-commerce, streaming services, social media, and online platforms because they help users discover relevant content from vast amounts of information.

1. Importance of Recommender Systems

- Information Overload Problem: With millions of products, videos, articles, and social connections available online, it becomes difficult for users to choose relevant items.
- Personalization: Recommender systems provide customized experiences for users, increasing engagement and satisfaction.
- Business Benefits: They help increase sales, user retention, content consumption, and improve the overall user experience.

2. Types of Recommender Systems

Recommender systems are mainly categorized into three types:

A. Collaborative Filtering (CF)

- Idea: Recommendations are made based on the preferences of similar users or items.
- Approach: Leverages historical interactions, like ratings, clicks, or purchases.

Subtypes:

1. User-based CF:
 - Finds users similar to the target user and recommends items they liked.
 - Example: "Users who liked this movie also liked..."
2. Item-based CF:
 - Finds items similar to the ones the target user liked and recommends them.
 - Example: "Customers who bought this item also bought..."

Pros: Works well with implicit feedback.

Cons: Suffers from cold-start problems (new users or items).

B. Content-Based Filtering (CBF)

- Idea: Recommends items similar to those the user liked in the past based on item attributes.
- Approach: Uses metadata or features of items (genres, keywords, categories).

Example:

- A user watches sci-fi movies → the system recommends other sci-fi movies.

Pros: Handles new items better than CF.

Cons: Requires detailed item feature information and may over-specialize.

C. Hybrid Recommender Systems

- Idea: Combines collaborative filtering and content-based filtering to leverage the strengths of both.
- Approach: Can combine in parallel or sequentially, or use a weighted score.

Example:

- Netflix uses a hybrid approach: combines viewing history (CF) and movie metadata (CBF) for recommendations.

Pros: Reduces cold-start problems and improves accuracy.

Cons: More complex to implement.

3. Key Components of a Recommender System

1. User Profile:
 - Stores user preferences, history, and behavior.
2. Item Profile:
 - Stores item attributes like category, genre, description, price, etc.

3. Interaction Data:

- Ratings, clicks, purchases, likes, views, or any feedback.

4. Recommendation Algorithm:

- Collaborative filtering, content-based, hybrid, or advanced methods (e.g., deep learning).

5. Prediction Engine:

- Scores and ranks items for a particular user.

6. Evaluation Module:

- Measures accuracy, precision, recall, and user satisfaction.

4. Advanced Techniques

1. Matrix Factorization – Reduces user-item interaction matrix to latent factors (e.g., SVD).
2. Deep Learning – Uses neural networks to model complex user-item interactions.
3. Association Rule Mining – Detects co-occurrence patterns in user behavior.
4. Graph-Based Methods – Uses social networks or item-item graphs to propagate recommendations.
5. Context-Aware Recommenders – Considers time, location, or mood to improve recommendations.

5. Applications of Recommender Systems

Domain	Example	Benefit
E-commerce	Amazon product recommendations	Increased sales and cross-selling
Streaming Services	Netflix, YouTube suggestions	Enhanced user engagement
Social Media	Friend suggestions on Facebook, Twitter	Increased network connectivity
Online Ads	Personalized ad recommendations	Higher click-through rate
Music & Podcasts	Spotify playlists, podcast suggestions	Better content discovery
News & Articles	Medium or Google News personalized feed	Relevant content delivery
E-learning	Course recommendations on Coursera, Khan Academy	Personalized learning paths

6. Challenges in Recommender Systems

1. Cold-Start Problem – Difficulty recommending for new users or items.
2. Sparsity – Many users rate only a few items, making the interaction matrix sparse.
3. Scalability – Handling millions of users and items efficiently.
4. Diversity vs Accuracy – Balancing popular items vs niche recommendations.
5. Privacy Concerns – Collecting user data may raise privacy issues.
6. Real-Time Recommendations – Generating recommendations instantly as user behavior changes.

7. Evaluation Metrics

- Accuracy Metrics: RMSE (Root Mean Square Error), MAE (Mean Absolute Error)
- Ranking Metrics: Precision@k, Recall@k, F1-score
- Coverage: Fraction of items or users for which recommendations can be made
- Diversity & Novelty: Measures how diverse or new the recommendations are

8. Summary

- Recommender Systems are tools to filter information and suggest relevant items to users.
- They are essential for personalization in modern digital platforms.
- Types include Collaborative Filtering, Content-Based, and Hybrid approaches.
- Advanced methods use matrix factorization, deep learning, graph-based models, and context-aware systems.
- Applications span e-commerce, media, social networks, ads, music, and e-learning.
- Challenges include cold-start, sparsity, scalability, privacy, and real-time processing.